

Curated File Guide: server.mjs

The local backend that saves drafts, runs compilers, checks GitHub publishing access, imports/pushes site files, handles updates, and relays AI.

Before The Code: File Overview

Abstract

server.mjs is the local nervous system of the OMB Portfolio Builder. It is not the public website and it is not the Electron shell. It is the private backend that runs on the owner's machine and gives the builder the powers a browser page should not have by itself: reading and writing local files, saving drafts, accepting uploads, running compilers, using Git, checking publishing authorization, importing a target site, launching installers, and relaying AI requests. The file is intentionally broad because the builder is local-first. Instead of depending on a hosted database and a remote application server, the desktop app starts this Node server beside the user-owned workspace and talks to it through local API endpoints.

The most important idea is boundary control. The frontend can draw buttons and collect input, but it cannot safely decide where files are written, which Git repository receives a push, whether a command should run, or whether a request came from the local computer. server.mjs owns those decisions. It is the layer that turns a UI action into a guarded filesystem, compiler, Git, installer, or AI operation.

Introduction

When the builder starts, Electron loads a local page. That page needs catalog data, templates, upload handling, compile actions, publishing actions, and update checks. A static HTML page cannot perform those operations directly without either exposing dangerous privileges or running into browser security walls. server.mjs solves that by becoming a small local application server. It serves the builder files, exposes API endpoints under /api, and keeps privileged work in Node where the filesystem, process runner, and Git commands are available.

A useful mental model is to treat this file like a workshop manager. The frontend is the workbench surface where the user sees projects, editors, buttons, and dialogs. server.mjs is the person behind the counter who knows where the tools are, where files belong, whether a customer has permission to ship something, and whether the requested operation is safe. If the frontend says 'save this PDF under this project,' server.mjs sanitizes the path and writes it under the correct docs folder. If the frontend says 'compile this C++ program,' server.mjs finds g++, creates a temporary run folder, captures terminal output, and returns a result object. If the frontend says 'apply to site,' server.mjs checks Git authorization first, then synchronizes publishable files and pushes them.

Background Information

The file uses built-in Node modules rather than a large web framework. createServer from node:http accepts requests, node:fs/promises handles files, node:child_process runs compilers and Git, node:path normalizes paths, node:crypto hashes authorization cache scopes, and node:os identifies the current machine. This keeps the backend predictable and package-light. The tradeoff is that routing and request handling are written manually. That is why handleApi is large: it plays the role that Express or another framework would normally play.

There are three important roots. root is the builder workspace that contains the site files and local draft catalog. portfolioRoot is the publish mirror, which may be separate from root so generated website files can be pushed without mixing in builder-only files. compileRoot is where project-local code workspaces and temporary compiler run folders live. Most path-related functions exist to make sure a request stays inside one of these roots.

The backend is also intentionally local-sensitive. Some endpoints are read-only, but write endpoints reject non-local callers. That is why functions such as `isLocalRequest` matter: the builder may be opened in a browser or accessed on a LAN during testing, but operations that change files, compile code, install tools, or publish should be performed only from the owner's computer.

Purpose Of The File

The purpose of `server.mjs` is to make the builder operational instead of decorative. Without it, `template-preview.html` and `template-preview.js` would still be able to render a screen, but they would not be able to reliably save drafts, upload project evidence, run compilers, authenticate GitHub targets, import an existing website, launch updates, or call private AI providers. The file therefore holds the application's real side effects.

The file is also a protection layer. It validates filenames, normalizes repository URLs, keeps publish authentication scoped to a machine and target, prevents stageable publishing paths from wandering outside the publish mirror, blocks remote write requests, caps text fetched from external sources, and strips terminal output into UI-safe strings. Those decisions are not cosmetic. They prevent the portfolio builder from becoming a general-purpose remote command runner or a tool that accidentally pushes the wrong files.

Primary Responsibilities

The first responsibility is serving local files. The same server that handles `/api` requests also serves `index.html`, `styles.css`, `script.js`, `template-preview.html`, project catalogs, assets, and uploaded docs during local preview. The static-serving logic checks that the requested path stays inside the workspace before reading the file. That simple check is what prevents a URL from escaping into arbitrary folders on the machine.

The second responsibility is compilation. Compile Code works because the backend can write source files to a project workspace, discover tools such as `node`, `python`, `gcc`, `g++`, `javac`, `iverilog`, `vvp`, and `LTspice`, run those tools with timeouts, collect stdout and stderr, cache successful builds, and return terminal output immediately to the frontend. The backend treats each language differently because JavaScript, Python, C, C++, Java, Verilog, SystemVerilog, LTspice, and HTML do not build the same way.

The third responsibility is publishing. Publishing is not just 'write projects.json.' The backend must verify that the publish mirror is a Git repository, know its origin remote and branch, check compatibility with a static portfolio site, synchronize with the remote to avoid non-fast-forward errors, stage only approved publish paths, commit if there are changes, and push to the selected branch. This is why the publishing code is careful and verbose.

The fourth responsibility is intelligence. For local AI, the backend can use an OpenAI API key or a local Ollama model. It also enriches portfolio context by reading public project links, same-site files, GitHub repositories, GitHub profiles, README files, and selected source files when the question calls for it. The browser cannot safely hold private API keys, so `server.mjs` keeps that path private.

The fifth responsibility is application maintenance. The update functions check GitHub Releases, download an installer, write a PowerShell handoff script, close the running app, run the installer, and relaunch the updated executable. This is backend work because the browser cannot replace the running desktop app.

Important Data Objects And Why They Matter

`compileLanguageProfiles` is the backend's language map. It tells the compiler system what default filename to use, which file extensions belong to each language, what human label to show, which tools are required, and which Winget package can install the tool if automatic installation exists. Without this object, `compileAndRunCode` would need hardcoded language assumptions scattered throughout the file.

`compileToolCandidates` is the tool-discovery map. It lists command names and likely Windows install paths for Git-adjacent and compiler-adjacent executables. `findTool` uses this object so the app can work even when a tool is installed but not perfectly exposed on PATH. The caches `compileToolCache` and `compileToolVersionCache` exist so repeated compiles do not waste time rediscovering the same executables.

`publishPaths` is the publishing allowlist. It lists the site files and folders the backend is allowed to copy into the publish mirror and stage in Git. This is one of the most important safety objects in the file. Without an allowlist, an Apply to site operation could accidentally stage builder-only files, private drafts, temporary compile output, or unrelated local files.

`portfolioAiInstructions` is the system behavior contract for the portfolio assistant. It tells the model how to distinguish general conversation, general engineering, general knowledge, and portfolio-specific questions. It also tells the model when to use context, when not to invent, and how to handle public GitHub source excerpts. That string is long because it is the policy boundary for the AI feature.

Chapter 1: Syntax, Runtime, And Framework Foundations

Runtime context: `server.mjs` is an ES module running on Node.js. That matters before any individual function is discussed, because the file is not browser JavaScript. It can read and write files, start child processes, run Git, call compilers, listen on a local port, and keep private API keys on the machine. The browser side asks for work; this file performs the work.

Framework choice: The file intentionally does not use Express or another web framework. Instead, Node's `createServer` receives each request and the file routes it manually. That is why request parsing, security headers, JSON responses, local-write checks, and static-file serving all appear in this file.

Analogy: *This is like wiring a circuit by hand instead of plugging into a premade development board: it takes more explicit routing, but every connection is visible and owned by the file.*

Node module imports used by `server.mjs`

```
import { createServer } from "node:http";
import { execFile, spawn } from "node:child_process";
import { createHash } from "node:crypto";
import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from "node:fs/promises";
import os from "node:os";
import path from "node:path";
import { promisify } from "node:util";
import { fileURLToPath } from "node:url";
```

Module map: The imports are the backend toolboxes. `node:http` creates the local HTTP server. `node:fs/promises` gives Promise-based file access. `node:child_process` runs Git, compilers, simulators, and installers. `node:path` and `node:url` make paths predictable on Windows. `node:crypto` supports hashing for authorization scopes. `node:os` lets the backend reason about the current machine. Once you understand these modules, the file reads less like random utility code and more like a small operating station for the builder.

Function Syntax Rules Used Later

Reading rule: The later chapters include function entry points and exact source blocks, but they do not re-explain the same syntax over and over. Read `async` as a marker that the function may wait for file work, network work, Git, compilers, or AI responses. Read parameters as the caller's contract with the function. Read default values after `=` as fallback behavior. Read destructured parameters in braces as one object being unpacked into named fields. Read `const`, `let`, and `var` as places where the function or file stores information while it works.

Compact syntax example used as a reference for later chapters

```
async function exampleOperation({ projectId, fileName = "main.c" }, options = {}) {
  const target = resolveInsideCompileRoot(projectId, fileName);
  return await runProcess("tool", [target], options);
}
```

How to use later chapters: After this chapter, a function section will only explain what is specific to that function: why it exists, what data it receives, what exact code implements it, what helpers it calls, and what local objects or variables carry the work.

Chapter 2: Top-Level Objects And Variables

This chapter explains the long-lived objects and variables before the function walkthrough. Think of these top-level values as the workshop benches, labeled drawers, shared measuring tools, and safety rails of the file. A function can walk into the workshop, use these shared objects, and leave without recreating the whole environment from scratch. They are explained by meaning and use, not by line number.

Filesystem Location

root

root is the absolute folder that contains server.mjs. Every local static-file lookup starts from this folder unless the app is deliberately publishing or compiling somewhere else. It is calculated from import.meta.url instead of the current shell directory so the backend behaves the same whether Electron, PowerShell, or another launcher starts it. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include installCompilerTools, resolveInsideRoot, getPublishTargetInfo, runGit, runOptionalCommand, writeTargetCustomDomain, publishAuthCacheScope, syncFromPublishTarget, syncPortfolioPublishFiles, handleApi.

Source excerpt for root

```
const root = path.dirname(fileURLToPath(import.meta.url));
```

portfolioRoot

portfolioRoot is the folder treated as the publishable website mirror. It normally equals root, but OMB_PORTFOLIO_WORKSPACE can redirect it to a separate portfolio folder. Publishing functions use this boundary when copying and staging public website files. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include resolveInsidePortfolioRoot, getPublishTargetInfo, runPublishGit, runGitWithInput, storeGitCredentials, writeTargetCustomDomain, ensureGitRepository, publishAuthCacheScope, authenticateGitHubForTarget, syncPortfolioPublishFiles.

Source excerpt for portfolioRoot

```
const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
```

compileRoot

compileRoot is the separate workspace for Compile Code projects. It is deliberately outside the public site folder so temporary source files, binaries, VCD waveforms, and compiler output do not get mixed into the portfolio website. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include runProcess, compileToolStatus, resolveInsideCompileRoot.

Source excerpt for compileRoot

```
const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root), "compile-code"));
```

draftPath

draftPath is the private local draft catalog path. It stores builder-side edits that have not necessarily been applied to the public projects.json file. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include syncFromPublishTarget, handleApi.

Source excerpt for draftPath

```
const draftPath = path.join(root, "projects.local.json");
```

catalogPath

catalogPath is the public portfolio catalog path. The website runtime reads this file to render the public project list, profile, sections, fun facts, links, and rich content. It controls a boundary or destination for file work, so an

incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include `handleApi`.

Source excerpt for `catalogPath`

```
const catalogPath = path.join(root, "projects.json");
```

`publishAuthCachePath`

`publishAuthCachePath` is the local file that remembers recent publishing authorization. It is intentionally local to the machine and target so GitHub verification does not have to repeat on every Apply to site click. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include `readPublishAuthCache`, `writePublishAuthCache`.

Source excerpt for `publishAuthCachePath`

```
const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
```

`publishPaths`

`publishPaths` is the allowlist of files and folders that can be copied and staged during publishing. This is a critical safety object: it prevents builder-only files, temp compiler output, credentials, and internal docs from being pushed as website content. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include `syncPortfolioPublishFiles`, `publishSiteChanges`.

Source excerpt for `publishPaths`

```
const publishPaths = [
  "projects.json",
  "docs",
  "assets",
  "index.html",
  "styles.css",
  "script.js",
  "electronics-search.js",
  "Backgrounds",
  ".nojekyll",
  "_headers",
  ".well-known",
  "robots.txt",
  "sitemap.xml",
  "CNAME"
];
```

Temporary Calculation

`port`

`port` is the TCP port used by the local HTTP server. `PORT` can override it for development or packaging, while the default supports the local builder preview without requiring a user to choose a port. Functions that reference it include `handleApi`.

Source excerpt for `port`

```
const port = Number(process.env.PORT || 8080);
```

`host`

`host` is the network interface binding for the server. The default allows the local runtime to listen broadly, but write operations still rely on `isLocalRequest` so privilege is not granted merely because a browser can reach the server. Functions that reference it include `parseGitHubSourceUrl`, `callOllamaPortfolioAi`, `storeGitCredentials`, `handleApi`.

Source excerpt for `host`

```
const host = process.env.HOST || "0.0.0.0";
```

execFileAsync

execFileAsync is the Promise-based wrapper around Node's execFile. It lets Git, version checks, and other short commands use async/await instead of nested callbacks. Functions that reference it include findTool, runGit, runOptionalCommand.

Source excerpt for execFileAsync

```
const execFileAsync = promisify(execFile);
```

packageJson

packageJson holds metadata from package.json, especially the current builder version used by update checks, release comparisons, and status messages. Functions that reference it include getGitStatus, getUpdateInfo.

Source excerpt for packageJson

```
const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
```

types

types maps file extensions to HTTP Content-Type headers. Static serving uses it so browsers interpret CSS as CSS, JavaScript as JavaScript, PDFs as PDFs, icons as icons, and uploaded assets as the right media type. Visible object fields include .css, .csv, .html, .ico, .js, .json, .md, .pdf, .png, .jpg, .jpeg, .svg, .txt, .webp, .xml, .xlsx, .zip. Those fields form the contract consumed by related functions. Functions that reference it include handleApi.

Source excerpt for types

```
const types = {
  ".css": "text/css",
  ".csv": "text/csv",
  ".html": "text/html",
  ".ico": "image/x-icon",
  ".js": "application/javascript",
  ".json": "application/json",
  ".md": "text/markdown",
  ".pdf": "application/pdf",
  ".png": "image/png",
  ".jpg": "image/jpeg",
  ".jpeg": "image/jpeg",
  ".svg": "image/svg+xml",
  ".txt": "text/plain",
  ".webp": "image/webp",
  ".xml": "application/xml",
  ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
  ".zip": "application/zip"
};
```

defaultSiteRepository

defaultSiteRepository stores the optional repository configured through the environment. It gives packaged or developer-launched builds a default target without hardcoding one into the code. Functions that reference it include getPublishTargetInfo, assertPublishAccess.

Source excerpt for defaultSiteRepository

```
const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY || "";
```

Runtime State Or Cache

publishAuthCacheTtlMs

publishAuthCacheTtlMs is the normal one-day authorization window. It implements the rule that GitHub publishing should not ask again every time the app opens, but also should not trust an old session forever. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache, publishAuthCacheExpiresAt.

Source excerpt for publishAuthCacheTtlMs

```
const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
```

publishAuthHistoryWindowMs

publishAuthHistoryWindowMs is the seven-day window used to count successful authorizations. The cache logic uses it to decide whether the user qualifies for extended trust. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

Source excerpt for publishAuthHistoryWindowMs

```
const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
```

compileToolCache

compileToolCache remembers the resolved path for compiler tools. It prevents every compile click from scanning the filesystem again. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include findTool, installCompilerTools.

Source excerpt for compileToolCache

```
const compileToolCache = new Map();
```

compileToolVersionCache

compileToolVersionCache remembers version strings for tools after they have been checked. This makes status panels faster and keeps repeated diagnostics from slowing the UI. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include toolVersionLine.

Source excerpt for compileToolVersionCache

```
const compileToolVersionCache = new Map();
```

Git Publishing And Authorization State

publishAuthExtendedTtlMs

publishAuthExtendedTtlMs is the thirty-day extended authorization window. It is used after repeated successful authorizations prove the same device and target are consistently owned by the same publisher. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

Source excerpt for publishAuthExtendedTtlMs

```
const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
```

publishAuthExtendedThreshold

publishAuthExtendedThreshold is the number of successful authorizations required before the builder extends trust. It turns the user's requested security rule into a concrete number. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

Source excerpt for publishAuthExtendedThreshold

```
const publishAuthExtendedThreshold = 3;
```

publishAuthorizationHelp

publishAuthorizationHelp is the human-readable troubleshooting text shown when publishing is blocked. It explains the sequence the user should follow: configure target, authenticate, load if needed, then apply. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include publishAccessError, handleApi.

Source excerpt for publishAuthorizationHelp

```
const publishAuthorizationHelp = [
  "Publishing was blocked before live website files were applied.",
  "Open Publishing target, enter the repository URL, then click Authenticate target.",
  "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write access to the selected Pages repository.",
  "Authenticate target only verifies write access and remembers it for the matching repository and branch.",
];
```



```
"Load from target only imports compatible website files from the authenticated repository into the local builder workspace.",
"Until a compatible writable website repository is associated, the builder remains local-only."
].join(" ");
```

gitCandidates

gitCandidates lists command names and likely Windows Git paths. Git operations use this list to find Git even when PATH is incomplete, which is common on Windows installs. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include runGit, runGitWithInput.

Source excerpt for gitCandidates

```
const gitCandidates = [
  process.env.GIT_EXE,
  "git",
  "C:\\Program Files\\Git\\cmd\\git.exe",
  "C:\\Program Files\\Git\\bin\\git.exe",
  "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
  "C:\\Program Files (x86)\\Git\\bin\\git.exe",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "cmd", "git.exe") : "",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "bin", "git.exe") : ""
].filter(Boolean);
```

githubTextFilePattern

githubTextFilePattern tells the AI source reader which GitHub files are worth fetching as text. It includes code, README, HDL, schematic-like text formats, scripts, and engineering project files. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include scoreGitHubFile, fetchGitHubRepositorySource.

Source excerpt for githubTextFilePattern

```
const githubTextFilePattern = /\.(\.txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|cc|hh|py|js|mjs|ts|tsx|v|sv|vhd1?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl|yaml|yml)$/i;
```

githubSkipFilePattern

githubSkipFilePattern tells the AI source reader which GitHub files to skip because they are binary, huge, media-oriented, or not useful as prompt text. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include fetchGitHubRepositorySource.

Source excerpt for githubSkipFilePattern

```
const githubSkipFilePattern = /\.(\.png|jpe?g|gif|webp|svg|pdf|zip|7z|rar|exe|dll|bin|obj|o|a|so|dylib|mp4|mov|avi|mp3|wav|xlsx?|pptx?|docx?)/i;
```

Lookup Collection

blockedAppUpdateVersions

blockedAppUpdateVersions is a Set of release versions the updater must skip. It exists so a known bad release can be blocked without deleting every release artifact from GitHub. Functions that reference it include getUpdateInfo.

Source excerpt for blockedAppUpdateVersions

```
const blockedAppUpdateVersions = new Set(["0.2.16"]);
```

Portfolio Data State

compileLanguageProfiles

compileLanguageProfiles is the language rulebook for the IDE feature. Each language entry defines the default filename, valid extensions, display label, required tools, and installer hints. Compile, build, run, simulate, and beautify operations all depend on this map. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Visible object fields include c, defaultFile, extensions, label, primaryTools, winget, cpp, verilog, systemverilog, ltspice, java, javascript, python, html. Those fields form the contract consumed by related functions. Functions that reference it include normalizeCodeLanguage,

languageFromFileName, safeCodeFileName, compileToolStatus, hdlFilesFromPayload, compileWorkspaceFilesFromPayload, compileAndRunCode, installCompilerTools.

Source excerpt for compileLanguageProfiles

```
const compileLanguageProfiles = {
  c: {
    defaultFile: "main.c",
    extensions: [".c", ".h"],
    label: "C",
    primaryTools: ["gcc"],
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  cpp: {
    defaultFile: "main.cpp",
    extensions: [".cpp", ".cc", ".cxx", ".hpp", ".h"],
    label: "C++",
    primaryTools: ["g++"],
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  verilog: {
    defaultFile: "design.v",
    extensions: [".v"],
    label: "Verilog",
    primaryTools: ["iverilog", "vvp"],
    winget: ["Icarus.Verilog"]
  },
  systemverilog: {
    defaultFile: "design.sv",
    extensions: [".sv", ".svh"],
    label: "SystemVerilog",
    primaryTools: ["iverilog", "vvp"],
    winget: ["Icarus.Verilog"]
  },
  ltspice: {
    defaultFile: "simulation.cir",
    extensions: [".cir", ".net", ".sp", ".asc"],
    label: "LTspice",
    primaryTools: ["ltspice"],
    winget: []
  },
  java: {
    defaultFile: "Main.java",
    extensions: [".java"],
    label: "Java",
    primaryTools: ["javac", "java"],
    winget: ["EclipseAdoptium.Temurin.21.JDK"]
  },
  javascript: {
    defaultFile: "main.js",
    extensions: [".js", ".mjs", ".cjs"],
    label: "JavaScript",
    primaryTools: ["node"],
    winget: []
  },
  python: {
    defaultFile: "main.py",
    extensions: [".py"],
    label: "Python",
    primaryTools: ["python"],
    winget: []
  },
  html: {
    defaultFile: "index.html",
    extensions: [".html", ".htm"],
    label: "HTML",
    primaryTools: [],
    winget: []
  }
};
```

Compiler Or Language Configuration

compileToolCandidates

compileToolCandidates maps compiler tool names to possible executable locations. It lets the app find gcc, g++, javac, java, iverilog, vvp, python, node, and LTspice even when a normal terminal cannot. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Visible object fields include gcc, g++, clang, clang++, cl, javac, java, node, python, iverilog, vvp, ltspice. Those fields form the contract consumed by related functions. Functions that reference it include findTool.

Source excerpt for compileToolCandidates

```
const compileToolCandidates = {
  gcc: [
    "gcc",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
    "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "gcc.exe") : ""
  ],
  "g++": [
    "g++",
    "c++",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
    "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "g++.exe") : ""
  ],
  clang: ["clang"],
  "clang++": ["clang++"],
  cl: ["cl"],
  javac: [
    "javac",
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\javac.exe"
  ],
  java: [
    "java",
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\java.exe"
  ],
  node: [
    /(?:^|[/\\])node(?:\.exe)?$/i.test(process.execPath || "") ? process.execPath : "",
    "node",
    "C:\\Program Files\\nodejs\\node.exe",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
  ],
  python: [process.env.PYTHON, "python", "py"],
  iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"],
  vvp: ["vvp", "C:\\iverilog\\bin\\vvp.exe"],
  ltspice: [
    process.env.LTSPICE_EXE,
    "C:\\Program Files\\ADI\\LTspice\\LTspice.exe",
    "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe",
    "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") : ""
  ]
};
```

Ai Configuration Or Conversation State

portfolioAiInstructions

portfolioAiInstructions is the system prompt for the portfolio assistant. It defines how the model should distinguish greetings, general electronics questions, portfolio-specific questions, public GitHub code requests, and uncertain answers. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include callOllamaPortfolioAi, handlePortfolioAi.

Source excerpt for portfolioAiInstructions

```
const portfolioAiInstructions = [
  "You are the AI assistant for the portfolio website described by the supplied portfolio context.",
  "Behave like a careful senior electrical and computer engineering mentor who can also navigate the saved portfolio.",
  "Answer the visitor's question first in a concise ChatGPT-like format, then add portfolio links or context only when they help.",
  "Use the supplied question intent to decide whether this is a general engineering question or a portfolio-specific question.",
];
```

```

    "For general_conversation intent, respond naturally and briefly. Use the recent conversation instead of a fixed
    template. If the visitor says hi, a good answer is a short greeting such as: Hi, what can I do for you?",
    "If the visitor asks 'what is my name?' or 'who am I?' and they have not identified themselves, do not pretend to
    know the visitor. Say you are an AI agent for this portfolio and identify the portfolio owner only if the supplied
    profile context includes a name.",
    "For general_knowledge intent, answer the question directly using broad general knowledge. Do not force portfolio
    context unless the visitor asks to connect the answer to the portfolio owner's work.",
    "For general_engineering intent, begin with the general electronics or engineering explanation. Do not lead with
    project context unless the visitor asks to connect it.",
    "For portfolio_specific intent, answer from the portfolio context first, then explain related engineering concepts
    only when useful.",
    "Use recent conversation history for follow-up questions, pronouns, comparisons, and corrections.",
    "Use the supplied portfolio context as the trusted source for the portfolio owner's projects, links, files,
    resume, and contact information.",
    "Use sourceExcerpts when present as higher-detail evidence from uploaded text files, extracted resume text, same-
    site files, GitHub pages, or other safe public sources.",
    "Use knowledgeManifest to understand project files, image evidence, public profiles, resumes, and project areas.
    Treat filenames, captions, surrounding text, and descriptions as evidence.",
    "Do not claim to visually inspect an image unless actual image analysis is provided. If only image metadata is
    supplied, say what the caption/path/context suggests.",
    "For GitHub, LinkedIn, resume, or uploaded-file questions, cite what is present in the supplied context or fetched
    excerpts and then point to the link when useful. Access public pages and fetched excerpts when the website allows
    it.",
    "When a visitor asks for source code, use public GitHub source excerpts when provided. Show concise relevant
    snippets with file paths, and explain what the code is doing. Do not imply private repository access.",
    "If sourceExcerpts include lines labeled Source file, include at least one fenced code block with the file path
    immediately before it, unless the visitor explicitly asks for links only.",
    "Never invent, infer, or write hypothetical code for the portfolio owner's repositories. Every fenced code block
    about repository work must be copied from a fetched Source file excerpt. If no source file excerpt is available, say
    that the code was not available in the fetched public sources and give the repository link instead.",
    "For questions about public GitHub repositories, prefer fetched public GitHub source excerpts over portfolio
    summaries or project descriptions. You may fetch and display concise code from public GitHub URLs supplied in the
    portfolio context or safe source fetches, but never imply access to private repositories.",
    "You may answer related electronics, hardware, analog, mixed-signal, digital, embedded, FPGA, ASIC, PCB, and
    firmware questions even when the answer is broader than the saved portfolio.",
    "Do not invent portfolio projects, credentials, employers, files, or test results that are not in the context.",
    "If context is missing, say what is missing and answer generally only for the engineering concept.",
    "When useful, state assumptions, define terms, explain signal or data flow, identify tradeoffs, and name what
    evidence would prove the claim.",
    "Only mention or recommend portfolio links that are directly relevant to the visitor's current question or the
    active follow-up context. Do not append random links.",
    "Keep the answer recruiter-friendly, specific, and easy to skim. Use short paragraphs and bullets when helpful.",
    "Do not expose chain-of-thought. Give the polished answer only."
].join("\n");

```

Chapter 3: Function Walkthrough With Source Blocks

This walkthrough is function-by-function. Each section now follows a textbook rhythm: first the idea of the function, then the syntax, then how to read that syntax, then the implementation and the local objects that make the implementation work. Line numbers are deliberately avoided because the source will keep changing.

HTTP API surface

The functions in this group all support http api surface. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function securityHeaders(extra = {})

Role: Begin with the role of securityHeaders. securityHeaders belongs to http api surface. This function belongs to the local HTTP/API layer. It either shapes responses, reads requests, or decides whether a request is allowed to perform local work. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function securityHeaders(extra = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. extra is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more

predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for securityHeaders

```
function securityHeaders(extra = {}) {  
  return {  
    "X-Content-Type-Options": "nosniff",  
    "Referrer-Policy": "strict-origin-when-cross-origin",  
    "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",  
    "Cross-Origin-Opener-Policy": "same-origin",  
    ...extra  
  };  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means securityHeaders performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without securityHeaders, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside securityHeaders, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function sendJson(response, status, data)

Role: Begin with the role of sendJson. sendJson belongs to http api surface. This function belongs to the local HTTP/API layer. It either shapes responses, reads requests, or decides whether a request is allowed to perform local work. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sendJson(response, status, data) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. response is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. status is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. data is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for sendJson

```
function sendJson(response, status, data) {  
  response.writeHead(status, {  
    ...securityHeaders(),  
    "Content-Type": "application/json",  
    "Cache-Control": "no-store, no-cache, must-revalidate"  
  });  
}
```

```
response.end(JSON.stringify(data, null, 2));
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; writes the HTTP response directly.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with securityHeaders. Those calls show that sendJson is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without sendJson, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside sendJson, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function isLocalRequest(request)

Role: Begin with the role of isLocalRequest. isLocalRequest is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Function entry point:

```
function isLocalRequest(request) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. request is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for isLocalRequest

```
function isLocalRequest(request) {
  const address = request.socket.remoteAddress || "";
  return address === "127.0.0.1" || address === "::1" || address === "::ffff:127.0.0.1";
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means isLocalRequest performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without isLocalRequest, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside isLocalRequest explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

address: address stores a computed value for temporary calculation. It is initialized from `request.socket.remoteAddress || ""`. Within the function it is returned to the caller; assigned or updated later in the function.

async function readRequestJson(request)

Role: The best entry point for `readRequestJson` is its responsibility in the surrounding workflow. `readRequestJson` reads `RequestJson` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Function entry point:

```
async function readRequestJson(request) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for readRequestJson

```
async function readRequestJson(request) {  
  const chunks = [];  
  let size = 0;  
  
  for await (const chunk of request) {  
    size += chunk.length;  
    if (size > 60 * 1024 * 1024) {  
      throw new Error("Request body is too large.");  
    }  
    chunks.push(chunk);  
  }  
  
  return JSON.parse((Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\uFEFF/, ""));  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data; throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `readRequestJson` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `readRequestJson`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `readRequestJson` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

chunks: `chunks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; passed into `parse`; assigned or updated later in the function.

size: `size` stores a numeric value for temporary calculation. It is initialized from `0`. Within the function it is assigned or updated later in the function.

chunk: chunk starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods length; passed into await, push.

async function handleApi(request, response, url)

Role: Read handleApi first as a named idea, not as syntax. handleApi is the backend router. It receives the parsed URL and decides which local API feature should run: project catalog reads, uploads, draft saves, compile actions, compiler installation, target authentication, load from target, apply to site, update checks, security reports, AI calls, or builder download reports. In a framework such as Express this would be spread across route handlers. Here it is one explicit switch-like controller. The function protects the boundary between a browser click and local machine power. Before write operations, it checks whether the request is local. It reads request JSON only when needed, calls the specific operation function, and returns a normalized JSON response. Without handleApi, the server could still serve files, but the builder would have no organized command surface.

Function entry point:

```
async function handleApi(request, response, url) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. request is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. response is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for handleApi

```
async function handleApi(request, response, url) {
  if (request.method === "GET" && url.pathname === "/api/catalog") {
    try {
      const catalog = await readJsonFile(draftPath);
      sendJson(response, 200, { source: "draft", catalog });
    } catch {
      sendJson(response, 200, { source: "published", catalog: await readJsonFile(catalogPath) });
    }
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/templates") {
    sendJson(response, 200, await readJsonFile(path.join(root, "project-templates.json")));
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/publish-target") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Publishing target details are only available from this computer." });
      return true;
    }
    sendJson(response, 200, { ok: true, target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/system-check") {
    sendJson(response, 200, { ok: true, system: await getGitStatus(), target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/app-update") {
    sendJson(response, 200, await getUpdateInfo());
    return true;
  }
}
```



```

if (request.method === "GET" && url.pathname === "/api/security-report") {
  if (!isLocalRequest(request)) {
    sendJson(response, 403, { error: "Security reports are only available from this computer." });
    return true;
  }
  sendJson(response, 200, await getSecurityReport());
  return true;
}

if (request.method === "GET" && url.pathname === "/api/code/tools") {
  if (!isLocalRequest(request)) {
    sendJson(response, 403, { error: "Compiler details are only available from this computer." });
    return true;
  }
  sendJson(response, 200, { ok: true, tools: await compileToolStatus() });
  return true;
}

if (request.method !== "POST") return false;

if (!isLocalRequest(request)) {
  sendJson(response, 403, { error: "Write actions are only allowed from this computer." });
  return true;
}

if (url.pathname === "/api/portfolio-ai") {
  await handlePortfolioAi(request, response);
  return true;
}

if (url.pathname === "/api/code/beautify") {
  try {
    const body = await readRequestJson(request);
    const language = normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code,
body.fileName));
    sendJson(response, 200, {
      ok: true,
      language,
      code: beautifyCode(body.code || "", language)
    });
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Code could not be beautified." });
  }
  return true;
}

if (url.pathname === "/api/code/save") {
  try {
    const body = await readRequestJson(request);
    const saved = await saveCompileSource(body);
    sendJson(response, 200, { ok: true, saved });
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Code could not be saved." });
  }
  return true;
}

if (url.pathname === "/api/code/compile") {
  try {
    const body = await readRequestJson(request);
    const result = await compileAndRunCode(body);
    sendJson(response, 200, { ok: result.ok, result });
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Code could not be compiled.", result: null });
  }
  return true;
}

if (url.pathname === "/api/code/install-tools") {
  try {
    const body = await readRequestJson(request);
    const result = await installCompilerTools(body.language || "");
    sendJson(response, 200, result);
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Compiler tools could not be installed." });
  }
}

```

```

    }
    return true;
}

if (url.pathname === "/api/publish-target") {
    await readRequestJson(request).catch(() => ({}));
    sendJson(response, 400, {
        ok: false,
        error: "Publishing target setup now requires GitHub authentication.",
        details: "Use Authenticate target. The builder keeps the previous target until GitHub write access is
verified."
    });
    return true;
}

if (url.pathname === "/api/install-git") {
    try {
        const install = await installGitForWindows();
        sendJson(response, 200, { ok: true, install, system: await getGitStatus() });
    } catch (error) {
        sendJson(response, 400, {
            ok: false,
            error: error.message || "Git for Windows installer could not be started.",
            downloadUrl: "https://git-scm.com/download/win"
        });
    }
    return true;
}

if (url.pathname === "/api/app-update/install") {
    try {
        await readRequestJson(request).catch(() => ({}));
        const update = await downloadAndLaunchAppUpdate();
        sendJson(response, 200, { ok: true, update });
    } catch (error) {
        sendJson(response, 400, {
            ok: false,
            error: error.message || "The update could not be started."
        });
    }
    return true;
}

if (url.pathname === "/api/github-authenticate") {
    try {
        const body = await readRequestJson(request);
        const auth = await authenticateGitHubForTarget(body || {});
        sendJson(response, 200, { ok: true, auth, target: await getPublishTargetInfo() });
    } catch (error) {
        sendJson(response, 400, {
            ok: false,
            error: error.message || "GitHub authentication did not complete.",
            details: error.details || publishAuthorizationHelp,
            publishAccess: error.publishAccess || null,
            system: await getGitStatus()
        });
    }
    return true;
}

if (url.pathname === "/api/sync-from-publish-target") {
    try {
        const sync = await syncFromPublishTarget();
        sendJson(response, 200, { ok: true, sync, target: await getPublishTargetInfo() });
    } catch (error) {
        sendJson(response, 400, {
            ok: false,
            error: error.message || "Publishing target could not be imported.",
            details: error.details || publishAuthorizationHelp,
            publishAccess: error.publishAccess || null
        });
    }
    return true;
}

if (url.pathname === "/api/save-draft" || url.pathname === "/api/apply-catalog") {

```

```

const body = await readRequestJson(request);
const catalog = body.catalog;

if (!catalog || !Array.isArray(catalog.categories) || !Array.isArray(catalog.projects) || !
catalog.categories.length) {
  sendJson(response, 400, { error: "Catalog must include categories and projects arrays." });
  return true;
}

const applyingToSite = url.pathname === "/api/apply-catalog";
let publishAccess = null;
if (applyingToSite) {
  try {
    publishAccess = await assertPublishAccess();
  } catch (error) {
    sendJson(response, 200, {
      ok: false,
      file: path.relative(root, catalogPath),
      publish: {
        pushed: false,
        authorizationRequired: true,
        error: error.message || "Publishing authorization failed.",
        details: error.details || publishAuthorizationHelp,
        ...(error.publishAccess || {})
      }
    });
    return true;
  }
}

const target = applyingToSite ? catalogPath : draftPath;
await writeFile(target, `${JSON.stringify(catalog, null, 2)}\n`, "utf8");
if (applyingToSite) {
  try {
    const publish = await publishSiteChanges(publishAccess);
    sendJson(response, 200, { ok: true, file: path.relative(root, target), publish });
  } catch (error) {
    sendJson(response, 200, {
      ok: true,
      file: path.relative(root, target),
      publish: {
        pushed: false,
        error: error.stderr || error.stdout || error.message || "Git push failed."
      }
    });
  }
  return true;
}
sendJson(response, 200, { ok: true, file: path.relative(root, target) });
return true;
}

if (url.pathname === "/api/upload") {
  const body = await readRequestJson(request);
  const projectId = safeSegment(body.projectId, "project");
  const section = safeSegment(body.section, "documents");
  const fileName = safeFileName(body.fileName);
  const base64 = String(body.data || "").replace(/^data:[^;]+;base64/, "");

  if (!base64) {
    sendJson(response, 400, { error: "Upload data is missing." });
    return true;
  }

  const folder = resolveInsideRoot("docs", projectId, section);
  const filePath = resolveInsideRoot("docs", projectId, section, fileName);
  await mkdir(folder, { recursive: true });
  await writeFile(filePath, Buffer.from(base64, "base64"));

  sendJson(response, 200, {
    ok: true,
    url: path.relative(root, filePath).replaceAll(path.sep, "/")
  });
  return true;
}

```

```

    return false;
  }

  createServer(async (request, response) => {
    try {
      const url = new URL(request.url || "/", `http://${request.headers.host}`);
      if (url.pathname.startsWith("/api/") && await handleApi(request, response, url)) return;

      const pathname = url.pathname === "/" ? "/index.html" : decodeURIComponent(url.pathname);
      const filePath = path.normalize(path.join(root, pathname));

      if (!filePath.startsWith(root)) {
        response.writeHead(403);
        response.end("Forbidden");
        return;
      }

      const body = await readFile(filePath);
      response.writeHead(200, {
        ...securityHeaders(),
        "Content-Type": types[path.extname(filePath)] || "application/octet-stream",
        "Cache-Control": "no-store, no-cache, must-revalidate"
      });
      response.end(body);
    } catch {
      response.writeHead(404);
      response.end("Not found");
    }
  }).listen(port, host, () => {
    console.log(`Portfolio preview running at http://localhost:${port}`);
    console.log("For phone access, use this computer's Wi-Fi/LAN IP address with the same port.");
  })
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; uses Node path utilities so Windows path separators and relative paths are handled consistently; writes the HTTP response directly; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

Collaborators: The function also has to be read through the helpers it calls. It works with `readJsonFile`, `sendJson`, `isLocalRequest`, `getPublishTargetInfo`, `getGitStatus`, `getUpdateInfo`, `getSecurityReport`, `compileToolStatus`. Those calls show that `handleApi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `handleApi`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `handleApi` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

catalog: `catalog` stores a resolved filesystem or route value. It is initialized from `await readJsonFile(draftPath)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `categories`, `projects`; passed into `isArray`, `sendJson`, `writeFile`; assigned or updated later in the function.

body: `body` stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

language: language stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code, body.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `installCompilerTools`, `normalizeCodeLanguage`, `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

saved: saved stores the completed result of an awaited operation. It is initialized from `await saveCompileSource(body)`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from `await compileAndRunCode(body)`. Within the function it is accesses properties/methods ok; passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from `await installCompilerTools(body.language || "")`. Within the function it is accesses properties/methods ok; passed into `sendJson`; assigned or updated later in the function.

install: install stores the completed result of an awaited operation. It is initialized from `await installGitForWindows()`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

update: update stores the completed result of an awaited operation. It is initialized from `await downloadAndLaunchAppUpdate()`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

auth: auth stores the completed result of an awaited operation. It is initialized from `await authenticateGitHubForTarget(body || {})`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `sendJson`; assigned or updated later in the function.

sync: sync stores the completed result of an awaited operation. It is initialized from `await syncFromPublishTarget()`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

catalog: catalog stores a computed value for portfolio data state. It is initialized from `body.catalog`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `categories`, `projects`; passed into `isArray`, `sendJson`, `writeFile`; assigned or updated later in the function.

applyingToSite: applyingToSite stores a computed value for temporary calculation. It is initialized from `url.pathname === "/api/apply-catalog"`. Within the function it is assigned or updated later in the function.

publishAccess: publishAccess stores a computed value for Git publishing and authorization state. It is initialized from `null`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `publishSiteChanges`, `sendJson`; assigned or updated later in the function.

target: target stores a resolved filesystem or route value. It is initialized from `applyingToSite ? catalogPath : draftPath`. Within the function it is passed into `sendJson`, `writeFile`; assigned or updated later in the function.

publish: publish stores the completed result of an awaited operation. It is initialized from `await publishSiteChanges(publishAccess)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

projectId: projectId stores a computed value for portfolio data state. It is initialized from `safeSegment(body.projectId, "project")`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `resolveInsideRoot`, `safeSegment`; assigned or updated later in the function.

section: section stores a computed value for portfolio data state. It is initialized from `safeSegment(body.section, "documents")`. Within the function it is passed into `resolveInsideRoot`, `safeSegment`; assigned or updated later in the function.

fileName: fileName stores a computed value for temporary calculation. It is initialized from `safeFileName(body.fileName)`. Within the function it is passed into `normalizeCodeLanguage`, `resolveInsideRoot`, `safeFileName`; assigned or updated later in the function.

base64: base64 stores a computed value for temporary calculation. It is initialized from `String(body.data || "").replace(/^(data:[^`. Within the function it is passed into `replace`, `writeFile`; assigned or updated later in the function.

folder: folder stores a computed value for filesystem location. It is initialized from `resolveInsideRoot("docs", projectId, section)`. Within the function it is passed into `mkdir`; assigned or updated later in the function.

filePath: filePath stores a computed value for filesystem location. It is initialized from `resolveInsideRoot("docs", projectId, section, fileName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `startsWith`; passed into `extname`, `readFile`, `sendJson`, `writeFile`; assigned or updated later in the function.

url: url stores a constructed object for network or routing value. It is initialized from new URL(request.url || "/", `http://\${request.headers.host}`). Within the function it is accesses properties/methods pathname; passed into URL, decodeURIComponent, handleApi, sendJson; assigned or updated later in the function.

pathname: pathname stores a computed value for filesystem location. It is initialized from url.pathname === "/" ? "/index.html" : decodeURIComponent(url.pathname). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods startsWith; passed into decodeURIComponent, normalize; assigned or updated later in the function.

filePath: filePath stores a resolved filesystem or route value. It is initialized from path.normalize(path.join(root, pathname)). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods startsWith; passed into extname, readFile, sendJson, writeFile; assigned or updated later in the function.

body: body stores a resolved filesystem or route value. It is initialized from await readFile(filePath). Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into String, authenticateGitHubForTarget, compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

Backend utility

The functions in this group all support backend utility. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function clampText(value, maxLength = 12000)

Role: Read clampText first as a named idea, not as syntax. clampText belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function clampText(value, maxLength = 12000) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. maxLength is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for clampText

```
function clampText(value, maxLength = 12000) {  
  return String(value || "").slice(0, maxLength);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means clampText performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without clampText, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside clampText, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function stripHtmlToText(value = "")

Role: The best entry point for stripHtmlToText is its responsibility in the surrounding workflow. stripHtmlToText belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function stripHtmlToText(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for stripHtmlToText

```
function stripHtmlToText(value = "") {  
  return String(value || "")  
    .replace(/<script[\s\S]*?<\script>/gi, " ")  
    .replace(/<style[\s\S]*?<\style>/gi, " ")  
    .replace(/<[>]+>/g, " ")  
    .replace(/&nbsp;/gi, " ")  
    .replace(/&amp;/gi, "&")  
    .replace(/&lt;/gi, "<")  
    .replace(/&gt;/gi, ">")  
    .replace(/&quot;/gi, "'")  
    .replace(/&#39;/g, "'")  
    .replace(/\\s+/g, " ")  
    .trim();  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means stripHtmlToText performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without stripHtmlToText, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside stripHtmlToText, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeCodeLanguage(value = "")

Role: Begin with the role of normalizeCodeLanguage. normalizeCodeLanguage standardizes CodeLanguage data before another part of the app uses it. The point is not decoration; it prevents different callers from

interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCodeLanguage(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `normalizeCodeLanguage`

```
function normalizeCodeLanguage(value = "") {  
  const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");  
  const aliases = {  
    c: "c",  
    cpp: "cpp",  
    cplusplus: "cpp",  
    "c++": "cpp",  
    systemverilog: "systemverilog",  
    sv: "systemverilog",  
    verilog: "verilog",  
    v: "verilog",  
    ltspice: "ltspice",  
    spice: "ltspice",  
    cir: "ltspice",  
    java: "java",  
    javascript: "javascript",  
    js: "javascript",  
    node: "javascript",  
    python: "python",  
    py: "python",  
    html: "html",  
    htm: "html"  
  };  
  return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `normalizeCodeLanguage` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `normalizeCodeLanguage`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `normalizeCodeLanguage` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "")`. Within the function it is assigned or updated later in the function.

aliases: aliases stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; assigned or updated later in the function.

function indentBraceCode(code = "")

Role: Read indentBraceCode first as a named idea, not as syntax. indentBraceCode belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function indentBraceCode(code = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for indentBraceCode

```
function indentBraceCode(code = "") {  
  let depth = 0;  
  return String(code || "")  
    .replace(/\r\n?/g, "\n")  
    .split("\n")  
    .map((rawLine) => {  
      const line = rawLine.trim();  
      if (!line) return "";  
      if (/^[{}]\]/.test(line)) depth = Math.max(0, depth - 1);  
      const formatted = `${" ".repeat(depth)}${line}`;  
      const opens = (line.match(/[{\[(]/g) || []).length;  
      const closes = (line.match(/[}\)]/g) || []).length;  
      depth = Math.max(0, depth + opens - closes);  
      return formatted;  
    })  
    .join("\n")  
    .trimEnd();  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means indentBraceCode performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without indentBraceCode, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside indentBraceCode show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

depth: depth stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into max, repeat; assigned or updated later in the function.

line: line stores a computed value for temporary calculation. It is initialized from rawLine.trim(). Within the function it is accesses properties/methods match; passed into test; assigned or updated later in the function.

formatted: formatted stores a string value for temporary calculation. It is initialized from ``${" ".repeat(depth)}${line}``. Within the function it is returned to the caller; assigned or updated later in the function.

opens: opens stores a function or callback value for temporary calculation. It is initialized from `(line.match(/[{([/g) || []]).length`. Within the function it is passed into max; assigned or updated later in the function.

closes: closes stores a function or callback value for temporary calculation. It is initialized from `(line.match(/[\}\]\])/g) || []).length`. Within the function it is passed into max; assigned or updated later in the function.

function beautifyCode(code = "", language = "javascript")

Role: Begin with the role of beautifyCode. beautifyCode belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function beautifyCode(code = "", language = "javascript") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for beautifyCode

```
function beautifyCode(code = "", language = "javascript") {
  const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
  const lang = normalizeCodeLanguage(language);
  if (["c", "cpp", "java", "javascript", "verilog", "systemverilog"].includes(lang)) {
    return indentBraceCode(normalized)
      .replace(/[ \t]+$/gm, "")
      .replace(/\n{4,}/g, "\n\n\n");
  }
  if (lang === "html") {
    return normalized
      .replace(>\s+</g, ">\n<")
      .split("\n")
      .map((line) => line.trim())
      .join("\n")
      .trim();
  }
  return normalized
    .split("\n")
    .map((line) => line.trimEnd())
    .join("\n")
    .replace(/\n{4,}/g, "\n\n\n")
    .trimEnd();
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with normalizeCodeLanguage, indentBraceCode. Those calls show that beautifyCode is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `beautifyCode`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `beautifyCode` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ")`. Within the function it is returned to the caller; passed into `indentBraceCode`; assigned or updated later in the function.

lang: `lang` stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language)`. Within the function it is assigned or updated later in the function.

async function findExecutableUnder(folder, names = [], maxDepth = 5)

Role: To understand `findExecutableUnder`, start with the problem it owns. `findExecutableUnder` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function findExecutableUnder(folder, names = [], maxDepth = 5) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `folder` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `names` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `maxDepth` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for findExecutableUnder

```
async function findExecutableUnder(folder, names = [], maxDepth = 5) {
  if (!folder || maxDepth < 0 || !(await pathExists(folder))) return "";
  let entries = [];
  try {
    entries = await readdir(folder, { withFileTypes: true });
  } catch {
    return "";
  }
  for (const entry of entries) {
    const fullPath = path.join(folder, entry.name);
    if (entry.isFile() && names.some((name) => entry.name.toLowerCase() === name.toLowerCase())) return fullPath;
  }
  for (const entry of entries) {
    if (!entry.isDirectory()) continue;
    const found = await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1);
    if (found) return found;
  }
  return "";
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close

reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Collaborators: Next, trace the helper calls that surround the function. It works with `pathExists`. Those calls show that `findExecutableUnder` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `findExecutableUnder`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `findExecutableUnder` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

entries: `entries` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is assigned or updated later in the function.

entry: `entry` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `isDirectory`, `isFile`, `name`; passed into `findExecutableUnder`, `join`.

fullPath: `fullPath` stores a resolved filesystem or route value. It is initialized from `path.join(folder, entry.name)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is returned to the caller; assigned or updated later in the function.

entry: `entry` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `isDirectory`, `isFile`, `name`; passed into `findExecutableUnder`, `join`.

found: `found` stores a resolved filesystem or route value. It is initialized from `await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1)`. Within the function it is returned to the caller; assigned or updated later in the function.

function terminalLine(label, text = "")

Role: Begin with the role of `terminalLine`. `terminalLine` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function terminalLine(label, text = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `label` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for terminalLine

```
function terminalLine(label, text = "") {  
  return [`$ ${label}`, String(text || "").trim()].filter(Boolean).join("\n");  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `terminalLine` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `terminalLine`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `terminalLine`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function cachedBuildLine(type, outputPath)

Role: Begin with the role of `cachedBuildLine`. `cachedBuildLine` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cachedBuildLine(type, outputPath) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `type` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `outputPath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for cachedBuildLine

```
function cachedBuildLine(type, outputPath) {  
  return `${type} cache hit: using existing artifact\n${outputPath}`;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `cachedBuildLine` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `cachedBuildLine`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `cachedBuildLine`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function fetchLimitedText(url, maxLength = 12000)

Role: The best entry point for `fetchLimitedText` is its responsibility in the surrounding workflow. `fetchLimitedText` retrieves `LimitedText` from outside the immediate function. It wraps network or repository

access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Function entry point:

```
async function fetchLimitedText(url, maxLength = 12000) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `maxLength` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `fetchLimitedText`

```
async function fetchLimitedText(url, maxLength = 12000) {  
  const response = await fetch(url, {  
    headers: githubHeaders("text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1")  
  });  
  if (!response.ok) return "";  
  const rawText = clampText(await response.text(), maxLength);  
  const contentType = response.headers.get("Content-Type") || "";  
  return /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `githubHeaders`, `clampText`, `stripHtmlToText`. Those calls show that `fetchLimitedText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchLimitedText`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `fetchLimitedText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

response: `response` stores data returned from a network call. It is initialized from `await fetch(url, {`. Within the function it is accesses properties/methods `headers`, `ok`, `text`; passed into `clampText`; assigned or updated later in the function.

rawText: `rawText` stores the completed result of an awaited operation. It is initialized from `clampText(await response.text(), maxLength)`. Within the function it is passed into `stripHtmlToText`; assigned or updated later in the function.

contentType: `contentType` stores a computed value for temporary calculation. It is initialized from `response.headers.get("Content-Type") || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

async function runOptionalCommand(command, args = [], options = {})

Role: Read runOptionalCommand first as a named idea, not as syntax. runOptionalCommand executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Function entry point:

```
async function runOptionalCommand(command, args = [], options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. command is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. args is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for runOptionalCommand

```
async function runOptionalCommand(command, args = [], options = {}) {  
  try {  
    const result = await execFileAsync(command, args, {  
      cwd: options.cwd || root,  
      maxBuffer: options.maxBuffer || 2 * 1024 * 1024,  
      timeout: options.timeout || 15000,  
      windowsHide: options.windowsHide !== false  
    });  
    return { ok: true, stdout: result.stdout || "", stderr: result.stderr || "" };  
  } catch (error) {  
    return {  
      ok: false,  
      stdout: error.stdout || "",  
      stderr: error.stderr || "",  
      error: error.message || String(error)  
    };  
  }  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means runOptionalCommand performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without runOptionalCommand, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside runOptionalCommand show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

result: result stores the completed result of an awaited operation. It is initialized from `await execFileAsync(command, args, {`. Within the function it is accessed properties/methods `stderr`, `stdout`; assigned or updated later in the function.

function powershellSingleQuoted(value = "")

Role: To understand `powershellSingleQuoted`, start with the problem it owns. `powershellSingleQuoted` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function powershellSingleQuoted(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for powershellSingleQuoted

```
function powershellSingleQuoted(value = "") {  
  return `${String(value).replaceAll("'", "'')}``;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `powershellSingleQuoted` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `powershellSingleQuoted`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `powershellSingleQuoted`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

Compiler and IDE workspace

The functions in this group all support compiler and ide workspace. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function sourceLooksCpp(source = "")

Role: Begin with the role of `sourceLooksCpp`. `sourceLooksCpp` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sourceLooksCpp(source = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `sourceLooksCpp`

```
function sourceLooksCpp(source = "") {  
  const text = String(source || "");  
  return /#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|  
variant)>/.test(text) ||  
  /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w|delete\s+|constexpr|nullptr|  
using\s+namespace)\b/.test(text);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceLooksCpp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceLooksCpp`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside `sourceLooksCpp` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: `text` stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into `test`; assigned or updated later in the function.

`function sourceLooksC(source = "")`

Role: Begin with the role of `sourceLooksC`. `sourceLooksC` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sourceLooksC(source = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `sourceLooksC`

```
function sourceLooksC(source = "") {  
  const text = String(source || "");  
  return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
```

```

    /\b(sprintf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means sourceLooksC performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without sourceLooksC, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside sourceLooksC explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into `test`; assigned or updated later in the function.

function detectCodeLanguageFromSource(code = "", fileName = "")

Role: To understand `detectCodeLanguageFromSource`, start with the problem it owns. `detectCodeLanguageFromSource` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Function entry point:

```
function detectCodeLanguageFromSource(code = "", fileName = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for detectCodeLanguageFromSource

```

function detectCodeLanguageFromSource(code = "", fileName = "") {
  const byName = languageFromFileName(fileName, code);
  if (byName) return byName;
  const source = String(code || "");
  if (/<\/?[a-z][\s\S]*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";
  if (/^(\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+\w+)\b/.test(source)) return "systemverilog";
  if (/^(\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(source)) return "verilog";
  if (/^(\s*(?:tran|ac|dc|op|model|subckt|ends|param)\b/im.test(source) || /\bV\w+\s+\w+\s+\w+\s+(?:DC|SIN|PULSE)?/i.test(source)) return "Itpice";
  if (/^(\b(def|elif|from\s+\w+\s+import|self|None|True|False)\b/.test(source)) return "python";
  if (/^(\b(public\s+class|static\s+void\s+main|System\.out)\b/.test(source)) return "java";
  if (sourceLooksCpp(source) || sourceLooksC(source) || /\b(#include|main\s*\(|puts\s*\(|fprintf|sizeof\s*\(|\b/.test(source)) {
    return sourceLooksCpp(source) ? "cpp" : "c";
  }
  if (/^(\b(const|let|var|function|=>|console\.log|document\.|window\.)\b/.test(source)) return "javascript";
  return "javascript";
}

```

```
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with `languageFromFileName`, `sourceLooksC`, `sourceLooksCpp`. Those calls show that `detectCodeLanguageFromSource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `detectCodeLanguageFromSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside `detectCodeLanguageFromSource` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

byName: `byName` stores a computed value for temporary calculation. It is initialized from `languageFromFileName(fileName, code)`. Within the function it is returned to the caller; assigned or updated later in the function.

source: `source` stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is passed into `sourceLooksC`, `sourceLooksCpp`, `test`; assigned or updated later in the function.

async function findTool(toolName)

Role: The best entry point for `findTool` is its responsibility in the surrounding workflow. `findTool` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function findTool(toolName) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `toolName` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for findTool

```
async function findTool(toolName) {
  if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);
  const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
  for (const candidate of candidates) {
    if (path.isAbsolute(candidate)) {
      if (await pathExists(candidate)) {
        compileToolCache.set(toolName, candidate);
        return candidate;
      }
      continue;
    }
  }
  try {
    const command = process.platform === "win32" ? "where" : "command";
    const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
    const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
    const found = String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
    if (found) {
```

```

        compileToolCache.set(toolName, found);
        return found;
    }
} catch {
    // Try the next candidate.
}
}

if (["gcc", "g++"].includes(toolName) && process.env.LOCALAPPDATA) {
    const found = await findExecutableUnder(
        path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages"),
        [toolName === "gcc" ? "gcc.exe" : "g++.exe"],
        7
    );
    compileToolCache.set(toolName, found || "");
    return found || "";
}

if (["javac", "java"].includes(toolName)) {
    const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`], 5);
    compileToolCache.set(toolName, found || "");
    return found || "";
}

compileToolCache.set(toolName, "");
return "";
}
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and runs an external command. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `pathExists`, `findExecutableUnder`. Those calls show that `findTool` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `findTool`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `findTool` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

candidates: `candidates` stores a function or callback value for temporary calculation. It is initialized from `(compileToolCandidates[toolName] || [toolName]).filter(Boolean)`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is returned to the caller; passed into `set`.

command: `command` stores a computed value for temporary calculation. It is initialized from `process.platform === "win32" ? "where" : "command"`. Within the function it is passed into `execFileAsync`; assigned or updated later in the function.

args: `args` stores a computed value for temporary calculation. It is initialized from `process.platform === "win32" ? [candidate] : ["-v", candidate]`. Within the function it is passed into `execFileAsync`; assigned or updated later in the function.

result: `result` stores the completed result of an awaited operation. It is initialized from `await execFileAsync(command, args, { timeout: 5000, windowsHide: true })`. Within the function it is accesses properties/methods `stdout`; passed into `String`; assigned or updated later in the function.

found: found stores a function or callback value for lookup collection. It is initialized from `String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean)`. Within the function it is returned to the caller; passed into set; assigned or updated later in the function.

found: found stores the completed result of an awaited operation. It is initialized from `await findExecutableUnder(. Within the function it is returned to the caller; passed into set; assigned or updated later in the function.`

found: found stores the completed result of an awaited operation. It is initialized from `await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`], 5)`. Within the function it is returned to the caller; passed into set; assigned or updated later in the function.

function processTerminalText(result = {})

Role: Begin with the role of `processTerminalText`. `processTerminalText` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function processTerminalText(result = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for processTerminalText

```
function processTerminalText(result = {}) {  
  const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";  
  const elapsed = elapsedSeconds ? ` in ${elapsedSeconds}s` : "";  
  const status = result.timedOut  
    ? `Timed out after ${elapsedSeconds}s` : "the configured timeout"  
    : `Exit code ${result.code ?? "unknown"}${elapsed}`;  
  const output = [result.stdout, result.stderr].map((part) => String(part ||  
    "").trimEnd()).filter(Boolean).join("\n");  
  return [status, output || (result.ok ? "Completed without diagnostic output." : "No compiler output was  
    returned.")] .join("\n");  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `processTerminalText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `processTerminalText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside `processTerminalText` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

elapsedSeconds: `elapsedSeconds` stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

elapsed: elapsed stores a computed value for temporary calculation. It is initialized from `elapsedSeconds ? ` in {elapsedSeconds}s` : "". Within the function it is assigned or updated later in the function.`

status: status stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

output: output stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is assigned or updated later in the function.

function processTerminalTextWithPaths(result = {}, replacements = [])

Role: Begin with the role of `processTerminalTextWithPaths`. `processTerminalTextWithPaths` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function processTerminalTextWithPaths(result = {}, replacements = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `replacements` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for processTerminalTextWithPaths

```
function processTerminalTextWithPaths(result = {}, replacements = []) {  
  return replacePathReferences(processTerminalText(result), replacements);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `replacePathReferences`, `processTerminalText`. Those calls show that `processTerminalTextWithPaths` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `processTerminalTextWithPaths`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: Inside `processTerminalTextWithPaths`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function cFamilyCompileProfile(language = "c", fileName = "main.c")

Role: The best entry point for `cFamilyCompileProfile` is its responsibility in the surrounding workflow. `cFamilyCompileProfile` belongs to compiler and ide workspace. This function supports the Compile Code

workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cFamilyCompileProfile(language = "c", fileName = "main.c") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `cFamilyCompileProfile`

```
function cFamilyCompileProfile(language = "c", fileName = "main.c") {  
  const isCpp = language === "cpp";  
  const ext = path.extname(String(fileName || "").toLowerCase());  
  const header = [".h", ".hpp", ".hh", ".hxx"].includes(ext);  
  return {  
    compiler: isCpp ? "g++" : "gcc",  
    standard: isCpp ? "-std=c++20" : "-std=c17",  
    languageFlag: isCpp ? "c++" : "c",  
    label: isCpp ? "C++" : "C",  
    header,  
    flags: [  
      isCpp ? "-std=c++20" : "-std=c17",  
      "-Wall",  
      "-Wextra",  
      "-Wpedantic",  
      "-Wshadow",  
      "-O0",  
      "-g3",  
      "-fdiagnostics-color=never"  
    ]  
  };  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `cFamilyCompileProfile` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `cFamilyCompileProfile`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `cFamilyCompileProfile` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

isCpp: `isCpp` stores a computed value for temporary calculation. It is initialized from `language === "cpp"`. Within the function it is assigned or updated later in the function.

ext: `ext` stores a resolved filesystem or route value. It is initialized from `path.extname(String(fileName || "").toLowerCase())`. Within the function it is passed into `includes`; assigned or updated later in the function.

header: header stores a list for temporary calculation. It is initialized from [".h", ".hpp", ".hh", ".hxx"].includes(ext). Within the function it is assigned or updated later in the function.

function cFamilyBinaryName(fileName = "program")

Role: The best entry point for cFamilyBinaryName is its responsibility in the surrounding workflow. cFamilyBinaryName belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cFamilyBinaryName(fileName = "program") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for cFamilyBinaryName

```
function cFamilyBinaryName(fileName = "program") {  
  const parsed = path.parse(String(fileName || "program"));  
  return `${safeSegment(parsed.name, "program")}.exe`;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with safeSegment. Those calls show that cFamilyBinaryName is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without cFamilyBinaryName, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of cFamilyBinaryName is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

parsed: parsed stores a resolved filesystem or route value. It is initialized from path.parse(String(fileName || "program")). Within the function it is accesses properties/methods name; passed into safeSegment; assigned or updated later in the function.

async function toolVersionLine(toolPath = "")

Role: Begin with the role of toolVersionLine. toolVersionLine belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function toolVersionLine(toolPath = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. toolPath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using

it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for toolVersionLine

```
async function toolVersionLine(toolPath = "") {
  if (!toolPath) return "";
  if (compileToolVersionCache.has(toolPath)) return compileToolVersionCache.get(toolPath);
  const result = await runProcess(toolPath, ["--version"], { timeoutMs: 7000 });
  const version = [result.stdout, result.stderr]
    .map((part) => String(part || "").trim())
    .filter(Boolean)
    .join("\n")
    .split(/\r?\n/)
    .map((line) => line.trim())
    .find(Boolean) || path.basename(toolPath);
  compileToolVersionCache.set(toolPath, version);
  return version;
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and runs an external command. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: delegates command execution to runProcess so timeout and terminal capture remain consistent.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with runProcess. Those calls show that toolVersionLine is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without toolVersionLine, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside toolVersionLine explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

result: result stores a resolved filesystem or route value. It is initialized from await runProcess(toolPath, ["--version"], { timeoutMs: 7000 }). Within the function it is accesses properties/methods stderr, stdout; assigned or updated later in the function.

version: version stores a list for temporary calculation. It is initialized from [result.stdout, result.stderr]. Within the function it is returned to the caller; passed into runProcess, set; assigned or updated later in the function.

function cFamilyRunOutput(result = {})

Role: The best entry point for cFamilyRunOutput is its responsibility in the surrounding workflow. cFamilyRunOutput belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cFamilyRunOutput(result = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for cFamilyRunOutput

```
function cFamilyRunOutput(result = {}) {  
  const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";  
  const status = result.timedOut  
    ? `Program timed out after ${elapsedSeconds} s` : "the configured timeout"`  
    : `Program exit code ${result.code ?? "unknown"}${elapsedSeconds} s` : ""`;  
  const output = [result.stdout, result.stderr].map((part) => String(part ||  
    "").trimEnd()).filter(Boolean).join("\n");  
  return [status, output || (result.ok ? "Program completed without output." : "No program output was  
    returned.")].join("\n");  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means cFamilyRunOutput performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without cFamilyRunOutput, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of cFamilyRunOutput is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

elapsedSeconds: elapsedSeconds stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

status: status stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

output: output stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is assigned or updated later in the function.

function cleanHdlSimulationOutput(result = {})

Role: To understand cleanHdlSimulationOutput, start with the problem it owns. cleanHdlSimulationOutput belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cleanHdlSimulationOutput(result = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for cleanHdlSimulationOutput

```
function cleanHdlSimulationOutput(result = {}) {
  const raw = [result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n");
  const lines = raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean);
  const finishLines = [];
  const signalLines = [];
  for (const line of lines) {
    if (/\/$finish called at/i.test(line)) {
      finishLines.push(line.replace(/^.*?:\d+:\s*/, ""));
    } else {
      signalLines.push(line);
    }
  }
  const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
  const status = result.timedOut
    ? `Simulation timed out after ${elapsedSeconds} s : "the configured timeout"`
    : `Simulation exit code ${result.code ?? "unknown"}${elapsedSeconds} s in ${elapsedSeconds} s : ""`;
  const body = signalLines.length
    ? signalLines.join("\n")
    : finishLines.length
    ? finishLines.join("\n")
    : result.ok
    ? "Simulation completed without printed output."
    : "No simulator output was returned.";
  const suffix = signalLines.length && finishLines.length ? `\n${finishLines.join("\n")}` : "";
  return `${status}\n${body}${suffix}`;
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means cleanHdlSimulationOutput performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without cleanHdlSimulationOutput, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside cleanHdlSimulationOutput reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

raw: raw stores a list for lookup collection. It is initialized from [result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n"). Within the function it is accesses properties/methods split; assigned or updated later in the function.

lines: lines stores a function or callback value for lookup collection. It is initialized from raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean). Within the function it is assigned or updated later in the function.

finishLines: finishLines stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods join, length, push; assigned or updated later in the function.

signalLines: signalLines stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods join, length, push; assigned or updated later in the function.

line: line starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods replace, trimEnd; passed into map, push.

elapsedSeconds: elapsedSeconds stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

status: status stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

body: body stores a computed value for temporary calculation. It is initialized from `signalLines.length`. Within the function it is assigned or updated later in the function.

suffix: suffix stores a resolved filesystem or route value. It is initialized from `signalLines.length && finishLines.length ? `\\n${finishLines.join("\\n")}` : ""`. Within the function it is assigned or updated later in the function.

async function runProcess(command, args = [], options = {})

Role: Begin with the role of runProcess. runProcess is the safe wrapper around external commands. Compilers, Git, Winget, Java, vvp, and installer helpers all eventually need child processes. This function starts the command with controlled options, captures stdout and stderr, enforces a timeout, kills process trees on Windows when necessary, and resolves to a structured result instead of letting raw process behavior leak through the API. Without this wrapper, every compiler and Git function would need to repeat timeout logic, stdout/stderr collection, and error handling. More importantly, failures would be inconsistent and the frontend terminal would receive unreliable output.

Function entry point:

```
async function runProcess(command, args = [], options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. command is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. args is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for runProcess

```
async function runProcess(command, args = [], options = {}) {
  const timeoutMs = options.timeoutMs || 20000;
  const cwd = options.cwd || compileRoot;
  return new Promise((resolve) => {
    const startedAt = Date.now();
    const child = spawn(command, args, {
      cwd,
      env: { ...process.env, ...(options.env || {}) },
      shell: false,
      windowsHide: true
    });
    let stdout = "";
    let stderr = "";
    let timedOut = false;
    const timer = setTimeout(() => {
      timedOut = true;
    }, timeoutMs);
  });
}
```



```

    if (process.platform === "win32" && child.pid) {
      execFile("taskkill", ["/pid", String(child.pid), "/T", "/F"], { windowsHide: true }, () => {});
    }
    child.kill("SIGKILL");
  }, timeoutMs);
  child.stdout?.on("data", (chunk) => {
    stdout += chunk.toString();
  });
  child.stderr?.on("data", (chunk) => {
    stderr += chunk.toString();
  });
  if (Object.prototype.hasOwnProperty.call(options, "input")) {
    child.stdin?.write(String(options.input || ""));
  }
  child.stdin?.end();
  child.on("error", (error) => {
    clearTimeout(timer);
    resolve({ ok: false, code: -1, stdout, stderr: `${stderr}\n${error.message}`.trim(), timedOut, elapsedMs:
Date.now() - startedAt });
  });
  child.on("close", (code) => {
    clearTimeout(timer);
    resolve({ ok: !timedOut && code === 0, code, stdout, stderr, timedOut, elapsedMs: Date.now() - startedAt });
  });
});
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it runs an external command and uses a timer to avoid blocking or to wait for another process. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `runProcess` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `runProcess`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside `runProcess` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

timeoutMs: `timeoutMs` stores a computed value for temporary calculation. It is initialized from `options.timeoutMs || 20000`. Within the function it is assigned or updated later in the function.

cwd: `cwd` stores a computed value for temporary calculation. It is initialized from `options.cwd || compileRoot`. Within the function it is passed into `spawn`; assigned or updated later in the function.

startedAt: `startedAt` stores a computed value for temporary calculation. It is initialized from `Date.now()`. Within the function it is assigned or updated later in the function.

child: `child` stores a computed value for temporary calculation. It is initialized from `spawn(command, args, {`. Within the function it accesses properties/methods `kill`, `on`, `pid`, `stderr`, `stdin`, `stdout`; passed into `execFile`; assigned or updated later in the function.

stdout: `stdout` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is passed into `resolve`; assigned or updated later in the function.

stderr: `stderr` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is passed into `resolve`; assigned or updated later in the function.

timedOut: `timedOut` stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is passed into `resolve`; assigned or updated later in the function.

timer: `timer` stores a function or callback value for temporary calculation. It is initialized from `setTimeout(() => {`. Within the function it is passed into `clearTimeout`; assigned or updated later in the function.

async function compileToolStatus()

Role: The best entry point for compileToolStatus is its responsibility in the surrounding workflow. compileToolStatus belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function compileToolStatus() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for compileToolStatus

```
async function compileToolStatus() {  
  const tools = {};  
  const uniqueTools = [...new Set(Object.values(compileLanguageProfiles).flatMap((profile) =>  
    profile.primaryTools))];  
  for (const tool of uniqueTools) {  
    tools[tool] = await findTool(tool);  
  }  
  const languages = {};  
  for (const [id, profile] of Object.entries(compileLanguageProfiles)) {  
    const required = profile.primaryTools;  
    languages[id] = {  
      label: profile.label,  
      defaultFile: profile.defaultFile,  
      extensions: profile.extensions,  
      ready: required.every((tool) => tools[tool]),  
      tools: Object.fromEntries(required.map((tool) => [tool, tools[tool] || "missing"])),  
      installHints: profile.winget || []  
    };  
  }  
  return { compileRoot, languages, tools };  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with findTool. Those calls show that compileToolStatus is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without compileToolStatus, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of compileToolStatus is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

tools: tools stores a structured object for compiler or language configuration. It is initialized from {}. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

uniqueTools: uniqueTools stores a list for compiler or language configuration. It is initialized from [...new Set(Object.values(compileLanguageProfiles).flatMap((profile) => profile.primaryTools))]. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

tool: tool starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It supports the compile workspace by describing language

rules, executable locations, tool status, or compiler output. Within the function it is passed into every, findTool, fromEntries.

languages: languages stores a structured object for compiler or language configuration. It is initialized from {}. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

destructured [id, profile]: destructured [id, profile] starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent.

required: required stores a computed value for temporary calculation. It is initialized from profile.primaryTools. Within the function it is iterated or transformed as a collection; accesses properties/methods every, map; passed into fromEntries; assigned or updated later in the function.

function isHdlLanguage(language = "")

Role: Begin with the role of isHdlLanguage. isHdlLanguage is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Function entry point:

```
function isHdlLanguage(language = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for isHdlLanguage

```
function isHdlLanguage(language = "") {  
  return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with normalizeCodeLanguage. Those calls show that isHdlLanguage is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without isHdlLanguage, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: Inside isHdlLanguage, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function inferCompileFileRole(fileName = "", code = "", language = "")

Role: The best entry point for inferCompileFileRole is its responsibility in the surrounding workflow. inferCompileFileRole makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Function entry point:

```
function inferCompileFileRole(fileName = "", code = "", language = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `inferCompileFileRole`

```
function inferCompileFileRole(fileName = "", code = "", language = "") {  
  if (!isHdlLanguage(language)) return "source";  
  const haystack = `${fileName}\n${code}`.toLowerCase();  
  if (/\\b(tb|testbench)\\b|^|[\\_\\-])tb([\\_\\-]|\\.)|test[\\_\\-]?bench|\\$dumpfile|\\$dumpvars|module\\s+tb[_a-z0-9]*/i.test(haystack)) {  
    return "testbench";  
  }  
  return "design";  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `isHdlLanguage`. Those calls show that `inferCompileFileRole` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `inferCompileFileRole`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `inferCompileFileRole` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

haystack: `haystack` stores a string value for runtime state or cache. It is initialized from ``${fileName}\\n${code}`.toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

`function normalizeCompileFileRole(value = "", language = "")`

Role: Begin with the role of `normalizeCompileFileRole`. `normalizeCompileFileRole` standardizes `CompileFileRole` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCompileFileRole(value = "", language = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain

readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeCompileFileRole

```
function normalizeCompileFileRole(value = "", language = "") {  
  if (!isHdlLanguage(language)) return "source";  
  const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");  
  return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with isHdlLanguage. Those calls show that normalizeCompileFileRole is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without normalizeCompileFileRole, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside normalizeCompileFileRole explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, ""). Within the function it is passed into includes; assigned or updated later in the function.

async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code, stdin = "" })

Role: To understand saveCompileSource, start with the problem it owns. saveCompileSource belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code,  
  stdin = "" }) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. projectId identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. fileId points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. title is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. role is the caller-supplied input used by this function. The

function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. stdin is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for saveCompileSource

```
async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code, stdin = "" }) {
  const lang = normalizeCodeLanguage(language || detectCodeLanguageFromSource(code, fileName));
  const projectFolder = safeSegment(projectId, "project");
  const codeFileName = safeCodeFileName(fileName, lang);
  const fileFolder = safeSegment(fileId || path.parse(codeFileName).name, path.parse(codeFileName).name);
  const folder = resolveInsideCompileRoot(projectFolder, fileFolder);
  const sourcePath = resolveInsideCompileRoot(projectFolder, fileFolder, codeFileName);
  await mkdir(folder, { recursive: true });
  await writeFile(sourcePath, String(code || ""), "utf8");
  if (stdin) await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "stdin.txt"), String(stdin),
    "utf8");
  const metadata = {
    id: fileFolder,
    title: clampText(title || codeFileName, 180),
    fileName: codeFileName,
    language: lang,
    role: normalizeCompileFileRole(role || inferCompileFileRole(codeFileName, code, lang), lang),
    sourcePath,
    savedAt: new Date().toISOString()
  };
  await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "compile-meta.json"), `
${JSON.stringify(metadata, null, 2)}\n`, "utf8");
  return metadata;
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeSegment`, `safeCodeFileName`, `resolveInsideCompileRoot`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `saveCompileSource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `saveCompileSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside `saveCompileSource` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lang: lang stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language || detectCodeLanguageFromSource(code, fileName))`. Within the function it is passed into `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

projectFolder: projectFolder stores a computed value for filesystem location. It is initialized from safeSegment(projectId, "project"). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into resolveInsideCompileRoot, writeFile; assigned or updated later in the function.

codeFileName: codeFileName stores a computed value for temporary calculation. It is initialized from safeCodeFileName(fileName, lang). Within the function it is passed into clampText, normalizeCompileFileRole, parse, resolveInsideCompileRoot, safeSegment; assigned or updated later in the function.

fileFolder: fileFolder stores a resolved filesystem or route value. It is initialized from safeSegment(fileId || path.parse(codeFileName).name, path.parse(codeFileName).name). Within the function it is passed into resolveInsideCompileRoot, writeFile; assigned or updated later in the function.

folder: folder stores a computed value for filesystem location. It is initialized from resolveInsideCompileRoot(projectFolder, fileFolder). Within the function it is passed into mkdir; assigned or updated later in the function.

sourcePath: sourcePath stores a computed value for filesystem location. It is initialized from resolveInsideCompileRoot(projectFolder, fileFolder, codeFileName). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into writeFile; assigned or updated later in the function.

metadata: metadata stores a structured object for temporary calculation. It is initialized from {}. Within the function it is returned to the caller; passed into stringify; assigned or updated later in the function.

function javaMainClassName(code = "", fileName = "Main.java")

Role: Read javaMainClassName first as a named idea, not as syntax. javaMainClassName belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function javaMainClassName(code = "", fileName = "Main.java") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for javaMainClassName

```
function javaMainClassName(code = "", fileName = "Main.java") {  
  const source = String(code || "");  
  const publicMatch = source.match(/bpublic\s+class\s+([A-Za-z_$][\w$]*)/);  
  if (publicMatch) return publicMatch[1];  
  const classMatch = source.match(/bclass\s+([A-Za-z_$][\w$]*)/);  
  return classMatch?.[1] || path.parse(fileName).name || "Main";  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `javaMainClassName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `javaMainClassName`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local objects and variables inside `javaMainClassName` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

source: `source` stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is accesses properties/methods `match`; assigned or updated later in the function.

publicMatch: `publicMatch` stores a computed value for temporary calculation. It is initialized from `source.match(/bpublic\s+class\s+([A-Za-z_$][\w$]*)/)`. Within the function it is returned to the caller; assigned or updated later in the function.

classMatch: `classMatch` stores a computed value for appearance configuration. It is initialized from `source.match(/bclass\s+([A-Za-z_$][\w$]*)/)`. Within the function it is returned to the caller; assigned or updated later in the function.

function compileCacheKey(parts = {})

Role: Read `compileCacheKey` first as a named idea, not as syntax. `compileCacheKey` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function compileCacheKey(parts = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `parts` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for compileCacheKey

```
function compileCacheKey(parts = {}) {  
  return createHash("sha256")  
    .update(JSON.stringify(parts))  
    .digest("hex")  
    .slice(0, 24);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `compileCacheKey` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `compileCacheKey`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: Inside `compileCacheKey`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function compileCacheDirectory(projectId, language, key)

Role: Read `compileCacheDirectory` first as a named idea, not as syntax. `compileCacheDirectory` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function compileCacheDirectory(projectId, language, key) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. `key` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for compileCacheDirectory

```
function compileCacheDirectory(projectId, language, key) {  
  return resolveInsideCompileRoot(safeSegment(projectId, "project"), ".build-cache", safeSegment(language,  
    "language"), key);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `resolveInsideCompileRoot`, `safeSegment`. Those calls show that `compileCacheDirectory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `compileCacheDirectory`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: Inside `compileCacheDirectory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function hdlModuleNames(code = "")

Role: Read `hdlModuleNames` first as a named idea, not as syntax. `hdlModuleNames` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function hdlModuleNames(code = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `hdlModuleNames`

```
function hdlModuleNames(code = "") {  
  return [...String(code || "").matchAll(/\\bmodule\\s+([A-Za-z_\\$][\\w\\$]*)/g)].map((match) => match[1]);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `hdlModuleNames` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `hdlModuleNames`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: Inside `hdlModuleNames`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `hdlHasWaveDump`(`code = ""`)

Role: To understand `hdlHasWaveDump`, start with the problem it owns. `hdlHasWaveDump` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function hdlHasWaveDump(code = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `hdlHasWaveDump`

```
function hdlHasWaveDump(code = "") {  
  const source = String(code || "");  
  return /\\$dumpfile\\s*(\\.test(source) && /\\$dumpvars\\s*(\\.test(source);  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or

scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `hdlHasWaveDump` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `hdlHasWaveDump`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside `hdlHasWaveDump` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

source: `source` stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is passed into `test`; assigned or updated later in the function.

function `hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog")`

Role: To understand `hdlFilesFromPayload`, start with the problem it owns. `hdlFilesFromPayload` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog") { ...
}
```

Inputs: The function-specific inputs are worth reading before the body. `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeLanguage` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `hdlFilesFromPayload`

```
function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog") {
  const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
  const byId = new Map();
  const addFile = (item = {}, fallbackId = "") => {
    const code = String(item.code || "");
    const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName));
    if (!isHdlLanguage(language) || !code.trim()) return;
    const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language].defaultFile, language);
    const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
    byId.set(id, {
      id,
      title: clampText(item.title || fileName, 180),
      fileName,
      language,
      role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language), language),
    });
  };
  for (const item of incoming) addFile(item);
  return [...byId.values()];
}
```

```

        code
    });
});
incoming.forEach((item, index) => addFile(item, `hdl-${index + 1}`));
addFile({
  id: payload.fileId,
  title: payload.title,
  fileName: activeFileName,
  language: activeLanguage,
  role: payload.role,
  code: payload.code
}, "active");
return [...byId.values()].sort((a, b) => {
  if (a.role === b.role) return a.fileName.localeCompare(b.fileName);
  return a.role === "testbench" ? 1 : -1;
});
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `isHdlLanguage`, `safeCodeFileName`, `safeSegment`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `hdlFilesFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `hdlFilesFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside `hdlFilesFromPayload` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

incoming: `incoming` stores a computed value for temporary calculation. It is initialized from `Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

byId: `byId` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `set`, `values`; assigned or updated later in the function.

addFile: `addFile` stores a function or callback value for temporary calculation. It is initialized from `(item = {}, fallbackId = "") => {}`. Within the function it is assigned or updated later in the function.

code: `code` stores a computed value for temporary calculation. It is initialized from `String(item.code || "")`. Within the function it is accesses properties/methods `trim`; passed into `String`, `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

fileName: `fileName` stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(item.fileName || activeFileName || compileLanguageProfiles[language].defaultFile, language)`. Within the function it is accesses properties/methods `localeCompare`; passed into `addFile`, `localeCompare`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`, `safeSegment`, `set`; assigned or updated later in the function.

id: id stores a resolved filesystem or route value. It is initialized from `safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name)`. Within the function it is passed into `addFile`, `safeSegment`, `set`; assigned or updated later in the function.

function compileActionFromPayload(value = "", language = "")

Role: To understand `compileActionFromPayload`, start with the problem it owns. `compileActionFromPayload` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function compileActionFromPayload(value = "", language = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for compileActionFromPayload

```
function compileActionFromPayload(value = "", language = "") {  
  const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "-");  
  if (["compile", "build", "run", "simulate"].includes(clean)) return clean;  
  return isHdlLanguage(language) ? "simulate" : "run";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `isHdlLanguage`. Those calls show that `compileActionFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `compileActionFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside `compileActionFromPayload` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "-")`. Within the function it is returned to the caller; assigned or updated later in the function.

function compileActionLabel(action = "run", language = "")

Role: The best entry point for `compileActionLabel` is its responsibility in the surrounding workflow. `compileActionLabel` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function compileActionLabel(action = "run", language = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `action` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for compileActionLabel

```
function compileActionLabel(action = "run", language = "") {  
  if (action === "compile") return "Compile";  
  if (action === "build") return "Build project";  
  if (action === "simulate") return "Simulate";  
  return isHdlLanguage(language) ? "Simulate" : "Run";  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `isHdlLanguage`. Those calls show that `compileActionLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `compileActionLabel`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: Inside `compileActionLabel`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

```
function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage =  
"javascript")
```

Role: To understand `compileWorkspaceFilesFromPayload`, start with the problem it owns. `compileWorkspaceFilesFromPayload` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage =  
"javascript") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeLanguage` names the programming

or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for compileWorkspaceFilesFromPayload

```
function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "javascript") {
  const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
  const byId = new Map();
  const addFile = (item = {}, fallbackId = "") => {
    const code = String(item.code || "");
    if (!code.trim()) return;
    const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName));
    const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language]?.defaultFile || "source.txt", language);
    const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
    byId.set(id, {
      id,
      title: clampText(item.title || fileName, 180),
      fileName,
      language,
      role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language), language),
      code
    });
  };
  incoming.forEach((item, index) => addFile(item, `workspace-${index + 1}`));
  addFile({
    id: payload.fileId,
    title: payload.title,
    fileName: activeFileName,
    language: activeLanguage,
    role: payload.role,
    code: payload.code
  }, "active");
  return [...byId.values()].sort((a, b) => a.fileName.localeCompare(b.fileName));
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeCodeFileName`, `safeSegment`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `compileWorkspaceFilesFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `compileWorkspaceFilesFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside `compileWorkspaceFilesFromPayload` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

incoming: `incoming` stores a computed value for temporary calculation. It is initialized from `Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods for `Each`; assigned or updated later in the function.

byld: byld stores a constructed object for lookup collection. It is initialized from new Map(). Within the function it is mutated as a collection or DOM container; accesses properties/methods set, values; assigned or updated later in the function.

addFile: addFile stores a function or callback value for temporary calculation. It is initialized from (item = {}, fallbackId = "") => {}. Within the function it is assigned or updated later in the function.

code: code stores a computed value for temporary calculation. It is initialized from String(item.code || ""). Within the function it is accesses properties/methods trim; passed into String, addFile, normalizeCodeLanguage, normalizeCompileFileRole; assigned or updated later in the function.

language: language stores a computed value for compiler or language configuration. It is initialized from normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName)). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into addFile, normalizeCodeLanguage, normalizeCompileFileRole, safeCodeFileName; assigned or updated later in the function.

fileName: fileName stores a computed value for temporary calculation. It is initialized from safeCodeFileName(item.fileName || activeFileName || compileLanguageProfiles[language]?.defaultFile || "source.txt", language). Within the function it is accesses properties/methods localeCompare; passed into addFile, localeCompare, normalizeCodeLanguage, normalizeCompileFileRole, safeCodeFileName, safeSegment, set; assigned or updated later in the function.

id: id stores a resolved filesystem or route value. It is initialized from safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name). Within the function it is passed into addFile, safeSegment, set; assigned or updated later in the function.

async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {})

Role: The best entry point for writeCompileWorkspaceSources is its responsibility in the surrounding workflow. writeCompileWorkspaceSources writes CompileWorkspaceSources to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Function entry point:

```
async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. files points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. targetDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for writeCompileWorkspaceSources

```
async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) {  
  await mkdir(targetDir, { recursive: true });  
  const seen = new Map();  
  const written = [];  
  for (const file of files) {  
    if (options.languages?.length && !options.languages.includes(normalizeCodeLanguage(file.language))) continue;  
  }  
}
```

```

    if (options.extensions?.length && !options.extensions.includes(path.extname(file.fileName).toLowerCase()))
    continue;
    const parsed = path.parse(file.fileName);
    const count = seen.get(file.fileName) || 0;
    seen.set(file.fileName, count + 1);
    const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
    const sourcePath = path.join(targetDir, uniqueName);
    await writeFile(sourcePath, file.code, "utf8");
    written.push({ ...file, uniqueName, sourcePath });
  }
  return written;
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`. Those calls show that `writeCompileWorkspaceSources` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `writeCompileWorkspaceSources`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `writeCompileWorkspaceSources` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `get`, `set`; assigned or updated later in the function.

written: `written` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

file: `file` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `code`, `fileName`, `language`; passed into `get`, `parse`, `push`, `set`, `writeFile`.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(file.fileName)`. Within the function it is accesses properties/methods `ext`, `name`; assigned or updated later in the function.

count: `count` stores a computed value for temporary calculation. It is initialized from `seen.get(file.fileName) || 0`. Within the function it is passed into `set`; assigned or updated later in the function.

uniqueName: `uniqueName` stores a computed value for temporary calculation. It is initialized from `count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName`. Within the function it is passed into `join`, `push`; assigned or updated later in the function.

sourcePath: `sourcePath` stores a resolved filesystem or route value. It is initialized from `path.join(targetDir, uniqueName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`, `writeFile`; assigned or updated later in the function.

function sourceDisplayName(file = {})

Role: Begin with the role of `sourceDisplayName`. `sourceDisplayName` belongs to `compiler` and `ide` workspace. This function supports the `Compile Code` workspace: detecting languages, writing files, finding tools,

running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sourceDisplayName(file = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `file` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `sourceDisplayName`

```
function sourceDisplayName(file = {}) {  
  return file.uniqueName || file.fileName || path.basename(file.sourcePath || "source");  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceDisplayName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceDisplayName`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: Inside `sourceDisplayName`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "")`

Role: Begin with the role of `cFamilyWorkspaceSources`. `cFamilyWorkspaceSources` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `files` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string*

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for cFamilyWorkspaceSources

```
function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") {  
  const exts = language === "cpp"  
    ? new Set([".cpp", ".cc", ".cxx", ".c++", ".hpp", ".hh", ".hxx", ".h"])  
    : new Set([".c", ".h"]);  
  const selected = files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase()));  
  if (!selected.some((file) => file.fileName === activeFileName)) {  
    selected.push(...files.filter((file) => file.fileName === activeFileName));  
  }  
  return selected;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means cFamilyWorkspaceSources performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without cFamilyWorkspaceSources, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside cFamilyWorkspaceSources explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

exts: exts stores a computed value for temporary calculation. It is initialized from language === "cpp". Within the function it is accesses properties/methods has; assigned or updated later in the function.

selected: selected stores a function or callback value for temporary calculation. It is initialized from files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase())). Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push, some; assigned or updated later in the function.

async function writeHdlSimulationSources(files = [], cacheDir = "")

Role: Begin with the role of writeHdlSimulationSources. writeHdlSimulationSources writes HdlSimulationSources to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Function entry point:

```
async function writeHdlSimulationSources(files = [], cacheDir = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. files points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. cacheDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for writeHdlSimulationSources

```
async function writeHdlSimulationSources(files = [], cacheDir = "") {
  const sourceDir = path.join(cacheDir, "sources");
  await rm(sourceDir, { recursive: true, force: true });
  await mkdir(sourceDir, { recursive: true });
  const seen = new Map();
  const written = [];
  for (const file of files) {
    const parsed = path.parse(file.fileName);
    const count = seen.get(file.fileName) || 0;
    seen.set(file.fileName, count + 1);
    const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
    const sourcePath = path.join(sourceDir, uniqueName);
    await writeFile(sourcePath, file.code, "utf8");
    written.push({ ...file, uniqueName, sourcePath });
  }
  return written;
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means writeHdlSimulationSources performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without writeHdlSimulationSources, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside writeHdlSimulationSources explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

sourceDir: sourceDir stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, "sources"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into join, mkdir, rm; assigned or updated later in the function.

seen: seen stores a constructed object for lookup collection. It is initialized from new Map(). Within the function it is mutated as a collection or DOM container; accesses properties/methods get, set; assigned or updated later in the function.

written: written stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

file: file starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods code, fileName; passed into get, parse, push, set, writeFile.

parsed: parsed stores a resolved filesystem or route value. It is initialized from path.parse(file.fileName). Within the function it is accesses properties/methods ext, name; assigned or updated later in the function.

count: count stores a computed value for temporary calculation. It is initialized from seen.get(file.fileName) || 0. Within the function it is passed into set; assigned or updated later in the function.

uniqueName: uniqueName stores a computed value for temporary calculation. It is initialized from count ? `\${parsed.name}\_\${count}\${parsed.ext}` : file.fileName. Within the function it is passed into join, push; assigned or updated later in the function.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from path.join(sourceDir, uniqueName). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push, writeFile; assigned or updated later in the function.

function normalizeVcdValue(raw = "")

Role: The best entry point for normalizeVcdValue is its responsibility in the surrounding workflow. normalizeVcdValue standardizes VcdValue data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeVcdValue(raw = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. raw is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeVcdValue

```
function normalizeVcdValue(raw = "") {  
  const clean = String(raw || "").trim();  
  if (!clean) return "x";  
  if (/^[01xz]$/i.test(clean)) return clean.toLowerCase();  
  if (/^[br][01xz_]+$/i.test(clean)) return clean.slice(1).replace(/_/g, "").toLowerCase();  
  return clean.toLowerCase();  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means normalizeVcdValue performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without normalizeVcdValue, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of normalizeVcdValue is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from String(raw || "").trim(). Within the function it is returned to the caller; accesses properties/methods slice, toLowerCase; assigned or updated later in the function.

function parseVcdScopeText(text = "", source = "waveform.vcd")

Role: The best entry point for parseVcdScopeText is its responsibility in the surrounding workflow. parseVcdScopeText reads VCD waveform text from HDL simulation and converts it into signals, time points, values, and scope metadata that a frontend scope viewer can draw. VCD files are compact event logs, not friendly UI data. This parser turns symbol changes over time into understandable signal histories. The function is essential for the SystemVerilog/Verilog simulator feature. A simulation that produces only text is useful, but a

scope view requires structured waveform data. Without this parser, the scope icon would have no signal timeline to show.

Function entry point:

```
function parseVcdScopeText(text = "", source = "waveform.vcd") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `parseVcdScopeText`

```
function parseVcdScopeText(text = "", source = "waveform.vcd") {
  const lines = String(text || "").split(/\r?\n/);
  const scopes = [];
  const signalsByCode = new Map();
  const timeScaleParts = [];
  let inTimescale = false;
  let time = 0;
  let maxTime = 0;
  let definitionDone = false;
  for (const rawLine of lines) {
    const line = rawLine.trim();
    if (!line) continue;
    if (line.startsWith("$timescale")) {
      inTimescale = true;
      const inline = line.replace("$timescale", "").replace("$end", "").trim();
      if (inline) timeScaleParts.push(inline);
      continue;
    }
    if (inTimescale) {
      if (line === "$end") {
        inTimescale = false;
      } else {
        timeScaleParts.push(line.replace("$end", "").trim());
        if (line.includes("$end")) inTimescale = false;
      }
      continue;
    }
    const scopeMatch = line.match(/^$scope\s+\S+\s+(.+?)\s+\$end$/);
    if (scopeMatch) {
      scopes.push(scopeMatch[1]);
      continue;
    }
    if (/^$upscope\b/.test(line)) {
      scopes.pop();
      continue;
    }
    const varMatch = line.match(/^$var\s+\S+\s+(\d+)\s+(\S+)\s+(.+?)\s+\$end$/);
    if (varMatch) {
      const [, width, code, reference] = varMatch;
      if (!signalsByCode.has(code) && signalsByCode.size < 64) {
        const cleanReference = reference.replace(/\s+\[([^\]]+)\]/, "");
        signalsByCode.set(code, {
          code,
          name: [...scopes, cleanReference].filter(Boolean).join("."),
          reference: cleanReference,
          width: Number(width) || 1,
          changes: []
        });
      }
      continue;
    }
    if (line.startsWith("$enddefinitions")) {
```



```

    definitionDone = true;
    continue;
  }
  if (!definitionDone) continue;
  if (line.startsWith("#")) {
    time = Number(line.slice(1)) || 0;
    maxTime = Math.max(maxTime, time);
    continue;
  }
  let code = "";
  let value = "";
  const vectorMatch = line.match(/^(br)[01xz_]+\s+(\S+)$/i);
  if (vectorMatch) {
    value = normalizeVcdValue(vectorMatch[1]);
    code = vectorMatch[2];
  } else if (/^[01xz]/i.test(line)) {
    value = normalizeVcdValue(line[0]);
    code = line.slice(1).trim();
  }
  const signal = signalsByCode.get(code);
  if (!signal || !value) continue;
  const previous = signal.changes[signal.changes.length - 1];
  if (!previous || previous.value !== value || previous.time !== time) {
    if (signal.changes.length < 600) signal.changes.push({ time, value });
  }
}
const signals = [...signalsByCode.values()]
  .filter((signal) => signal.changes.length)
  .slice(0, 32);
return {
  source: path.basename(source),
  timeScale: timeScaleParts.join(" ").replace(/\s+/g, " ").trim() || "ticks",
  maxTime,
  signals
};
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeVcdValue`. Those calls show that `parseVcdScopeText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next steps runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `parseVcdScopeText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `parseVcdScopeText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

lines: `lines` stores a computed value for temporary calculation. It is initialized from `String(text || "").split(/r?\n/)`. Within the function it is assigned or updated later in the function.

scopes: `scopes` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `set`; assigned or updated later in the function.

signalsByCode: `signalsByCode` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `get`, `has`, `set`, `size`, `values`; assigned or updated later in the function.

timeScaleParts: `timeScaleParts` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

inTimescale: inTimescale stores a boolean value for temporary calculation. It is initialized from false. Within the function it is assigned or updated later in the function.

time: time stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into max, push; assigned or updated later in the function.

maxTime: maxTime stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into max; assigned or updated later in the function.

definitionDone: definitionDone stores a boolean value for temporary calculation. It is initialized from false. Within the function it is assigned or updated later in the function.

rawLine: rawLine starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods trim.

line: line stores a computed value for temporary calculation. It is initialized from rawLine.trim(). Within the function it is accesses properties/methods includes, match, replace, slice, startsWith; passed into Number, normalizeVcdValue, push; assigned or updated later in the function.

inline: inline stores a computed value for temporary calculation. It is initialized from line.replace("\$timescale", "").replace("\$end", "").trim(). Within the function it is passed into push; assigned or updated later in the function.

scopeMatch: scopeMatch stores a computed value for temporary calculation. It is initialized from line.match(/^\sscope\s+\S+\s+(\.+?)\s+\\$end\$/). Within the function it is passed into push; assigned or updated later in the function.

varMatch: varMatch stores a computed value for temporary calculation. It is initialized from line.match(/^\\$var\s+\S+\s+(\d+)\s+(\S+)\s+(\.+?)\s+\\$end\$/). Within the function it is assigned or updated later in the function.

destructured [, width, code, reference]: destructured [, width, code, reference] stores a computed value for imported or unpacked API handles. It is initialized from varMatch. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

cleanReference: cleanReference stores a computed value for temporary calculation. It is initialized from reference.replace(/s+\[([^\]]+)\]/, ""). Within the function it is passed into set; assigned or updated later in the function.

code: code stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into get, set; assigned or updated later in the function.

value: value stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into push; assigned or updated later in the function.

vectorMatch: vectorMatch stores a computed value for temporary calculation. It is initialized from line.match(/^(
[01xz_]+)\s+(\S+)\\$/i). Within the function it is passed into normalizeVcdValue; assigned or updated later in the function.

signal: signal stores a computed value for temporary calculation. It is initialized from signalsByCode.get(code). Within the function it is accesses properties/methods changes; passed into filter; assigned or updated later in the function.

previous: previous stores a computed value for temporary calculation. It is initialized from signal.changes[signal.changes.length - 1]. Within the function it is accesses properties/methods time, value; assigned or updated later in the function.

signals: signals stores a list for temporary calculation. It is initialized from [...signalsByCode.values()]. Within the function it is assigned or updated later in the function.

async function readHdlWaveform(cacheDir = "")

Role: The best entry point for readHdlWaveform is its responsibility in the surrounding workflow. readHdlWaveform reads HdlWaveform from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Function entry point:

```
async function readHdlWaveform(cacheDir = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. cacheDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for readHdlWaveform

```
async function readHdlWaveform(cacheDir = "") {  
  const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);  
  if (!vcdFiles.length) return null;  
  const selected = vcdFiles.sort((a, b) => a.length - b.length)[0];  
  const text = await readFile(selected, "utf8");  
  const limited = text.length > 6_000_000 ? text.slice(0, 6_000_000) : text;  
  return parseVcdScopeText(limited, selected);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with findFilesByExtension, parseVcdScopeText. Those calls show that readHdlWaveform is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without readHdlWaveform, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of readHdlWaveform is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

vcdFiles: vcdFiles stores the completed result of an awaited operation. It is initialized from await findFilesByExtension(cacheDir, ".vcd", 4). Within the function it is iterated or transformed as a collection; accesses properties/methods length, sort; assigned or updated later in the function.

selected: selected stores a function or callback value for temporary calculation. It is initialized from vcdFiles.sort((a, b) => a.length - b.length)[0]. Within the function it is passed into parseVcdScopeText, readFile; assigned or updated later in the function.

text: text stores the completed result of an awaited operation. It is initialized from await readFile(selected, "utf8"). Within the function it is accesses properties/methods length, slice; assigned or updated later in the function.

limited: limited stores a computed value for temporary calculation. It is initialized from text.length > 6_000_000 ? text.slice(0, 6_000_000) : text. Within the function it is passed into parseVcdScopeText; assigned or updated later in the function.

async function clearHdlWaveforms(cacheDir = "")

Role: To understand clearHdlWaveforms, start with the problem it owns. clearHdlWaveforms belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function clearHdlWaveforms(cacheDir = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. cacheDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for clearHdlWaveforms

```
async function clearHdlWaveforms(cacheDir = "") {  
  const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);  
  await Promise.all(vcdFiles.map((filePath) => rm(filePath, { force: true })).catch(() => {}));  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Next, trace the helper calls that surround the function. It works with findFilesByExtension. Those calls show that clearHdlWaveforms is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without clearHdlWaveforms, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside clearHdlWaveforms reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

vcdFiles: vcdFiles stores the completed result of an awaited operation. It is initialized from await findFilesByExtension(cacheDir, ".vcd", 4). Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into all; assigned or updated later in the function.

async function compileAndRunCode(payload = {})

Role: Begin with the role of compileAndRunCode. This is the central Compile Code brain. It receives the project id, active source file, language, action requested by the user, optional stdin, and any workspace files that belong to the project. From that one payload it decides whether the user is only beautifying code, saving code, compiling, building, running, simulating HDL, or reading waveform output. It is async because every serious action here touches the filesystem, discovers tools, writes temporary workspaces, runs external compiler processes, and waits for terminal output. The function is needed because the builder treats each project like a small IDE workspace. JavaScript, Python, Java, C, C++, Verilog, SystemVerilog, LTspice, and HTML all have different compile/run rules. This function keeps those rules behind one API endpoint so the frontend can ask for a compile operation without knowing every compiler command. Without it, Compile Code would degrade into a text box with no reliable build, run, simulate, cache, or output behavior. Important internal helpers include ensureRunSource, append, and missing. ensureRunSource writes the active source into the compile workspace

before a run. `append` converts command results into terminal-style output. `missing` returns a helpful response when a required tool is unavailable. Those small local functions matter because they keep the larger language branches readable.

Function entry point:

```
async function compileAndRunCode(payload = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `compileAndRunCode`

```
async function compileAndRunCode(payload = {}) {
  const requestedLanguage = String(payload.language || "").trim().toLowerCase();
  const language = requestedLanguage && requestedLanguage !== "auto"
    ? normalizeCodeLanguage(requestedLanguage)
    : detectCodeLanguageFromSource(payload.code, payload.fileName);
  const profile = compileLanguageProfiles[language] || compileLanguageProfiles.javascript;
  const fileName = safeCodeFileName(payload.fileName, language);
  const saved = await saveCompileSource({ ...payload, language, fileName });
  const projectFolder = safeSegment(payload.projectId, "project");
  const sourcePath = resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName);
  const sourceCode = String(payload.code || "");
  const action = compileActionFromPayload(payload.action, language);
  const allWorkspaceFiles = compileWorkspaceFilesFromPayload(payload, saved.fileName, language);
  let runDir = "";
  let runSourcePath = "";
  let runWorkspaceSources = [];
  const ensureRunSource = async (options = {}) => {
    if (runDir && runSourcePath) return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
    const runId = `${Date.now()}-${Math.random().toString(16).slice(2)}`;
    runDir = resolveInsideCompileRoot(projectFolder, ".runs", runId);
    await mkdir(runDir, { recursive: true });
    const filesToWrite = Array.isArray(options.files) && options.files.length ? options.files : allWorkspaceFiles;
    runWorkspaceSources = await writeCompileWorkspaceSources(filesToWrite, runDir, {
      languages: options.languages,
      extensions: options.extensions
    });
  };
  const active = runWorkspaceSources.find((file) => file.id === saved.id || file.fileName === saved.fileName);
  runSourcePath = active?.sourcePath || path.join(runDir, saved.fileName);
  if (!active) {
    await cp(sourcePath, runSourcePath, { force: true });
    runWorkspaceSources.push({
      id: saved.id,
      title: saved.title,
      fileName: saved.fileName,
      language,
      role: saved.role,
      code: sourceCode,
      uniqueName: saved.fileName,
      sourcePath: runSourcePath
    });
  }
  return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
};

const stdin = String(payload.stdin || "");
const forceRebuild = payload.forceRebuild === true;
const terminal = [];
terminal.push(`${compileActionLabel(action, language)} request for ${saved.fileName}`);
const append = (label, result) => {
  terminal.push(terminalLine(label, processTerminalText(result)));
};
const missing = async (tools) => {
  const found = {};
  for (const tool of tools) found[tool] = await findTool(tool);
  const missingTools = tools.filter((tool) => !found[tool]);
```

```

    return { found, missingTools };
};

if (language === "html") {
    const openTags = (String(payload.code || "").match(/<([a-z][\w:-]*)\b(?:>|"/gi) || []).length;
    const closeTags = (String(payload.code || "").match(/<\/([a-z][\w:-]*)>/gi) || []).length;
    const ok = openTags >= closeTags;
    return {
        ok,
        language,
        saved,
        terminal: [
            "HTML validation pass",
            ok ? "No obvious tag-balance issue was detected." : "The file may have more closing tags than opening
tags.",
            `Saved source: ${saved.sourcePath}`
        ].join("\n")
    };
}

const stdinOptions = stdin ? { input: stdin } : {};
let ok = false;

if (language === "javascript") {
    const node = await findTool("node");
    if (!node) return { ok: false, language, saved, terminal: "Node.js was not found. Install Node.js to run
JavaScript." };
    const jsFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "javascript");
    const runSource = await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles, languages:
["javascript"] });
    const targets = action === "build"
        ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "javascript")
        : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
    ok = true;
    for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName:
saved.fileName }]) {
        const check = await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000
});
        terminal.push(terminalLine(`${path.basename(node)} --check ${sourceDisplayName(target)}`,
processTerminalText(check)));
        ok = ok && check.ok;
    }
    if (action === "compile" || action === "build") {
        terminal.push(ok ? "JavaScript syntax check completed." : "JavaScript syntax check found errors.");
    } else if (ok) {
        const run = await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000,
...stdinOptions });
        append(`${path.basename(node)} ${saved.fileName}`, run);
        ok = run.ok;
    }
} else if (language === "python") {
    const python = await findTool("python");
    if (!python) return { ok: false, language, saved, terminal: "Python was not found. Install Python to run Python
code." };
    const pyFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "python");
    const runSource = await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles, languages:
["python"] });
    const targets = action === "build"
        ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "python")
        : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
    ok = true;
    for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName:
saved.fileName }]) {
        const check = await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd: runSource.runDir,
timeoutMs: 15000 });
        terminal.push(terminalLine(`${path.basename(python)} -m py_compile ${sourceDisplayName(target)}`,
processTerminalText(check)));
        ok = ok && check.ok;
    }
    if (action === "compile" || action === "build") {
        terminal.push(ok ? "Python bytecode compile check completed." : "Python compile check found errors.");
    } else if (ok) {
        const run = await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000,
...stdinOptions });
        append(`${path.basename(python)} ${saved.fileName}`, run);
        ok = run.ok;
    }
}

```



```

} else if (language === "c" || language === "cpp") {
  const cProfile = cFamilyCompileProfile(language, saved.fileName);
  const toolName = cProfile.compiler;
  const { found, missingTools } = await missing([toolName]);
  if (missingTools.length) {
    return { ok: false, language, saved, terminal: `${cProfile.label} compiler missing: ${missingTools.join(", ")}.
Install WinLibs/MinGW or add it to PATH.` };
  }
  const workspaceSources = cFamilyWorkspaceSources(allWorkspaceFiles, language, saved.fileName);
  const buildTargets = action === "build"
    ? workspaceSources.filter((file) => !cFamilyCompileProfile(language, file.fileName).header)
    : workspaceSources.filter((file) => file.fileName === saved.fileName);
  const compilerVersion = await toolVersionLine(found[toolName]);
  const cacheKey = compileCacheKey({
    language,
    action,
    activeFileName: saved.fileName,
    files: buildTargets.map((file) => ({ fileName: file.fileName, code: file.code })),
    compiler: found[toolName],
    compilerVersion,
    flags: cProfile.flags,
    header: cProfile.header
  });
  const cacheDir = compileCacheDirectory(payload.projectId, language, cacheKey);
  await mkdir(cacheDir, { recursive: true });
  const workspaceDir = path.join(cacheDir, "sources");
  await rm(workspaceDir, { recursive: true, force: true });
  const writtenSources = await writeCompileWorkspaceSources(workspaceSources, workspaceDir);
  const activeWritten = writtenSources.find((file) => file.fileName === saved.fileName) || writtenSources[0];
  const binaryName = cFamilyBinaryName(saved.fileName);
  const output = path.join(cacheDir, binaryName);
  const outputMarker = path.join(cacheDir, "syntax-ok.txt");
  const cached = !forceRebuild && await pathExists(output);
  const cachedHeader = cProfile.header && !forceRebuild && await pathExists(outputMarker);
  terminal.push(`${cProfile.label} build profile: ${cProfile.standard}, warnings enabled, debug symbols enabled`);
  terminal.push(`Workspace sources visible to compiler: ${writtenSources.map(sourceDisplayName).join(", ") ||
saved.fileName}`);
  if (compilerVersion) terminal.push(`Compiler: ${compilerVersion}`);
  if (cached || cachedHeader) {
    terminal.push(cachedBuildLine(cProfile.header ? `${cProfile.label} header syntax check` : `${cProfile.label}
binary`, cProfile.header ? outputMarker : output));
  } else {
    const args = cProfile.header
      ? ["-x", cProfile.languageFlag, "-fsyntax-only", ...cProfile.flags, activeWritten?.sourcePath || sourcePath]
      : [
        ...(action === "build"
          ? writtenSources.filter((file) => !cFamilyCompileProfile(language, file.fileName).header).map((file)
=> file.sourcePath)
          : [activeWritten?.sourcePath || sourcePath]),
        "-o",
        output,
        ...cProfile.flags
      ];
    const compile = await runProcess(found[toolName], args, { cwd: workspaceDir, timeoutMs: 30000 });
    const replacements = [
      { from: sourcePath, to: saved.fileName },
      { from: output, to: binaryName },
      ...writtenSources.map((file) => ({ from: file.sourcePath, to: sourceDisplayName(file) }))
    ];
    terminal.push(terminalLine(
      `${path.basename(found[toolName])} ${action === "build" ? "workspace sources" : saved.fileName} $
${cProfile.header ? "-fsyntax-only" : "-o"} ${binaryName}` + `${cProfile.flags.join(" ")}`,
      processTerminalTextWithPaths(compile, replacements)
    ));
    ok = compile.ok;
    if (ok && cProfile.header) {
      await writeFile(outputMarker, `${cProfile.label} header syntax check passed for ${saved.fileName}\n`,
"utf8");
    }
  }
  if (cProfile.header && (cachedHeader || ok)) {
    terminal.push(`Header syntax check passed: ${saved.fileName}`);
    ok = true;
  } else if (cached || ok) {
    terminal.push(`Generated binary: ${output}`);
    if (action === "run") {
      const toolPath = found[toolName];
      const run = await runProcess(output, [], {

```

```

        cwd: cacheDir,
        timeoutMs: 20000,
        env: { PATH: `${path.dirname(toolPath)}${path.delimiter}${process.env.PATH || ""}` },
        ...stdinOptions
    });
    terminal.push(terminalLine(binaryName, cFamilyRunOutput(run)));
    ok = run.ok;
} else {
    terminal.push(`${compileActionLabel(action, language)} completed. Use Run to execute the generated
binary.`);
    ok = true;
}
}
} else if (language === "java") {
    const { found, missingTools } = await missing(["javac", "java"]);
    if (missingTools.length) {
        return { ok: false, language, saved, terminal: `Java tools missing: ${missingTools.join(", ")}. Install a JDK
or add it to PATH.` };
    }
    const className = javaMainClassName(payload.code, saved.fileName);
    const javaFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "java");
    const cacheKey = compileCacheKey({
        language,
        fileName: saved.fileName,
        files: javaFiles.map((file) => ({ fileName: file.fileName, code: file.code })),
        compiler: found.javac,
        runtime: found.java,
        className
    });
    const cacheDir = compileCacheDirectory(payload.projectId, language, cacheKey);
    await mkdir(cacheDir, { recursive: true });
    const classPath = path.join(cacheDir, `${className}.class`);
    const sourceDir = path.join(cacheDir, "sources");
    await rm(sourceDir, { recursive: true, force: true });
    const writtenSources = await writeCompileWorkspaceSources(javaFiles.length ? javaFiles : allWorkspaceFiles,
sourceDir, { languages: ["java"] });
    const javaPath = writtenSources.find((file) => file.fileName === saved.fileName)?.sourcePath ||
path.join(sourceDir, `${className}.java`);
    const cached = !forceRebuild && await pathExists(classPath);
    terminal.push(`Java workspace sources: ${writtenSources.map(sourceDisplayName).join(", ") || saved.fileName}`);
    if (cached) {
        terminal.push(cachedBuildLine("Java class", classPath));
    } else {
        if (!writtenSources.length) await writeFile(javaPath, sourceCode, "utf8");
        const compileTargets = action === "build" ? writtenSources.map((file) => file.sourcePath) : [javaPath];
        const compile = await runProcess(found.javac, ["-d", cacheDir, ...compileTargets], { cwd: sourceDir,
timeoutMs: 30000 });
        terminal.push(terminalLine(`${path.basename(found.javac)} ${action === "build" ? "workspace sources" :
path.basename(javaPath)}`, processTerminalTextWithPaths(
            compile,
            writtenSources.map((file) => ({ from: file.sourcePath, to: sourceDisplayName(file) })))
        ));
        ok = compile.ok;
    }
    if ((cached || ok) && action !== "compile" && action !== "build") {
        terminal.push(`Generated class: ${classPath}`);
        const run = await runProcess(found.java, ["-cp", cacheDir, className], { cwd: cacheDir, timeoutMs: 20000,
...stdinOptions });
        append(`${path.basename(found.java)} ${className}`, run);
        ok = run.ok;
    } else if (cached || ok) {
        terminal.push(`${compileActionLabel(action, language)} completed. Use Run to execute ${className}`);
        ok = true;
    }
} else if (language === "verilog" || language === "systemverilog") {
    const { found, missingTools } = await missing(["iverilog", "vvp"]);
    if (missingTools.length) {
        return { ok: false, language, saved, terminal: `${profile.label} simulator tools missing: $
${missingTools.join(", ")}. Install Icarus Verilog and add iverilog/vvp to PATH.` };
    }
    const hdlFiles = hdlFilesFromPayload(payload, saved.fileName, language);
    const testbenchFiles = hdlFiles.filter((file) => file.role === "testbench");
    if (action === "simulate" && !testbenchFiles.length) {
        return {
            ok: false,
            language,
            saved,
            terminal: [

```

```

    `${profile.label} simulation requires a testbench.`;
    "Mark one HDL file as Testbench or click Add testbench, connect it to the design under test, then run
again.",
    "A design-only file can be saved and appended to the portfolio, but simulation needs stimulus from a
testbench."
  ].join("\n")
  };
}
if (action === "simulate" && !testbenchFiles.some((file) => hdlHasWaveDump(file.code))) {
  return {
    ok: false,
    language,
    saved,
    terminal: [
      "HDL scope requires waveform dump calls in the testbench.",
      'Add $dumpfile("waveform.vcd"); and $dumpvars(0, tb_module_name); inside an initial block.',
      "The simulator will not run without a waveform dump because there would be no signal-over-time data for
the scope."
    ].join("\n")
  };
}
const usesSystemVerilog = hdlFiles.some((file) => normalizeCodeLanguage(file.language) === "systemverilog");
const flags = usesSystemVerilog ? ["-g2012", "-Wall"] : ["-g2005-sv", "-Wall"];
const topModule = testbenchFiles.flatMap((file) => hdlModuleNames(file.code)).find(Boolean) || "";
const cacheKey = compileCacheKey({
  language: usesSystemVerilog ? "systemverilog" : "verilog",
  files: hdlFiles.map((file) => ({
    fileName: file.fileName,
    language: file.language,
    role: file.role,
    code: file.code
  })),
  compiler: found.iverilog,
  runtime: found.vvp,
  flags,
  topModule
});
const cacheDir = compileCacheDirectory(payload.projectId, language, cacheKey);
await mkdir(cacheDir, { recursive: true });
const output = path.join(cacheDir, "simulation.vvp");
const sourceFiles = await writeHdlSimulationSources(hdlFiles, cacheDir);
const designCount = sourceFiles.filter((file) => file.role !== "testbench").length;
terminal.push(`${profile.label} simulation set: ${designCount} design file${designCount === 1 ? "" : "s"}, $
{testbenchFiles.length} testbench file${testbenchFiles.length === 1 ? "" : "s"}`);
if (topModule) terminal.push(`Testbench top: ${topModule}`);
const cached = !forceRebuild && await pathExists(output);
if (cached) {
  terminal.push(cachedBuildLine(`${profile.label} simulation`, output));
} else {
  const compileArgs = [
    ...flags,
    ...(topModule ? ["-s", topModule] : []),
    "-o",
    output,
    ...sourceFiles.map((file) => file.sourcePath)
  ];
  const compile = await runProcess(found.iverilog, compileArgs, { cwd: cacheDir, timeoutMs: 30000 });
  const replacements = sourceFiles.map((file) => ({ from: file.sourcePath, to: file.uniqueName }));
  terminal.push(terminalLine(
    `${path.basename(found.iverilog)} ${flags.join(" ")}${topModule ? ` -s ${topModule}` : ""} -o
simulation.vvp`,
    processTerminalTextWithPaths(compile, replacements)
  ));
  ok = compile.ok;
}
let waveform = null;
if (cached || ok) {
  terminal.push(`Generated simulation: ${output}`);
  if (action === "compile" || action === "build") {
    terminal.push(`${compileActionLabel(action, language)} completed. Use Simulate to run the generated
simulation and produce scope data.`);
  }
  return {
    ok: true,
    language,
    saved,
    waveform: null,
    terminal: terminal.join("\n\n").trim() || "No compiler output was returned."
  };
}

```

```

    }
    await clearHdlWaveforms(cacheDir);
    const run = await runProcess(found.vvp, [output], { cwd: cacheDir, timeoutMs: 20000 });
    terminal.push(terminalLine(`${path.basename(found.vvp)} simulation.vvp`, cleanHdlSimulationOutput(run)));
    waveform = run.ok ? await readHdlWaveform(cacheDir) : null;
    if (waveform?.signals?.length) {
        terminal.push(`Scope waveform: ${waveform.source} (${waveform.signals.length} signals, 0 to ${
            waveform.maxTime} ${waveform.timeScale})`);
    } else {
        terminal.push("Scope waveform was not generated. Confirm the testbench executes $dumpfile and $dumpvars
before $finish.");
    }
    ok = run.ok && Boolean(waveform?.signals?.length);
    return {
        ok,
        language,
        saved,
        waveform,
        terminal: terminal.join("\n\n").trim() || "No compiler output was returned."
    };
}
} else if (language === "ltspace") {
    const ltspace = await findTool("ltspace");
    if (!ltspace) {
        return { ok: false, language, saved, terminal: "LTspace executable was not found. Install LTspace or set
LTSPACE_EXE to the executable path." };
    }
    const runSource = await ensureRunSource();
    const run = await runProcess(ltspace, ["-b", runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 45000
});
    append(`${path.basename(ltspace)} -b ${saved.fileName}`, run);
    const logPath = runSource.runSourcePath.replace(/\.(\.)*$/, ".log");
    if (await pathExists(logPath)) terminal.push(await readFile(logPath, "utf8"));
    ok = run.ok;
}

return {
    ok,
    language,
    saved,
    terminal: terminal.join("\n\n").trim() || "No compiler output was returned."
};
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work, runs an external command and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; delegates command execution to `runProcess` so timeout and terminal capture remain consistent; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeCodeFileName`, `saveCompileSource`, `safeSegment`, `resolveInsideCompileRoot`, `compileActionFromPayload`, `compileWorkspaceFilesFromPayload`. Those calls show that `compileAndRunCode` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `compileAndRunCode`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside `compileAndRunCode` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

requestedLanguage: `requestedLanguage` stores a computed value for compiler or language configuration. It is initialized from `String(payload.language || "").trim().toLowerCase()`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `normalizeCodeLanguage`; assigned or updated later in the function.

language: language stores a computed value for compiler or language configuration. It is initialized from `requestedLanguage && requestedLanguage !== "auto"`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `String`, `cFamilyCompileProfile`, `cFamilyWorkspaceSources`, `compileActionFromPayload`, `compileCacheDirectory`, `compileCacheKey`, `compileWorkspaceFilesFromPayload`, `hdlFilesFromPayload`; assigned or updated later in the function.

profile: profile stores a computed value for portfolio data state. It is initialized from `compileLanguageProfiles[language] || compileLanguageProfiles.javascript`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `label`; passed into `push`; assigned or updated later in the function.

fileName: fileName stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(payload.fileName, language)`. Within the function it is passed into `cFamilyBinaryName`, `cFamilyCompileProfile`, `cFamilyWorkspaceSources`, `compileCacheKey`, `compileWorkspaceFilesFromPayload`, `detectCodeLanguageFromSource`, `hdlFilesFromPayload`; assigned or updated later in the function.

saved: saved stores the completed result of an awaited operation. It is initialized from `await saveCompileSource({ ...payload, language, fileName })`. Within the function it is accesses properties/methods `fileName`, `id`, `role`, `sourcePath`, `title`; passed into `cFamilyBinaryName`, `cFamilyCompileProfile`, `cFamilyWorkspaceSources`, `compileCacheKey`, `compileWorkspaceFilesFromPayload`, `hdlFilesFromPayload`, `javaMainClassName`; assigned or updated later in the function.

projectFolder: projectFolder stores a computed value for filesystem location. It is initialized from `safeSegment(payload.projectId, "project")`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `resolveInsideCompileRoot`; assigned or updated later in the function.

sourcePath: sourcePath stores a computed value for filesystem location. It is initialized from `resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`, `push`, `runProcess`; assigned or updated later in the function.

sourceCode: sourceCode stores a computed value for temporary calculation. It is initialized from `String(payload.code || "")`. Within the function it is passed into `push`, `writeFile`; assigned or updated later in the function.

action: action stores a computed value for temporary calculation. It is initialized from `compileActionFromPayload(payload.action, language)`. Within the function it is passed into `compileActionFromPayload`, `compileCacheKey`, `push`; assigned or updated later in the function.

allWorkspaceFiles: allWorkspaceFiles stores a computed value for temporary calculation. It is initialized from `compileWorkspaceFilesFromPayload(payload, saved.fileName, language)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`; passed into `cFamilyWorkspaceSources`, `ensureRunSource`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

runDir: runDir stores a string value for filesystem location. It is initialized from `""`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `mkdir`, `runProcess`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

runSourcePath: runSourcePath stores a string value for filesystem location. It is initialized from `""`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `replace`; passed into `cp`, `push`, `runProcess`; assigned or updated later in the function.

runWorkspaceSources: runWorkspaceSources stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods find, push; assigned or updated later in the function.

ensureRunSource: ensureRunSource stores a function or callback value for temporary calculation. It is initialized from async (options = {}) => {}. Within the function it is assigned or updated later in the function.

runId: runId stores a string value for temporary calculation. It is initialized from `\${Date.now()}-\${Math.random().toString(16).slice(2)}`. Within the function it is passed into resolveInsideCompileRoot; assigned or updated later in the function.

filesToWrite: filesToWrite stores a computed value for temporary calculation. It is initialized from Array.isArray(options.files) && options.files.length ? options.files : allWorkspaceFiles. Within the function it is passed into writeCompileWorkspaceSources; assigned or updated later in the function.

active: active stores a function or callback value for temporary calculation. It is initialized from runWorkspaceSources.find((file) => file.id === saved.id || file.fileName === saved.fileName). Within the function it is assigned or updated later in the function.

stdin: stdin stores a computed value for temporary calculation. It is initialized from String(payload.stdin || ""). Within the function it is passed into String; assigned or updated later in the function.

forceRebuild: forceRebuild stores a computed value for temporary calculation. It is initialized from payload.forceRebuild === true. Within the function it is assigned or updated later in the function.

terminal: terminal stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods join, push; assigned or updated later in the function.

append: append stores a function or callback value for temporary calculation. It is initialized from (label, result) => {}. Within the function it is assigned or updated later in the function.

missing: missing stores a function or callback value for temporary calculation. It is initialized from async (tools) => {}. Within the function it is assigned or updated later in the function.

found: found stores a structured object for temporary calculation. It is initialized from {}. Within the function it is accesses properties/methods iverilog, java, javac, vvp; passed into append, push, runProcess, toolVersionLine; assigned or updated later in the function.

tool: tool starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into filter, findTool.

missingTools: missingTools stores a function or callback value for compiler or language configuration. It is initialized from tools.filter((tool) => !found[tool]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods join, length; assigned or updated later in the function.

openTags: openTags stores a function or callback value for temporary calculation. It is initialized from (String(payload.code || "").match(/<([a-z][\w:-]*)b[>]*>/gi) || []).length. Within the function it is assigned or updated later in the function.

closeTags: closeTags stores a function or callback value for temporary calculation. It is initialized from (String(payload.code || "").match(/<\/([a-z][\w:-]*)>/gi) || []).length. Within the function it is assigned or updated later in the function.

ok: ok stores a computed value for temporary calculation. It is initialized from openTags >= closeTags. Within the function it is accesses properties/methods txt; passed into join, push; assigned or updated later in the function.

stdinOptions: stdinOptions stores a computed value for temporary calculation. It is initialized from stdin ? { input: stdin } : {}. Within the function it is passed into runProcess; assigned or updated later in the function.

ok: ok stores a boolean value for temporary calculation. It is initialized from false. Within the function it is accesses properties/methods txt; passed into join, push; assigned or updated later in the function.

node: node stores the completed result of an awaited operation. It is initialized from await findTool("node"). Within the function it is passed into append, findTool, push, runProcess; assigned or updated later in the function.

jsFiles: jsFiles stores a function or callback value for temporary calculation. It is initialized from allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "javascript"). Within the function it is accesses properties/methods length; passed into ensureRunSource; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles, languages: ["javascript"] }). Within the function it is accesses properties/methods runDir, runSourcePath, writtenSources; passed into runProcess; assigned or updated later in the function.

targets: targets stores a computed value for temporary calculation. It is initialized from action === "build". Within the function it is accesses properties/methods length; assigned or updated later in the function.

target: target starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods sourcePath; passed into runProcess, sourceDisplayName.

check: check stores a resolved filesystem or route value. It is initialized from await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 }). Within the function it is accesses properties/methods ok; passed into processTerminalText, push, runProcess, writeFile; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions }). Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

python: python stores the completed result of an awaited operation. It is initialized from await findTool("python"). Within the function it is passed into append, ensureRunSource, findTool, push, runProcess; assigned or updated later in the function.

pyFiles: pyFiles stores a function or callback value for temporary calculation. It is initialized from allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "python"). Within the function it is accesses properties/methods length; passed into ensureRunSource; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles, languages: ["python"] }). Within the function it is accesses properties/methods runDir, runSourcePath, writtenSources; passed into runProcess; assigned or updated later in the function.

targets: targets stores a computed value for temporary calculation. It is initialized from action === "build". Within the function it is accesses properties/methods length; assigned or updated later in the function.

target: target starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods sourcePath; passed into runProcess, sourceDisplayName.

check: check stores a resolved filesystem or route value. It is initialized from await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 }). Within the function it is accesses properties/methods ok; passed into processTerminalText, push, runProcess, writeFile; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from `await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

cProfile: cProfile stores a computed value for portfolio data state. It is initialized from `cFamilyCompileProfile(language, saved.fileName)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `compiler`, `flags`, `header`, `label`, `languageFlag`, `standard`; passed into `push`, `writeFile`; assigned or updated later in the function.

toolName: toolName stores a computed value for compiler or language configuration. It is initialized from `cProfile.compiler`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `missing`, `push`, `runProcess`, `toolVersionLine`; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from `await missing([toolName])`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

workspaceSources: workspaceSources stores a computed value for temporary calculation. It is initialized from `cFamilyWorkspaceSources(allWorkspaceFiles, language, saved.fileName)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`; passed into `writeCompileWorkspaceSources`; assigned or updated later in the function.

buildTargets: buildTargets stores a computed value for temporary calculation. It is initialized from `action === "build"`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; passed into `compileCacheKey`; assigned or updated later in the function.

compilerVersion: compilerVersion stores the completed result of an awaited operation. It is initialized from `await toolVersionLine(found[toolName])`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `push`; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from `compileCacheKey({}`. Within the function it is passed into `compileCacheDirectory`; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from `compileCacheDirectory(payload.projectId, language, cacheKey)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `clearHdlWaveforms`, `join`, `mkdir`, `readHdlWaveform`, `runProcess`, `writeHdlSimulationSources`; assigned or updated later in the function.

workspaceDir: workspaceDir stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "sources")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `rm`, `runProcess`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

writtenSources: writtenSources stores the completed result of an awaited operation. It is initialized from `await writeCompileWorkspaceSources(workspaceSources, workspaceDir)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`, `length`, `map`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

activeWritten: activeWritten stores a function or callback value for temporary calculation. It is initialized from `writtenSources.find((file) => file.fileName === saved.fileName) || writtenSources[0]`. Within the function it is assigned or updated later in the function.

binaryName: binaryName stores a computed value for temporary calculation. It is initialized from cFamilyBinaryName(saved.fileName). Within the function it is passed into join, push; assigned or updated later in the function.

output: output stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, binaryName). Within the function it is passed into pathExists, push, runProcess; assigned or updated later in the function.

outputMarker: outputMarker stores a resolved filesystem or route value. It is initialized from path.join(cacheDir, "syntax-ok.txt"). Within the function it is passed into pathExists, push, writeFile; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from !forceRebuild && await pathExists(output). Within the function it is assigned or updated later in the function.

cachedHeader: cachedHeader stores the completed result of an awaited operation. It is initialized from cProfile.header && !forceRebuild && await pathExists(outputMarker). Within the function it is assigned or updated later in the function.

args: args stores a computed value for temporary calculation. It is initialized from cProfile.header. Within the function it is passed into runProcess; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from await runProcess(found[toolName], args, { cwd: workspaceDir, timeoutMs: 30000 }). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods ok; passed into processTerminalTextWithPaths, push; assigned or updated later in the function.

replacements: replacements stores a list for temporary calculation. It is initialized from []. Within the function it is passed into processTerminalTextWithPaths; assigned or updated later in the function.

toolPath: toolPath stores a computed value for filesystem location. It is initialized from found[toolName]. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into runProcess; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from await runProcess(output, [], { Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from await missing(["javac", "java"]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

className: className stores a computed value for appearance configuration. It is initialized from javaMainClassName(payload.code, saved.fileName). Within the function it is passed into join, runProcess; assigned or updated later in the function.

javaFiles: javaFiles stores a function or callback value for temporary calculation. It is initialized from allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "java"). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; passed into compileCacheKey, writeCompileWorkspaceSources; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from compileCacheKey({ Within the function it is passed into compileCacheDirectory; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from compileCacheDirectory(payload.projectId, language, cacheKey). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into clearHdlWaveforms, join, mkdir, readHdlWaveform, runProcess, writeHdlSimulationSources; assigned or updated later in the function.

classPath: classPath stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, `${className}.class`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `pathExists`, `push`; assigned or updated later in the function.

sourceDir: sourceDir stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "sources")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `rm`, `runProcess`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

writtenSources: writtenSources stores the completed result of an awaited operation. It is initialized from `await writeCompileWorkspaceSources(javaFiles.length ? javaFiles : allWorkspaceFiles, sourceDir, { languages: ["java"] })`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`, `length`, `map`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

javaPath: javaPath stores a function or callback value for filesystem location. It is initialized from `writtenSources.find((file) => file.fileName === saved.fileName)?.sourcePath || path.join(sourceDir, `${className}.java`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `basename`, `writeFile`; assigned or updated later in the function.

cached: cached stores a resolved filesystem or route value. It is initialized from `!forceRebuild && await pathExists(classPath)`. Within the function it is assigned or updated later in the function.

compileTargets: compileTargets stores a function or callback value for compiler or language configuration. It is initialized from `action === "build" ? writtenSources.map((file) => file.sourcePath) : [javaPath]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `runProcess`; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from `await runProcess(found.javac, ["-d", cacheDir, ...compileTargets], { cwd: sourceDir, timeoutMs: 30000 })`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `ok`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from `await runProcess(found.java, ["-cp", cacheDir, className], { cwd: cacheDir, timeoutMs: 20000, ...stdinOptions })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from `await missing(["iverilog", "vvp"])`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

hdlFiles: hdlFiles stores a computed value for temporary calculation. It is initialized from `hdlFilesFromPayload(payload, saved.fileName, language)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `map`, `some`; passed into `compileCacheKey`, `writeHdlSimulationSources`; assigned or updated later in the function.

testbenchFiles: testbenchFiles stores a function or callback value for temporary calculation. It is initialized from `hdlFiles.filter((file) => file.role === "testbench")`. Within the function it is accesses properties/methods `flatMap`, `length`, `some`; passed into `push`; assigned or updated later in the function.

usesSystemVerilog: usesSystemVerilog stores a function or callback value for temporary calculation. It is initialized from `hdlFiles.some((file) => normalizeCodeLanguage(file.language) === "systemverilog")`. Within the function it is passed into `compileCacheKey`; assigned or updated later in the function.

flags: flags stores a computed value for temporary calculation. It is initialized from `usesSystemVerilog ? ["-g2012", "-Wall"] : ["-g2005-sv", "-Wall"]`. Within the function it is accesses properties/methods `join`; assigned or updated later in the function.

topModule: topModule stores a function or callback value for lookup collection. It is initialized from `testbenchFiles.flatMap((file) => hdlModuleNames(file.code)).find(Boolean) || ""`. Within the function it is passed into `push`; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from `compileCacheKey({}`. Within the function it is passed into `compileCacheDirectory`; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from `compileCacheDirectory(payload.projectId, language, cacheKey)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `clearHdlWaveforms`, `join`, `mkdir`, `readHdlWaveform`, `runProcess`, `writeHdlSimulationSources`; assigned or updated later in the function.

output: output stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "simulation.vvp")`. Within the function it is passed into `pathExists`, `push`, `runProcess`; assigned or updated later in the function.

sourceFiles: sourceFiles stores the completed result of an awaited operation. It is initialized from `await writeHdlSimulationSources(hdlFiles, cacheDir)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `map`; assigned or updated later in the function.

designCount: designCount stores a function or callback value for temporary calculation. It is initialized from `sourceFiles.filter((file) => file.role !== "testbench").length`. Within the function it is passed into `push`; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from `!forceRebuild && await pathExists(output)`. Within the function it is assigned or updated later in the function.

compileArgs: compileArgs stores a list for compiler or language configuration. It is initialized from `[]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `runProcess`; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from `await runProcess(found.iverilog, compileArgs, { cwd: cacheDir, timeoutMs: 30000 })`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `ok`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

replacements: replacements stores a function or callback value for lookup collection. It is initialized from `sourceFiles.map((file) => ({ from: file.sourcePath, to: file.uniqueName }))`. Within the function it is passed into `processTerminalTextWithPaths`; assigned or updated later in the function.

waveform: waveform stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is accesses properties/methods `maxTime`, `signals`, `source`, `timeScale`, `vcd`; passed into `Boolean`, `dumpfile`, `push`; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from `await runProcess(found.vvp, [output], { cwd: cacheDir, timeoutMs: 20000 })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

itspice: Itspice stores the completed result of an awaited operation. It is initialized from `await findTool("Ispice")`. Within the function it is passed into `append`, `findTool`, `runProcess`; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from await ensureRunSource(). Within the function it is accesses properties/methods runDir, runSourcePath, writtenSources; passed into runProcess; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from await runProcess(ltspice, ["-b", runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 45000 }). Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

logPath: logPath stores a resolved filesystem or route value. It is initialized from runSource.runSourcePath.replace(/\.([^.]*/, ".log"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push; assigned or updated later in the function.

async function installCompilerTools(language = "")

Role: The best entry point for installCompilerTools is its responsibility in the surrounding workflow. installCompilerTools installs or prepares external tooling. It belongs in the backend because installation touches the machine, not just the browser page.

Function entry point:

```
async function installCompilerTools(language = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for installCompilerTools

```
async function installCompilerTools(language = "") {
  const lang = normalizeCodeLanguage(language);
  const profile = compileLanguageProfiles[lang] || compileLanguageProfiles.javascript;
  if (!profile.winget?.length) {
    return {
      ok: false,
      language: lang,
      terminal: `${profile.label} does not have an automatic Winget compiler install configured. Install the
required tools manually, then restart the builder.`
    };
  }
  const winget = await findTool("winget") || "winget";
  const terminal = [];
  let ok = true;
  for (const packageId of profile.winget) {
    const args = [
      "install",
      "--source",
      "winget",
      "--accept-source-agreements",
      "--accept-package-agreements",
      "--silent",
      "--id",
      packageId,
      "--exact"
    ];
    const result = await runProcess(winget, args, { cwd: root, timeoutMs: 600000 });
    terminal.push(terminalLine(`winget ${args.join(" ")}`), processTerminalText(result));
    ok = ok && result.ok;
  }
  compileToolCache.clear();
  return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: delegates command execution to `runProcess` so timeout and terminal capture remain consistent.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`, `findTool`, `runProcess`, `terminalLine`, `processTerminalText`, `compileToolStatus`. Those calls show that `installCompilerTools` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `installCompilerTools`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `installCompilerTools` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

lang: `lang` stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language)`. Within the function it is assigned or updated later in the function.

profile: `profile` stores a computed value for portfolio data state. It is initialized from `compileLanguageProfiles[lang] || compileLanguageProfiles.javascript`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `label`, `winget`; assigned or updated later in the function.

winget: `winget` stores the completed result of an awaited operation. It is initialized from `await findTool("winget") || "winget"`. Within the function it is passed into `findTool`, `push`, `runProcess`; assigned or updated later in the function.

terminal: `terminal` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

ok: `ok` stores a boolean value for temporary calculation. It is initialized from `true`. Within the function it is assigned or updated later in the function.

packageld: `packageld` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

args: `args` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is accesses properties/methods `join`; passed into `push`, `runProcess`; assigned or updated later in the function.

result: `result` stores the completed result of an awaited operation. It is initialized from `await runProcess(winget, args, { cwd: root, timeoutMs: 600000 })`. Within the function it is accesses properties/methods `ok`; passed into `processTerminalText`; assigned or updated later in the function.

function sourceLooksTextual(source = {})

Role: Begin with the role of `sourceLooksTextual`. `sourceLooksTextual` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sourceLooksTextual(source = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights,

or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for sourceLooksTextual

```
function sourceLooksTextual(source = {}) {  
  const url = String(source.url || "");  
  const kind = String(source.kind || "").toLowerCase();  
  if (["text", "webpage", "github", "linkedin", "public_profile", "resume_text", "code"].includes(kind)) return  
  true;  
  return /\.(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|tsx|v|sv|vhd|?|spice|cir|net|asc|sch|  
  kicad_sch|kicad_pcb|ino|xdc|sdc|tcl)$/.test(url)  
    || /github\.com|raw\.githubusercontent\.com/i.test(url);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means sourceLooksTextual performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without sourceLooksTextual, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside sourceLooksTextual explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from String(source.url || ""). Within the function it is passed into String, test; assigned or updated later in the function.

kind: kind stores a computed value for temporary calculation. It is initialized from String(source.kind || "").toLowerCase(). Within the function it is passed into String; assigned or updated later in the function.

function sourceUrlAllowed(url)

Role: Begin with the role of sourceUrlAllowed. sourceUrlAllowed belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sourceUrlAllowed(url) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for sourceUrlAllowed

```
function sourceUrlAllowed(url) {  
  try {
```



```

const parsed = new URL(url);
const hostName = parsed.hostname.toLowerCase();
return [
  "localhost",
  "127.0.0.1",
  "github.com",
  "api.github.com",
  "raw.githubusercontent.com",
  "gist.githubusercontent.com",
  "linkedin.com",
  "www.linkedin.com"
].includes(hostName);
} catch {
  return false;
}
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceUrlAllowed` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceUrlAllowed`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside `sourceUrlAllowed` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(url)`. Within the function it is accesses properties/methods `hostname`; assigned or updated later in the function.

hostName: `hostName` stores a computed value for temporary calculation. It is initialized from `parsed.hostname.toLowerCase()`. Within the function it is passed into `includes`; assigned or updated later in the function.

function parseGitHubSourceUrl(url)

Role: Read `parseGitHubSourceUrl` first as a named idea, not as syntax. `parseGitHubSourceUrl` interprets `GitHubSourceUrl` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parseGitHubSourceUrl(url) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for parseGitHubSourceUrl

```

function parseGitHubSourceUrl(url) {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();
    const parts = parsed.pathname.split("/").filter(Boolean);
    if (host === "raw.githubusercontent.com" && parts.length >= 4) {
      return {
        type: "file",

```

```

    owner: parts[0],
    repo: parts[1],
    branch: parts[2],
    filePath: parts.slice(3).join("/")
  };
}
if (host !== "github.com" || !parts.length) return null;
if (parts.length === 1) return { type: "profile", owner: parts[0] };
const base = { owner: parts[0], repo: parts[1] };
if (parts[2] === "blob" && parts.length >= 5) {
  return { ...base, type: "file", branch: parts[3], filePath: parts.slice(4).join("/") };
}
if (parts[2] === "tree" && parts.length >= 4) {
  return { ...base, type: "repo", branch: parts[3] };
}
return { ...base, type: "repo" };
} catch {
  return null;
}
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `parseGitHubSourceUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `parseGitHubSourceUrl`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local objects and variables inside `parseGitHubSourceUrl` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(url)`. Within the function it is accesses properties/methods `hostname`, `pathname`; assigned or updated later in the function.

host: `host` stores a computed value for temporary calculation. It is initialized from `parsed.hostname.toLowerCase()`. Within the function it is assigned or updated later in the function.

parts: `parts` stores a computed value for temporary calculation. It is initialized from `parsed.pathname.split("/").filter(Boolean)`. Within the function it is accesses properties/methods `length`, `slice`; assigned or updated later in the function.

base: `base` stores a structured object for temporary calculation. It is initialized from `{ owner: parts[0], repo: parts[1] }`. Within the function it is assigned or updated later in the function.

async function fetchGitHubProfileSource(source = {}, parsed = {})

Role: The best entry point for `fetchGitHubProfileSource` is its responsibility in the surrounding workflow. `fetchGitHubProfileSource` retrieves `GitHubProfileSource` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Function entry point:

```

async function fetchGitHubProfileSource(source = {}, parsed = {}) { ... }

```

Inputs: The function-specific inputs are worth reading before the body. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a

predictable empty result when the caller omits that field. `parsed` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `fetchGitHubProfileSource`

```
async function fetchGitHubProfileSource(source = {}, parsed = {}) {
  const owner = parsed.owner;
  const [profile, repos] = await Promise.all([
    fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}`),
    fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}/repos?per_page=30&sort=updated`)
  ]);
  const repoList = Array.isArray(repos) ? repos : [];
  const includeCode = wantsGitHubCode(source.question || "");
  const selectedRepos = includeCode
    ? [...repoList]
      .sort((a, b) => scoreGitHubRepo(b, source.question || "") - scoreGitHubRepo(a, source.question || ""))
      .slice(0, 3)
    : [];
  const repoBlocks = [];

  for (const repo of selectedRepos) {
    if (!repo?.name || !repo?.html_url) continue;
    const repoSource = await fetchGitHubRepositorySource({
      ...source,
      context: `Public GitHub repository selected from ${owner}'s profile`,
      label: repo.full_name,
      maxCodeFiles: 3,
      maxFileTextLength: 3600,
      maxRepositoryTextLength: 9000,
      url: repo.html_url
    }, {
      owner,
      repo: repo.name,
      type: "repo",
      branch: repo.default_branch
    });
    if (repoSource?.text?.trim()) repoBlocks.push(repoSource.text.trim());
  }

  const lines = [
    `GitHub public profile: ${owner}`,
    profile?.name ? `Name: ${profile.name}` : "",
    profile?.bio ? `Bio: ${profile.bio}` : "",
    profile?.html_url ? `Profile URL: ${profile.html_url}` : source.url,
    "",
    "Public repositories visible from the profile:",
    ...repoList.slice(0, 20).map((repo) => [
      `- ${repo.full_name}`,
      repo.description ? `: ${repo.description}` : "",
      repo.language ? ` | Language: ${repo.language}` : "",
      repo.html_url ? ` | URL: ${repo.html_url}` : "",
      repo.updated_at ? ` | Updated: ${repo.updated_at}` : ""
    ]).join(" "),
    includeCode && selectedRepos.length ? "" : "",
    includeCode && selectedRepos.length ? "Selected public repositories inspected for source code:" : "",
    ...selectedRepos.map((repo) => ` - ${repo.full_name}${repo.language ? ` | Language: ${repo.language}` : ""}${
      repo.description ? ` | ${repo.description}` : ""`),
    ...repoBlocks.flatMap((block) => ["", "----", block])
  ].filter(Boolean);
  return {
    context: source.context || "Public GitHub profile",
    title: source.title || source.label || `GitHub profile: ${owner}`,
    type: includeCode ? "public GitHub profile and selected repository code" : "public GitHub profile",
    url: source.url,
    text: clampText(lines.join("\n"), includeCode ? 30000 : 9000)
  };
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `fetchGitHubJson`, `wantsGitHubCode`, `scoreGitHubRepo`, `fetchGitHubRepositorySource`, `clampText`. Those calls show that `fetchGitHubProfileSource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchGitHubProfileSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `fetchGitHubProfileSource` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

owner: `owner` stores a computed value for temporary calculation. It is initialized from `parsed.owner`. Within the function it is passed into all, `fetchGitHubJson`, `fetchGitHubRepositorySource`; assigned or updated later in the function.

destructured [profile, repos]: `destructured [profile, repos]` stores the completed result of an awaited operation. It is initialized from `await Promise.all([`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent.

repoList: `repoList` stores a computed value for temporary calculation. It is initialized from `Array.isArray(repos) ? repos : []`. Within the function it is accesses properties/methods `slice`; assigned or updated later in the function.

includeCode: `includeCode` stores a computed value for temporary calculation. It is initialized from `wantsGitHubCode(source.question || "")`. Within the function it is assigned or updated later in the function.

selectedRepos: `selectedRepos` stores a computed value for temporary calculation. It is initialized from `includeCode`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

repoBlocks: `repoBlocks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `flatMap`, `push`; assigned or updated later in the function.

repo: `repo` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `default_branch`, `description`, `full_name`, `html_url`, `language`, `name`, `updated_at`; passed into `fetchGitHubRepositorySource`, `map`.

repoSource: `repoSource` stores the completed result of an awaited operation. It is initialized from `await fetchGitHubRepositorySource({`. Within the function it is accesses properties/methods `text`; passed into `push`; assigned or updated later in the function.

lines: `lines` stores a list for temporary calculation. It is initialized from `[`. Within the function it is accesses properties/methods `join`; passed into `clampText`; assigned or updated later in the function.

async function fetchGitHubRepositorySource(source = {}, parsed = {})

Role: Read `fetchGitHubRepositorySource` first as a named idea, not as syntax. `fetchGitHubRepositorySource` retrieves `GitHubRepositorySource` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Function entry point:

```
async function fetchGitHubRepositorySource(source = {}, parsed = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `parsed` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `fetchGitHubRepositorySource`

```
async function fetchGitHubRepositorySource(source = {}, parsed = {}) {
  const { owner, repo } = parsed;
  const metadata = await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`);
  const branch = parsed.branch || metadata?.default_branch || "main";
  const question = source.question || "";
  const requestedMaxCodeFiles = Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3));
  const maxCodeFiles = Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ? requestedMaxCodeFiles : 3));
  const requestedFileTextLength = Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 : 3000));
  const maxFileTextLength = Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ? requestedFileTextLength : 3000));
  const requestedRepoTextLength = Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ? 22000 : 14000));
  const maxRepositoryTextLength = Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength) ? requestedRepoTextLength : 14000));
  const lines = [
    `GitHub repository: ${owner}/${repo}`,
    metadata?.description ? `Description: ${metadata.description}` : "",
    metadata?.language ? `Primary language: ${metadata.language}` : "",
    metadata?.html_url ? `Repository URL: ${metadata.html_url}` : source.url,
    metadata?.default_branch ? `Default branch: ${metadata.default_branch}` : "",
    ""
  ].filter(Boolean);

  if (parsed.type === "file" && parsed.filePath) {
    const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}`;
    const text = await fetchLimitedText(rawUrl, 14000);
    if (text.trim()) lines.push(`Source file: ${parsed.filePath}`, text.trim());
    return {
      context: source.context || "Public GitHub source file",
      title: source.title || source.label || `${owner}/${repo}/${parsed.filePath}`,
      type: "public GitHub source file",
      url: source.url,
      text: clampText(lines.join("\n"), 14000)
    };
  }

  const tree = await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1`);
  const files = Array.isArray(tree?.tree)
    ? tree.tree
      .filter((item) => item.type === "blob" && item.path && githubTextFilePattern.test(item.path) && !githubSkipFilePattern.test(item.path))
      .sort((a, b) => scoreGitHubFile(b.path, question) - scoreGitHubFile(a.path, question))
    : [];

  const readmePath = files.find((file) => /^(^|\/)readme\.md$/i.test(file.path)?.path || "README.md");
  const readmeText = await fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() => "");
  if (readmeText.trim()) lines.push(`README excerpt from ${readmePath}:`, readmeText.trim(), "");

  if (files.length) {
    lines.push(`Repository text/code file list:`, ...files.slice(0, 30).map((file) => ` - ${file.path}`), "");
  }

  const includeCode = wantsGitHubCode(question);
  const selectedFiles = files
```

```

    .filter((file) => !/readme\.md$/i.test(file.path))
    .slice(0, maxCodeFiles);

    for (const file of selectedFiles) {
      const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}`;
      const fileText = await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "");
      if (!fileText.trim()) continue;
      lines.push(`Source file: ${file.path}`, fileText.trim(), "");
    }

    return {
      context: source.context || "Public GitHub repository",
      title: source.title || source.label || `${owner}/${repo}`,
      type: includeCode ? "public GitHub repository and code" : "public GitHub repository",
      url: source.url,
      text: clampText(lines.join("\n"), maxRepositoryTextLength)
    };
  }
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. It works with `fetchGitHubJson`, `wantsGitHubCode`, `fetchLimitedText`, `clampText`, `scoreGitHubFile`. Those calls show that `fetchGitHubRepositorySource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `fetchGitHubRepositorySource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local objects and variables inside `fetchGitHubRepositorySource` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

destructured { owner, repo }: `destructured { owner, repo }` stores a computed value for imported or unpacked API handles. It is initialized from `parsed`. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

metadata: `metadata` stores the completed result of an awaited operation. It is initialized from `await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`)`. Within the function it is accesses properties/methods `default_branch`, `description`, `html_url`, `language`; assigned or updated later in the function.

branch: `branch` stores a computed value for Git publishing and authorization state. It is initialized from `parsed.branch || metadata?.default_branch || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `encodeURIComponent`, `fetchLimitedText`; assigned or updated later in the function.

question: `question` stores a computed value for temporary calculation. It is initialized from `source.question || ""`. Within the function it is passed into `Number`, `scoreGitHubFile`, `wantsGitHubCode`; assigned or updated later in the function.

requestedMaxCodeFiles: `requestedMaxCodeFiles` stores a computed value for temporary calculation. It is initialized from `Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxCodeFiles: `maxCodeFiles` stores a computed value for temporary calculation. It is initialized from `Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ? requestedMaxCodeFiles : 3))`. Within the function it is passed into `Number`, `slice`; assigned or updated later in the function.

requestedFileTextLength: requestedFileTextLength stores a computed value for temporary calculation. It is initialized from `Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 : 3000))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxFileTextLength: maxFileTextLength stores a computed value for temporary calculation. It is initialized from `Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ? requestedFileTextLength : 3000))`. Within the function it is passed into `Number`, `fetchLimitedText`; assigned or updated later in the function.

requestedRepoTextLength: requestedRepoTextLength stores a computed value for temporary calculation. It is initialized from `Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ? 22000 : 14000))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxRepositoryTextLength: maxRepositoryTextLength stores a computed value for temporary calculation. It is initialized from `Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength) ? requestedRepoTextLength : 14000))`. Within the function it is passed into `Number`; assigned or updated later in the function.

lines: lines stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; passed into `clampText`; assigned or updated later in the function.

rawUrl: rawUrl stores a string value for network or routing value. It is initialized from ``https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}``. Within the function it is passed into `fetchLimitedText`; assigned or updated later in the function.

text: text stores the completed result of an awaited operation. It is initialized from `await fetchLimitedText(rawUrl, 14000)`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

tree: tree stores the completed result of an awaited operation. It is initialized from `await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1`)`. Within the function it is accesses properties/methods `tree`; passed into `isArray`; assigned or updated later in the function.

files: files stores a computed value for temporary calculation. It is initialized from `Array.isArray(tree?.tree)`. Within the function it is accesses properties/methods `find`, `length`, `slice`; passed into `push`; assigned or updated later in the function.

readmePath: readmePath stores a function or callback value for filesystem location. It is initialized from `files.find((file) => /^(^|V)readme\.md$/i.test(file.path)).path || "README.md"`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `fetchLimitedText`, `push`; assigned or updated later in the function.

readmeText: readmeText stores a function or callback value for temporary calculation. It is initialized from `await fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() => "")`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

includeCode: includeCode stores a computed value for temporary calculation. It is initialized from `wantsGitHubCode(question)`. Within the function it is assigned or updated later in the function.

selectedFiles: selectedFiles stores a computed value for temporary calculation. It is initialized from `files`. Within the function it is assigned or updated later in the function.

file: file starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `path`; passed into `filter`, `find`, `map`, `push`, `test`.

rawUrl: rawUrl stores a string value for network or routing value. It is initialized from ``https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}``. Within the function it is passed into `fetchLimitedText`; assigned or updated later in the function.

fileText: fileText stores a function or callback value for temporary calculation. It is initialized from `await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "")`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

async function fetchGitHubSourceText(source = {})

Role: The best entry point for `fetchGitHubSourceText` is its responsibility in the surrounding workflow. `fetchGitHubSourceText` retrieves `GitHubSourceText` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Function entry point:

```
async function fetchGitHubSourceText(source = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for fetchGitHubSourceText

```
async function fetchGitHubSourceText(source = {}) {  
  const parsed = parseGitHubSourceUrl(source.url || "");  
  if (!parsed) return null;  
  if (parsed.type === "profile") return fetchGitHubProfileSource(source, parsed);  
  return fetchGitHubRepositorySource(source, parsed);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `parseGitHubSourceUrl`, `fetchGitHubProfileSource`, `fetchGitHubRepositorySource`. Those calls show that `fetchGitHubSourceText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchGitHubSourceText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The easiest way to follow the body of `fetchGitHubSourceText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a computed value for temporary calculation. It is initialized from `parseGitHubSourceUrl(source.url || "")`. Within the function it is accesses properties/methods `type`; passed into `fetchGitHubProfileSource`, `fetchGitHubRepositorySource`; assigned or updated later in the function.

async function readLocalSourceText(url)

Role: To understand readLocalSourceText, start with the problem it owns. readLocalSourceText reads LocalSourceText from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Function entry point:

```
async function readLocalSourceText(url) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for readLocalSourceText

```
async function readLocalSourceText(url) {
  let parsed;
  try {
    parsed = new URL(url);
  } catch {
    return "";
  }
  const localHosts = new Set(["localhost", "127.0.0.1"]);
  if (!localHosts.has(parsed.hostname.toLowerCase())) return "";
  const pathname = decodeURIComponent(parsed.pathname.replace(/^\/+/, ""));
  if (!pathname || pathname.includes(".")) return "";
  const ext = path.extname(pathname).toLowerCase();
  if (![".txt", ".md", ".json", ".csv", ".log", ".xml", ".js", ".mjs", ".css", ".c", ".h", ".cpp", ".hpp", ".py", ".v", ".sv", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(ext)) return "";
  try {
    return await readFile(resolveInsideRoot(pathname), "utf8");
  } catch {
    return "";
  }
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns the awaited result of another asynchronous operation, so its caller receives the completed value rather than a pending Promise. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

Collaborators: Next, trace the helper calls that surround the function. It works with resolveInsideRoot. Those calls show that readLocalSourceText is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without readLocalSourceText, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside readLocalSourceText reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

parsed: parsed starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods hostname, pathname; passed into decodeURIComponent; assigned or updated later in the function.

localHosts: localHosts stores a constructed object for lookup collection. It is initialized from new Set(["localhost", "127.0.0.1"]). Within the function it is accesses properties/methods has; assigned or updated later in the function.

pathname: pathname stores a computed value for filesystem location. It is initialized from decodeURIComponent(parsed.pathname.replace(/^\/+/, "")). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods includes, replace; passed into decodeURIComponent, extname, readFile; assigned or updated later in the function.

ext: ext stores a resolved filesystem or route value. It is initialized from path.extname(pathname).toLowerCase(). Within the function it is assigned or updated later in the function.

async function fetchSourceText(source = {})

Role: Begin with the role of fetchSourceText. fetchSourceText retrieves SourceText from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Function entry point:

```
async function fetchSourceText(source = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for fetchSourceText

```
async function fetchSourceText(source = {}) {
  const url = String(source.url || "");
  if (!url || !sourceLooksTextual(source) || !sourceUrlAllowed(url)) return null;
  if (/github\.com|raw\.githubusercontent\.com/i.test(url)) {
    const githubText = await fetchGitHubSourceText(source);
    if (githubText?.text?.trim()) return githubText;
  }
  const localText = await readLocalSourceText(url);
  if (localText.trim()) {
    return {
      context: source.context || "",
      title: source.title || source.label || url,
      type: source.type || source.kind || "text source",
      url,
      text: clampText(localText.replace(/\s+\n/g, "\n").trim(), 9000)
    };
  }

  try {
    const response = await fetch(url, {
      headers: { "Accept": "text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1" }
    });
    if (!response.ok) return null;
    const contentType = response.headers.get("Content-Type") || "";
    if (!/text|json|xml|html|markdown/i.test(contentType)) return null;
    const rawText = clampText(await response.text(), 12000);
    const text = /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
    if (!text.trim()) return null;
    return {
      context: source.context || "",
      title: source.title || source.label || url,
      type: source.type || source.kind || "public source",
      url,

```

```

    text: clampText(text.trim(), 9000)
  };
} catch {
  return null;
}
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `sourceLooksTextual`, `sourceUrlAllowed`, `fetchGitHubSourceText`, `readLocalSourceText`, `clampText`, `stripHtmlToText`. Those calls show that `fetchSourceText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `fetchSourceText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: The local state inside `fetchSourceText` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

url: `url` stores a computed value for network or routing value. It is initialized from `String(source.url || "")`. Within the function it is passed into `String`, `fetch`, `readLocalSourceText`, `sourceUrlAllowed`; assigned or updated later in the function.

githubText: `githubText` stores the completed result of an awaited operation. It is initialized from `await fetchGitHubSourceText(source)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; assigned or updated later in the function.

localText: `localText` stores the completed result of an awaited operation. It is initialized from `await readLocalSourceText(url)`. Within the function it is accesses properties/methods `replace`, `trim`; passed into `clampText`; assigned or updated later in the function.

response: `response` stores data returned from a network call. It is initialized from `await fetch(url, {`. Within the function it is accesses properties/methods `headers`, `ok`, `text`; passed into `clampText`; assigned or updated later in the function.

contentType: `contentType` stores a computed value for temporary calculation. It is initialized from `response.headers.get("Content-Type") || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

rawText: `rawText` stores the completed result of an awaited operation. It is initialized from `clampText(await response.text(), 12000)`. Within the function it is passed into `stripHtmlToText`; assigned or updated later in the function.

text: `text` stores a computed value for temporary calculation. It is initialized from `/html/i.test(contentType) ? stripHtmlToText(rawText) : rawText`. Within the function it is accesses properties/methods `trim`; passed into `clampText`, `fetch`; assigned or updated later in the function.

function resolveInsideCompileRoot(...segments)

Role: To understand `resolveInsideCompileRoot`, start with the problem it owns. `resolveInsideCompileRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function resolveInsideCompileRoot(...segments) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `resolveInsideCompileRoot`

```
function resolveInsideCompileRoot(...segments) {  
  const target = path.resolve(compileRoot, ...segments);  
  const relative = path.relative(compileRoot, target);  
  if (relative.startsWith("..") || path.isAbsolute(relative)) {  
    throw new Error("Resolved path is outside the compile workspace.");  
  }  
  return target;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node `path` utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `resolveInsideCompileRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `resolveInsideCompileRoot`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Local values: After the main flow, the working values inside `resolveInsideCompileRoot` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a resolved filesystem or route value. It is initialized from `path.resolve(compileRoot, ...segments)`. Within the function it is returned to the caller; passed into `relative`; assigned or updated later in the function.

relative: `relative` stores a resolved filesystem or route value. It is initialized from `path.relative(compileRoot, target)`. Within the function it accesses properties/methods `startsWith`; passed into `isAbsolute`; assigned or updated later in the function.

Filesystem safety

The functions in this group all support filesystem safety. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function `languageFromFileName(fileName = "", code = "")`

Role: To understand `languageFromFileName`, start with the problem it owns. `languageFromFileName` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function languageFromFileName(fileName = "", code = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `languageFromFileName`

```
function languageFromFileName(fileName = "", code = "") {  
  const ext = path.extname(String(fileName || "").toLowerCase());  
  if (ext === ".h") {  
    if (sourceLooksCpp(code)) return "cpp";  
    if (sourceLooksC(code)) return "c";  
    return "c";  
  }  
  return Object.entries(compileLanguageProfiles).find(([, profile]) => profile.extensions.includes(ext))?.[0] || "";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with `sourceLooksC`, `sourceLooksCpp`. Those calls show that `languageFromFileName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `languageFromFileName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `languageFromFileName` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

ext: `ext` stores a resolved filesystem or route value. It is initialized from `path.extname(String(fileName || "").toLowerCase())`. Within the function it is passed into `includes`; assigned or updated later in the function.

`function safeCodeFileName(value = "", language = "javascript")`

Role: The best entry point for `safeCodeFileName` is its responsibility in the surrounding workflow. `safeCodeFileName` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function safeCodeFileName(value = "", language = "javascript") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default

value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `safeCodeFileName`

```
function safeCodeFileName(value = "", language = "javascript") {  
  const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] || compileLanguageProfiles.javascript;  
  const fallback = profile.defaultFile;  
  const parsed = path.parse(String(value || fallback));  
  const safeName = safeSegment(parsed.name, path.parse(fallback).name);  
  let ext = String(parsed.ext || path.extname(fallback)).toLowerCase();  
  if (!profile.extensions.includes(ext)) ext = path.extname(fallback);  
  return `${safeName}${ext}`;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`, `safeSegment`. Those calls show that `safeCodeFileName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `safeCodeFileName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `safeCodeFileName` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

profile: profile stores a computed value for portfolio data state. It is initialized from `compileLanguageProfiles[normalizeCodeLanguage(language)] || compileLanguageProfiles.javascript`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `defaultFile`, `extensions`; assigned or updated later in the function.

fallback: fallback stores a computed value for temporary calculation. It is initialized from `profile.defaultFile`. Within the function it is passed into `String`, `extname`, `parse`, `safeSegment`; assigned or updated later in the function.

parsed: parsed stores a resolved filesystem or route value. It is initialized from `path.parse(String(value || fallback))`. Within the function it is accesses properties/methods `ext`, `name`; passed into `String`, `safeSegment`; assigned or updated later in the function.

safeName: `safeName` stores a resolved filesystem or route value. It is initialized from `safeSegment(parsed.name, path.parse(fallback).name)`. Within the function it is assigned or updated later in the function.

ext: ext stores a resolved filesystem or route value. It is initialized from `String(parsed.ext || path.extname(fallback)).toLowerCase()`. Within the function it is passed into `String`; assigned or updated later in the function.

function pathVariantsForReplacement(value = "")

Role: Begin with the role of pathVariantsForReplacement. pathVariantsForReplacement belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function pathVariantsForReplacement(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for pathVariantsForReplacement

```
function pathVariantsForReplacement(value = "") {  
  const clean = String(value || "");  
  return Array.from(new Set([  
    clean,  
    path.normalize(clean),  
    clean.replace(/\\/g, "/"),  
    path.normalize(clean).replace(/\\/g, "/")  
  ]).filter(Boolean));  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means pathVariantsForReplacement performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without pathVariantsForReplacement, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside pathVariantsForReplacement explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from String(value || ""). Within the function it is accesses properties/methods replace; passed into from, normalize; assigned or updated later in the function.

function replacePathReferences(text = "", replacements = [])

Role: The best entry point for replacePathReferences is its responsibility in the surrounding workflow. replacePathReferences belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function replacePathReferences(text = "", replacements = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `replacements` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `replacePathReferences`

```
function replacePathReferences(text = "", replacements = []) {
  let output = String(text || "");
  for (const replacement of replacements) {
    const from = replacement?.from || "";
    const to = replacement?.to || "";
    if (!from || !to) continue;
    for (const variant of pathVariantsForReplacement(from)) {
      output = output.split(variant).join(to);
    }
  }
  return output;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `pathVariantsForReplacement`. Those calls show that `replacePathReferences` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `replacePathReferences`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `replacePathReferences` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

output: `output` stores a computed value for temporary calculation. It is initialized from `String(text || "")`. Within the function it is returned to the caller; accesses properties/methods `split`; assigned or updated later in the function.

replacement: `replacement` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

from: `from` stores a computed value for temporary calculation. It is initialized from `replacement?.from || ""`. Within the function it is assigned or updated later in the function.

to: `to` stores a computed value for temporary calculation. It is initialized from `replacement?.to || ""`. Within the function it is passed into `join`; assigned or updated later in the function.

variant: `variant` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into `split`.

async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3)

Role: Begin with the role of findFilesByExtension. findFilesByExtension belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. folder is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. extension is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. maxDepth is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for findFilesByExtension

```
async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3) {  
  if (!folder || maxDepth < 0 || !(await pathExists(folder))) return [];  
  const entries = await readdir(folder, { withFileTypes: true }).catch(() => []);  
  const matches = [];  
  for (const entry of entries) {  
    const fullPath = path.join(folder, entry.name);  
    if (entry.isDirectory()) {  
      matches.push(...await findFilesByExtension(fullPath, extension, maxDepth - 1));  
    } else if (entry.isFile() && entry.name.toLowerCase().endsWith(extension.toLowerCase())) {  
      matches.push(fullPath);  
    }  
  }  
  return matches;  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and reads or writes files. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with pathExists. Those calls show that findFilesByExtension is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without findFilesByExtension, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside findFilesByExtension explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

entries: entries stores a function or callback value for temporary calculation. It is initialized from await readdir(folder, { withFileTypes: true }).catch(() => []). Within the function it is assigned or updated later in the function.

matches: matches stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

entry: entry starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods isDirectory, isFile, name; passed into join.

fullPath: fullPath stores a resolved filesystem or route value. It is initialized from path.join(folder, entry.name). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push; assigned or updated later in the function.

async function readJsonFile(filePath)

Role: Read readJsonFile first as a named idea, not as syntax. readJsonFile reads JsonFile from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Function entry point:

```
async function readJsonFile(filePath) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. filePath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for readJsonFile

```
async function readJsonFile(filePath) {  
  return JSON.parse(await readFile(filePath, "utf8"));  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data; reads data from disk as part of the operation.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means readJsonFile performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without readJsonFile, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside readJsonFile, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function safeSegment(value, fallback = "item")

Role: Read safeSegment first as a named idea, not as syntax. safeSegment is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function safeSegment(value, fallback = "item") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `safeSegment`

```
function safeSegment(value, fallback = "item") {  
  const segment = String(value || fallback)  
    .toLowerCase()  
    .trim()  
    .replace(/^[^a-z0-9._-]+/g, "-")  
    .replace(/^-+|-+$/g, "");  
  return segment || fallback;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `safeSegment` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `safeSegment`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `safeSegment` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

segment: `segment` stores a computed value for temporary calculation. It is initialized from `String(value || fallback)`. Within the function it is returned to the caller; assigned or updated later in the function.

function `safeFileName(value)`

Role: Begin with the role of `safeFileName`. `safeFileName` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function safeFileName(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `safeFileName`

```
function safeFileName(value) {
```

```

const parsed = path.parse(String(value || "file"));
const name = safeSegment(parsed.name, "file");
const ext = safeSegment(parsed.ext.replace(".", ""), "");
return ext ? `${name}.${ext}` : name;
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `safeSegment`. Those calls show that `safeFileName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `safeFileName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `safeFileName` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(String(value || "file"))`. Within the function it is accesses properties/methods `ext`, `name`; passed into `safeSegment`; assigned or updated later in the function.

name: `name` stores a computed value for temporary calculation. It is initialized from `safeSegment(parsed.name, "file")`. Within the function it is passed into `safeSegment`; assigned or updated later in the function.

ext: `ext` stores a computed value for temporary calculation. It is initialized from `safeSegment(parsed.ext.replace(".", ""), "")`. Within the function it is returned to the caller; accesses properties/methods `replace`; passed into `safeSegment`; assigned or updated later in the function.

function resolveInsideRoot(...segments)

Role: Begin with the role of `resolveInsideRoot`. `resolveInsideRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function resolveInsideRoot(...segments) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for resolveInsideRoot

```

function resolveInsideRoot(...segments) {
  const target = path.normalize(path.join(root, ...segments));
  if (!target.startsWith(root)) {
    throw new Error("Resolved path is outside the workspace.");
  }
  return target;
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this

rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `resolveInsideRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `resolveInsideRoot`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `resolveInsideRoot` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a resolved filesystem or route value. It is initialized from `path.normalize(path.join(root, ...segments))`. Within the function it is returned to the caller; accesses properties/methods `startsWith`; assigned or updated later in the function.

function resolveInsidePortfolioRoot(...segments)

Role: Read `resolveInsidePortfolioRoot` first as a named idea, not as syntax. `resolveInsidePortfolioRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function resolveInsidePortfolioRoot(...segments) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for resolveInsidePortfolioRoot

```
function resolveInsidePortfolioRoot(...segments) {  
  const target = path.resolve(portfolioRoot, ...segments);  
  const relative = path.relative(portfolioRoot, target);  
  if (relative.startsWith("..") || path.isAbsolute(relative)) {  
    throw new Error("Resolved path is outside the portfolio workspace.");  
  }  
  return target;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `resolveInsidePortfolioRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `resolveInsidePortfolioRoot`, the specific rule it owns would disappear from the named

workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `resolveInsidePortfolioRoot` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a resolved filesystem or route value. It is initialized from `path.resolve(portfolioRoot, ...segments)`. Within the function it is returned to the caller; passed into `relative`; assigned or updated later in the function.

relative: `relative` stores a resolved filesystem or route value. It is initialized from `path.relative(portfolioRoot, target)`. Within the function it accesses properties/methods `startsWith`; passed into `isAbsolute`; assigned or updated later in the function.

function samePath(left, right)

Role: The best entry point for `samePath` is its responsibility in the surrounding workflow. `samePath` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function samePath(left, right) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `left` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `right` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for samePath

```
function samePath(left, right) {  
  return path.resolve(left).toLowerCase() === path.resolve(right).toLowerCase();  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `samePath` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `samePath`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `samePath`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function pathExists(filePath)

Role: To understand pathExists, start with the problem it owns. pathExists belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function pathExists(filePath) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. filePath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for pathExists

```
async function pathExists(filePath) {  
  try {  
    await fsAccess(filePath);  
    return true;  
  } catch {  
    return false;  
  }  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means pathExists performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without pathExists, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside pathExists, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot)

Role: The best entry point for workspaceHasCompatibleSiteFiles is its responsibility in the surrounding workflow. workspaceHasCompatibleSiteFiles belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. baseRoot is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for workspaceHasCompatibleSiteFiles

```
async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) {  
  for (const fileName of ["index.html", "styles.css", "script.js"]) {  
    try {  
      await readFile(path.join(baseRoot, fileName));  
    } catch {  
      return false;  
    }  
  }  
  return true;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; reads data from disk as part of the operation.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means workspaceHasCompatibleSiteFiles performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without workspaceHasCompatibleSiteFiles, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of workspaceHasCompatibleSiteFiles is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

fileName: fileName starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into readFile.

Public source and AI context

The functions in this group all support public source and ai context. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function githubHeaders(accept = "application/vnd.github+json")

Role: The best entry point for githubHeaders is its responsibility in the surrounding workflow. githubHeaders belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function githubHeaders(accept = "application/vnd.github+json") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. accept is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for githubHeaders

```
function githubHeaders(accept = "application/vnd.github+json") {  
  return {  
    "Accept": accept,  
    "User-Agent": "OMB-Portfolio-Builder-AI"  
  };  
}
```

```
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `githubHeaders` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `githubHeaders`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `githubHeaders`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function fetchGithubJson(url)

Role: The best entry point for `fetchGithubJson` is its responsibility in the surrounding workflow. `fetchGithubJson` retrieves `GitHubJson` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

Function entry point:

```
async function fetchGithubJson(url) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for fetchGithubJson

```
async function fetchGithubJson(url) {  
  const response = await fetch(url, { headers: githubHeaders() });  
  if (!response.ok) return null;  
  return response.json().catch(() => null);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `githubHeaders`. Those calls show that `fetchGithubJson` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchGithubJson`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `fetchGithubJson` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

response: response stores data returned from a network call. It is initialized from `await fetch(url, { headers: githubHeaders() })`. Within the function it is returned to the caller; accesses properties/methods `json`, `ok`; assigned or updated later in the function.

function githubQuestionTokens(question = "")

Role: The best entry point for `githubQuestionTokens` is its responsibility in the surrounding workflow. `githubQuestionTokens` belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function githubQuestionTokens(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for githubQuestionTokens

```
function githubQuestionTokens(question = "") {  
  return [...new Set(String(question || "").  
    .toLowerCase()  
    .replace(/^[^a-z0-9_#+\s-]+/g, " ")  
    .split(/\s+/)  
    .filter((token) => token.length >= 3 && ![  
      "the", "and", "with", "from", "show", "code", "file",  
      "github"].includes(token))];  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `githubQuestionTokens` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `githubQuestionTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `githubQuestionTokens`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function scoreGitHubFile(filePath = "", question = "")

Role: The best entry point for `scoreGitHubFile` is its responsibility in the surrounding workflow. `scoreGitHubFile` ranks a candidate against a question or search term. Scoring functions are what make search and AI source selection feel relevant instead of random.

Function entry point:

```
function scoreGitHubFile(filePath = "", question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `filePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the

function fail softly or produce a predictable empty result when the caller omits that field. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for scoreGitHubFile

```
function scoreGitHubFile(filePath = "", question = "") {  
  const cleanPath = filePath.toLowerCase();  
  const tokens = githubQuestionTokens(question);  
  let score = githubTextFilePattern.test(cleanPath) ? 10 : 0;  
  if (/readme\.md$/i.test(cleanPath)) score += 55;  
  if (/\/\.(c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|ino|tcl|xdc|sdc)$/i.test(cleanPath)) score += 28;  
  if (/\/b(test|tb|bench|sim|simulation|src|firmware|hardware|rtl|driver|main)\b/i.test(cleanPath)) score += 12;  
  tokens.forEach((token) => {  
    if (cleanPath.includes(token)) score += 18;  
  });  
  return score;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with githubQuestionTokens. Those calls show that scoreGitHubFile is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without scoreGitHubFile, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of scoreGitHubFile is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanPath: cleanPath stores a resolved filesystem or route value. It is initialized from filePath.toLowerCase(). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accessed via properties/methods includes; passed into test; assigned or updated later in the function.

tokens: tokens stores a computed value for temporary calculation. It is initialized from githubQuestionTokens(question). Within the function it is iterated or transformed as a collection; accesses properties/methods forEach; assigned or updated later in the function.

score: score stores a resolved filesystem or route value. It is initialized from githubTextFilePattern.test(cleanPath) ? 10 : 0. Within the function it is returned to the caller; assigned or updated later in the function.

function wantsGitHubCode(question = "")

Role: The best entry point for wantsGitHubCode is its responsibility in the surrounding workflow. wantsGitHubCode belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function wantsGitHubCode(question = "") { ... }
```


Inputs: The function-specific inputs are worth reading before the body. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for wantsGitHubCode

```
function wantsGitHubCode(question = "") {  
  return /\b(code|source|snippet|implementation|firmware|driver|module|verilog|vhdl|python|javascript|typescript|c\+  
  \+|cpp|c\s+code|pull|show|display)\b/i.test(question);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means wantsGitHubCode performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without wantsGitHubCode, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside wantsGitHubCode, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function scoreGitHubRepo(repo = {}, question = "")

Role: To understand scoreGitHubRepo, start with the problem it owns. scoreGitHubRepo ranks a candidate against a question or search term. Scoring functions are what make search and AI source selection feel relevant instead of random.

Function entry point:

```
function scoreGitHubRepo(repo = {}, question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. repo is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for scoreGitHubRepo

```
function scoreGitHubRepo(repo = {}, question = "") {  
  const tokens = githubQuestionTokens(question);  
  const haystack = [  
    repo.full_name,  
    repo.name,  
    repo.description,  
    repo.language,  
  ]
```



```

    ...(Array.isArray(repo.topics) ? repo.topics : [])
].join(" ").toLowerCase();
let score = repo.fork ? -10 : 12;
if (!repo.archived) score += 6;
if (repo.language) score += 4;
if (repo.name && !/\.\github\.io$/i.test(repo.name)) score += 3;
tokens.forEach((token) => {
    if (haystack.includes(token)) score += 22;
});
if (/b(vco|oscillator|pwm|analog|mixed|pcb|firmware|stm32|fpga|asic|verilog|embedded|signal|design)
\b/i.test(haystack)) {
    score += 12;
}
return score;
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `githubQuestionTokens`. Those calls show that `scoreGitHubRepo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `scoreGitHubRepo`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `scoreGitHubRepo` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `githubQuestionTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

haystack: `haystack` stores a list for runtime state or cache. It is initialized from `[]`. Within the function it is accessed via `includes`; passed into `test`; assigned or updated later in the function.

score: `score` stores a computed value for temporary calculation. It is initialized from `repo.fork ? -10 : 12`. Within the function it is returned to the caller; assigned or updated later in the function.

async function `enrichPortfolioContext(context = {})`

Role: Begin with the role of `enrichPortfolioContext`. `enrichPortfolioContext` belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

Function entry point:

```

async function enrichPortfolioContext(context = {}) { ... }

```

Inputs: The function-specific inputs are worth reading before the body. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `enrichPortfolioContext`

```

async function enrichPortfolioContext(context = {}) {

```

```

const question = String(context.question || "");
const sources = Array.isArray(context.sourceFetches)
  ? context.sourceFetches.slice(0, 10).map((source) => ({ ...source, question: source.question || question }))
  : [];
const sourceExcerpts = (await Promise.all(sources.map(fetchSourceText))).filter(Boolean);
return {
  ...context,
  sourceExcerpts,
  sourceFetchPolicy: "Fetched excerpts are limited to safe text-like same-site, GitHub/raw GitHub, LinkedIn, and
public portfolio URLs. GitHub repository links can be expanded into repository metadata, README text, and selected
public source files when a question asks for code. PDFs/images are represented by captions, metadata, filenames, and
companion text files when available."
};
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `enrichPortfolioContext` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `enrichPortfolioContext`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `enrichPortfolioContext` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

question: `question` stores a computed value for temporary calculation. It is initialized from `String(context.question || "")`. Within the function it is passed into `String`; assigned or updated later in the function.

sources: `sources` stores a computed value for temporary calculation. It is initialized from `Array.isArray(context.sourceFetches)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; passed into `all`; assigned or updated later in the function.

sourceExcerpts: `sourceExcerpts` stores a function or callback value for lookup collection. It is initialized from `(await Promise.all(sources.map(fetchSourceText))).filter(Boolean)`. Within the function it is assigned or updated later in the function.

function `parseGitHubRemote(remoteUrl = "")`

Role: Read `parseGitHubRemote` first as a named idea, not as syntax. `parseGitHubRemote` interprets `GitHubRemote` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parseGitHubRemote(remoteUrl = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `parseGitHubRemote`

```

function parseGitHubRemote(remoteUrl = "") {
  const trimmed = String(remoteUrl || "").trim();
  const sshMatch = trimmed.match(/^git@github\.com:([^\s]+)\.(\.git)?$/i);
  if (sshMatch) return { owner: sshMatch[1], repo: sshMatch[2].replace(/\.git$/, "") };
}

```

```

try {
  const parsed = new URL(trimmed.replace(/^git\+/ , ""));
  if (!/github\.com$/i.test(parsed.hostname)) return null;
  const parts = parsed.pathname.replace(/^\/\+/ , "").split("/");
  if (parts.length < 2) return null;
  return { owner: parts[0], repo: parts[1].replace(/\.git$/i, "") };
} catch {
  return null;
}
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `parseGitHubRemote` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `parseGitHubRemote`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `parseGitHubRemote` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

trimmed: `trimmed` stores a computed value for temporary calculation. It is initialized from `String(remoteUrl || "").trim()`. Within the function it is accesses properties/methods `match`, `replace`; passed into `URL`; assigned or updated later in the function.

sshMatch: `sshMatch` stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^git@github\.com:([^\s]+)\.(.+)?(?:\.git)?$/i)`. Within the function it is assigned or updated later in the function.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(trimmed.replace(/^git\+/ , ""))`. Within the function it is accesses properties/methods `hostname`, `pathname`; assigned or updated later in the function.

parts: `parts` stores a computed value for temporary calculation. It is initialized from `parsed.pathname.replace(/^\/\+/ , "").split("/")`. Within the function it is accesses properties/methods `length`; assigned or updated later in the function.

async function `authenticateGitHubForTarget(options = {})`

Role: Begin with the role of `authenticateGitHubForTarget`. `authenticateGitHubForTarget` is the interactive setup path for a publishing target. It validates the target remote, initializes or fixes the local Git repository when necessary, stores credentials when credentials are supplied, verifies write access, saves the target configuration, and records the authorization cache. It is the correct first step before Load from target or Apply to site. The function exists because merely typing a GitHub URL does not prove the builder can read from or write to that repository. Authentication must be completed, verified, and remembered before the target is considered usable.

Function entry point:

```

async function authenticateGitHubForTarget(options = {}) { ... }

```

Inputs: The function-specific inputs are worth reading before the body. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for authenticateGitHubForTarget

```
async function authenticateGitHubForTarget(options = {}) {
  const {
    repositoryUrl = "",
    customDomain = "",
    authUsername = "",
    authPassword = ""
  } = options;
  const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
  const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
  const domain = validateCustomDomain(customDomain);
  const credentials = validateCredentialPair(authUsername, authPassword);
  if (!remoteUrl) {
    throw publishAccessError(
      "A GitHub repository URL is required before authentication.",
      "Enter the GitHub Pages or compatible static-site repository URL, then click Authenticate target.",
      await getPublishTargetInfo()
    );
  }

  await ensureGitRepository();
  const snapshot = await capturePublishTargetState();

  try {
    await setOriginRemote(remoteUrl);
    const status = await getGitStatus();
    const preliminaryTarget = await getPublishTargetInfo();
    if (!status.git.ok) {
      throw publishAccessError(
        "Git for Windows is required for publishing.",
        "Install Git for Windows, then reopen the builder and authenticate again.",
        { ...preliminaryTarget, system: status }
      );
    }

    const targetBranch = await detectRemoteDefaultBranch(remoteUrl) || "main";
    await checkoutPublishBranch(targetBranch);

    const targetBeforeAuth = await getPublishTargetInfo();
    const cached = await readPublishAuthCache();
    const cachedAccess = {
      branch: targetBeforeAuth.branch,
      remote: targetBeforeAuth.remote,
      repository: targetBeforeAuth.repository
    };
    if (!credentials.cleanUsername && publishAuthCacheIsFresh(cached, cachedAccess)) {
      await writeTargetCustomDomain(domain, customDomainProvided);
      const target = await getPublishTargetInfo();
      return {
        ...cachedAccess,
        branch: target.branch || cachedAccess.branch,
        authorizationCached: true,
        authorizationChecked: true,
        checkedAt: cached.checkedAt,
        expiresAt: publishAuthCacheExpiresAt(cached),
        extendedTrust: Boolean(cached.extendedTrust),
        successCountLastWeek: cached.successCountLastWeek || 0,
        trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1),
        credentialsStored: false,
        credentialUsername: "",
        system: status,
        target
      };
    }

    if (!status.credentialManager.ok && !credentials.cleanUsername) {
      throw publishAccessError(
        "Git Credential Manager is required for guided GitHub sign-in.",

```

```

    "Install or repair Git for Windows with Git Credential Manager enabled, or provide a GitHub username and
    personal access token.",
    { ...targetBeforeAuth, system: status }
  );
}

const storedCredentials = await storeGitCredentials(remoteUrl, authUsername, authPassword);
if (!storedCredentials.stored) {
  await runGit(["credential-manager", "github", "login"], {
    cwd: portfolioRoot,
    timeout: 5 * 60 * 1000,
    windowsHide: false
  });
}

const access = await assertPublishAccess({ force: true, requireCompatible: false });
await writeTargetCustomDomain(domain, customDomainProvided);
const target = await getPublishTargetInfo();
return {
  ...access,
  branch: target.branch || access.branch,
  credentialsStored: storedCredentials.stored,
  credentialUsername: storedCredentials.username || "",
  system: await getGitStatus(),
  target
};
} catch (error) {
  await restorePublishTargetState(snapshot);
  throw error;
}
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `validatePublishRemoteUrl`, `validateCustomDomain`, `validateCredentialPair`, `publishAccessError`, `getPublishTargetInfo`, `ensureGitRepository`, `capturePublishTargetState`, `setOriginRemote`. Those calls show that `authenticateGitHubForTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `authenticateGitHubForTarget`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `authenticateGitHubForTarget` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

customDomainProvided: `customDomainProvided` stores a computed value for temporary calculation. It is initialized from `Object.prototype.hasOwnProperty.call(options, "customDomain")`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

remoteUrl: `remoteUrl` stores a computed value for network or routing value. It is initialized from `validatePublishRemoteUrl(repositoryUrl)`. Within the function it is passed into `detectRemoteDefaultBranch`, `setOriginRemote`, `storeGitCredentials`; assigned or updated later in the function.

domain: `domain` stores a computed value for temporary calculation. It is initialized from `validateCustomDomain(customDomain)`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

credentials: `credentials` stores a computed value for Git publishing and authorization state. It is initialized from `validateCredentialPair(authUsername, authPassword)`. It affects publishing, where mistakes can change the

public website or expose the wrong files. Within the function it is accesses properties/methods cleanUsername; assigned or updated later in the function.

snapshot: snapshot stores the completed result of an awaited operation. It is initialized from await capturePublishTargetState(). Within the function it is passed into restorePublishTargetState; assigned or updated later in the function.

status: status stores the completed result of an awaited operation. It is initialized from await getGitStatus(). Within the function it is accesses properties/methods credentialManager, git; passed into publishAccessError; assigned or updated later in the function.

preliminaryTarget: preliminaryTarget stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is passed into publishAccessError; assigned or updated later in the function.

targetBranch: targetBranch stores the completed result of an awaited operation. It is initialized from await detectRemoteDefaultBranch(remoteUrl) || "main". It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into checkoutPublishBranch; assigned or updated later in the function.

targetBeforeAuth: targetBeforeAuth stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods branch, remote, repository; passed into publishAccessError; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from await readPublishAuthCache(). Within the function it is accesses properties/methods checkedAt, extendedTrust, successCountLastWeek, trustDays; passed into Boolean, publishAuthCacheExpiresAt; assigned or updated later in the function.

cachedAccess: cachedAccess stores a structured object for runtime state or cache. It is initialized from {. Within the function it is accesses properties/methods branch; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

storedCredentials: storedCredentials stores the completed result of an awaited operation. It is initialized from await storeGitCredentials(remoteUrl, authUsername, authPassword). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stored, username; assigned or updated later in the function.

access: access stores the completed result of an awaited operation. It is initialized from await assertPublishAccess({ force: true, requireCompatible: false }). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

AI backend

The functions in this group all support ai backend. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function cleanConversationHistory(value)

Role: Begin with the role of cleanConversationHistory. cleanConversationHistory belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cleanConversationHistory(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `cleanConversationHistory`

```
function cleanConversationHistory(value) {  
  if (!Array.isArray(value)) return [];  
  return value  
    .slice(-8)  
    .map((item) => ({  
      role: item?.role === "assistant" ? "assistant" : "user",  
      content: clampText(item?.content, 1400).trim()  
    }))  
    .filter((item) => item.content);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `clampText`. Those calls show that `cleanConversationHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `cleanConversationHistory`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `cleanConversationHistory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `extractOpenAiText(data)`

Role: Begin with the role of `extractOpenAiText`. `extractOpenAiText` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function extractOpenAiText(data) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `data` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `extractOpenAiText`

```
function extractOpenAiText(data) {  
  if (typeof data?.output_text === "string" && data.output_text.trim()) return data.output_text.trim();  
  const output = Array.isArray(data?.output) ? data.output : [];  
  const chunks = [];  
  
  output.forEach((item) => {  
    (item.content || []).forEach((content) => {
```



```

    if (typeof content.text === "string" && content.text.trim()) chunks.push(content.text.trim());
  });
});

return chunks.join("\n\n").trim();
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `extractOpenAiText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `extractOpenAiText`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `extractOpenAiText` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

output: output stores a computed value for temporary calculation. It is initialized from `Array.isArray(data?.output) ? data.output : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; passed into `isArray`; assigned or updated later in the function.

chunks: chunks stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

function `extractOllamaText(data = {})`

Role: Read `extractOllamaText` first as a named idea, not as syntax. `extractOllamaText` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function extractOllamaText(data = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `data` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `extractOllamaText`

```

function extractOllamaText(data = {}) {
  if (typeof data.message?.content === "string") return data.message.content.trim();
  if (typeof data.response === "string") return data.response.trim();
  return "";
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `extractOllamaText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `extractOllamaText`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `extractOllamaText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })

Role: Read `callOllamaPortfolioAi` first as a named idea, not as syntax. `callOllamaPortfolioAi` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })
{ ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `conversation` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. `allowWebSearch` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for callOllamaPortfolioAi

```
async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch }) {
  const host = process.env.OLLAMA_HOST || "http://127.0.0.1:11434";
  const model = process.env.OLLAMA_MODEL || "llama3.2";
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000));
  try {
    const response = await fetch(`${host.replace(/\/\+$/, "")}/api/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      signal: controller.signal,
      body: JSON.stringify({
        model,
        stream: false,
        messages: [
          { role: "system", content: portfolioAiInstructions },
          {
            role: "user",
            content: [
              `Visitor question: ${question}`,
              `Question intent: ${intent}`,
              `Web/public lookup requested by browser: ${allowWebSearch ? "yes" : "no"}`,
              "",
              "Recent conversation JSON:",
              clampText(JSON.stringify(conversation, null, 2), 8000),
              ""
            ]
          }
        ]
      })
    });
  }
}
```

```

        "Portfolio context JSON:",
        clampText(JSON.stringify(context, null, 2), 18000)
    ].join("\n")
    },
    options: {
        temperature: Number(process.env.OLLAMA_TEMPERATURE || 0.35)
    }
  })
});
const data = await response.json().catch(() => ({}));
if (!response.ok) {
  return { ok: false, error: data?.error || `Ollama returned HTTP ${response.status}.` };
}
const answer = extractOllamaText(data);
return answer
  ? { ok: true, answer, model }
  : { ok: false, error: "Ollama did not return an answer." };
} catch (error) {
  return { ok: false, error: error?.name === "AbortError" ? "Ollama timed out." : "Ollama is not reachable on this machine." };
} finally {
  clearTimeout(timeout);
}
}
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work, calls a network resource and uses a timer to avoid blocking or to wait for another process. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; performs a fetch and therefore must handle network delay and failure.

Collaborators: The function also has to be read through the helpers it calls. It works with `clampText`, `extractOllamaText`. Those calls show that `callOllamaPortfolioAi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `callOllamaPortfolioAi`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `callOllamaPortfolioAi` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

host: host stores an environment-derived value for temporary calculation. It is initialized from `process.env.OLLAMA_HOST || "http://127.0.0.1:11434"`. Within the function it is accessed; properties/methods replaced; passed into fetch; assigned or updated later in the function.

model: model stores an environment-derived value for temporary calculation. It is initialized from `process.env.OLLAMA_MODEL || "llama3.2"`. Within the function it is passed into `stringify`; assigned or updated later in the function.

controller: controller stores a constructed object for temporary calculation. It is initialized from `new AbortController()`. Within the function it is accessed; properties/methods accessed; assigned or updated later in the function.

timeout: timeout stores a function or callback value for temporary calculation. It is initialized from `setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000))`. Within the function it is passed into `clearTimeout`; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch(`${host.replace(/V+$/, '')}/api/chat`, {`. Within the function it is accessed; properties/methods accessed; assigned or updated later in the function.

data: data stores a function or callback value for temporary calculation. It is initialized from `await response.json().catch(() => ({}))`. Within the function it is passed into `extractOllamaText`; assigned or updated later in the function.

answer: answer stores a computed value for temporary calculation. It is initialized from `extractOllamaText(data)`. Within the function it is returned to the caller; assigned or updated later in the function.

function ruleBasedConversationAnswer(question = "")

Role: Read `ruleBasedConversationAnswer` first as a named idea, not as syntax.

`ruleBasedConversationAnswer` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function ruleBasedConversationAnswer(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for ruleBasedConversationAnswer

```
function ruleBasedConversationAnswer(question = "") {  
  const clean = String(question || "").toLowerCase();  
  if (/b(thanks|thank you|appreciate it)\b/.test(clean)) {  
    return "You're welcome. I can keep helping with this portfolio, project evidence, resume links, or related  
electronics and embedded-systems questions.";  
  }  
  
  if (/b(who are you|what are you)\b/.test(clean)) {  
    return "I am this portfolio's assistant. I can help visitors explore the saved engineering work, explain project  
details, summarize portfolio evidence, and answer related electronics, embedded, analog, digital, FPGA, ASIC, PCB,  
and firmware questions.";  
  }  
  
  return "Hi. I can help you explore this portfolio, explain projects, summarize project evidence, open relevant  
sections, or answer related electronics and embedded-systems questions. You can ask about a specific project, a tool  
like KiCad or STM32CubeIDE, or a general topic such as embedded systems, op amps, FPGA design, ASICs, or PCB  
testing.";  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `ruleBasedConversationAnswer` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `ruleBasedConversationAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `ruleBasedConversationAnswer` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(question || "").toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

async function `handlePortfolioAi(request, response)`

Role: Begin with the role of `handlePortfolioAi`. `handlePortfolioAi` is the backend side of Ask My Portfolio. It reads the visitor's question, conversation history, intent, and context, enriches that context when public sources are allowed, then chooses an AI provider path. It can call OpenAI when configured, call Ollama locally when available, or fall back to rule-based answers when no model is available. The function matters because the public script should not hold private API keys or perform privileged source fetching directly. It also gives the assistant one place to enforce prompt rules about portfolio-specific answers, general engineering answers, public source usage, and uncertainty.

Function entry point:

```
async function handlePortfolioAi(request, response) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. `response` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `handlePortfolioAi`

```
async function handlePortfolioAi(request, response) {
  const body = await readRequestJson(request);
  const question = clampText(body.question, 1200).trim();
  const conversation = cleanConversationHistory(body.conversation);
  const validIntents = new Set(["general_conversation", "general_engineering", "general_knowledge",
    "portfolio_specific"]);
  const intent = validIntents.has(body.intent) ? body.intent : "portfolio_specific";
  const allowWebSearch = body.allowWebSearch === true;

  if (!question) {
    sendJson(response, 400, { error: "Question is required." });
    return;
  }

  const context = await enrichPortfolioContext({ ...(body.context || {}), question });

  const apiKey = process.env.OPENAI_API_KEY || "";
  if (!apiKey) {
    const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch });
    if (ollama.ok) {
      sendJson(response, 200, {
        answer: ollama.answer,
        model: ollama.model,
        provider: "ollama",
        usedWebSearch: false
      });
      return;
    }
    sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI_API_KEY is available for the local backend." });
    return;
  }

  const model = process.env.OPENAI_MODEL || "gpt-5.4";
  const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
  const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
```

```

const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
const buildPayload = (selectedModel) => ({
  model: selectedModel,
  input: [
    {
      role: "developer",
      content: [{ type: "input_text", text: portfolioAiInstructions }]
    },
    {
      role: "user",
      content: [{
        type: "input_text",
        text: [
          `Visitor question: ${question}`,
          `Question intent: ${intent}`,
          `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
          "",
          "Recent conversation JSON:",
          clampText(JSON.stringify(conversation, null, 2), 8000),
          "",
          "Portfolio context JSON:",
          clampText(JSON.stringify(context, null, 2), 18000)
        ].join("\n")
      ]
    }
  ],
  max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
  reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
  text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
});

const callModel = async (selectedModel) => {
  const payload = buildPayload(selectedModel);
  if (enableWebSearch) {
    payload.tools = [{ type: "web_search", search_context_size: process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE ||
"low" }];
  }

  const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${apiKey}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
  });

  const data = await openAiResponse.json().catch(() => ({}));
  return { data, openAiResponse, selectedModel };
};

let { data, openAiResponse, selectedModel } = await callModel(model);
if (!openAiResponse.ok && !process.env.OPENAI_MODEL && fallbackModel && fallbackModel !== model && [400,
404].includes(openAiResponse.status)) {
  ({ data, openAiResponse, selectedModel } = await callModel(fallbackModel));
}
if (!openAiResponse.ok) {
  sendJson(response, openAiResponse.status, {
    error: data?.error?.message || "OpenAI request failed."
  });
  return;
}

sendJson(response, 200, {
  answer: extractOpenAiText(data),
  model: selectedModel,
  usedWebSearch: enableWebSearch
});
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; performs a fetch and therefore must handle network delay and failure.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `readRequestJson`, `clampText`, `cleanConversationHistory`, `sendJson`, `enrichPortfolioContext`, `callOllamaPortfolioAi`, `extractOpenAiText`. Those calls show that `handlePortfolioAi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `handlePortfolioAi`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `handlePortfolioAi` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

body: `body` stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `allowWebSearch`, `context`, `conversation`, `intent`, `question`; passed into `clampText`, `cleanConversationHistory`, `enrichPortfolioContext`, `fetch`, `has`; assigned or updated later in the function.

question: `question` stores a computed value for temporary calculation. It is initialized from `clampText(body.question, 1200).trim()`. Within the function it is passed into `callOllamaPortfolioAi`, `clampText`; assigned or updated later in the function.

conversation: `conversation` stores a computed value for temporary calculation. It is initialized from `cleanConversationHistory(body.conversation)`. Within the function it is passed into `callOllamaPortfolioAi`, `clampText`, `cleanConversationHistory`; assigned or updated later in the function.

validIntents: `validIntents` stores a constructed object for lookup collection. It is initialized from `new Set(["general_conversation", "general_engineering", "general_knowledge", "portfolio_specific"])`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

intent: `intent` stores a computed value for temporary calculation. It is initialized from `validIntents.has(body.intent) ? body.intent : "portfolio_specific"`. Within the function it is passed into `callOllamaPortfolioAi`, `has`; assigned or updated later in the function.

allowWebSearch: `allowWebSearch` stores a computed value for temporary calculation. It is initialized from `body.allowWebSearch === true`. Within the function it is passed into `callOllamaPortfolioAi`; assigned or updated later in the function.

context: `context` stores the completed result of an awaited operation. It is initialized from `await enrichPortfolioContext({ ...(body.context || {}), question })`. Within the function it is passed into `callOllamaPortfolioAi`, `clampText`, `enrichPortfolioContext`; assigned or updated later in the function.

apiKey: `apiKey` stores a environment-derived value for temporary calculation. It is initialized from `process.env.OPENAI_API_KEY || ""`. Within the function it is passed into `fetch`; assigned or updated later in the function.

ollama: `ollama` stores the completed result of an awaited operation. It is initialized from `await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })`. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Within the function it is accesses properties/methods `answer`, `error`, `model`, `ok`; passed into `sendJson`; assigned or updated later in the function.

model: `model` stores a environment-derived value for temporary calculation. It is initialized from `process.env.OPENAI_MODEL || "gpt-5.4"`. Within the function it is passed into `callModel`, `sendJson`; assigned or updated later in the function.

fallbackModel: `fallbackModel` stores a environment-derived value for temporary calculation. It is initialized from `process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1"`. Within the function it is passed into `callModel`; assigned or updated later in the function.

webSearchMode: webSearchMode stores a environment-derived value for temporary calculation. It is initialized from `String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase()`. Within the function it is assigned or updated later in the function.

enableWebSearch: enableWebSearch stores a computed value for temporary calculation. It is initialized from `webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch)`. Within the function it is assigned or updated later in the function.

buildPayload: buildPayload stores a function or callback value for temporary calculation. It is initialized from `(selectedModel) => {}`. Within the function it is assigned or updated later in the function.

callModel: callModel stores a function or callback value for temporary calculation. It is initialized from `async (selectedModel) => {}`. Within the function it is assigned or updated later in the function.

payload: payload stores a computed value for temporary calculation. It is initialized from `buildPayload(selectedModel)`. Within the function it is accesses properties/methods tools; passed into fetch; assigned or updated later in the function.

openAiResponse: openAiResponse stores data returned from a network call. It is initialized from `await fetch("https://api.openai.com/v1/responses", {})`. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Within the function it is accesses properties/methods json, ok, status; passed into `sendJson`; assigned or updated later in the function.

data: data stores a function or callback value for temporary calculation. It is initialized from `await openAiResponse.json().catch(() => ({}))`. Within the function it is passed into `sendJson`; assigned or updated later in the function.

destructured { data, openAiResponse, selectedModel }: destructured { data, openAiResponse, selectedModel } stores the completed result of an awaited operation. It is initialized from `await callModel(model)`. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection.

Publishing and GitHub authorization

The functions in this group all support publishing and github authorization. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function publishAccessError(message, details = "", extra = {})

Role: The best entry point for publishAccessError is its responsibility in the surrounding workflow. publishAccessError belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function publishAccessError(message, details = "", extra = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. message is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. details is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. extra is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for publishAccessError

```
function publishAccessError(message, details = "", extra = {}) {  
  const error = new Error(message);  
  error.code = "PUBLISH_AUTHORIZATION_REQUIRED";  
  error.details = details || publishAuthorizationHelp;  
  error.publishAccess = {  
    authorizationRequired: true,  
    help: publishAuthorizationHelp,  
    ...extra  
  };  
  return error;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means publishAccessError performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without publishAccessError, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of publishAccessError is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

error: error stores a constructed object for temporary calculation. It is initialized from new Error(message). Within the function it is returned to the caller; accesses properties/methods code, details, publishAccess; assigned or updated later in the function.

function gitFailureText(error)

Role: To understand gitFailureText, start with the problem it owns. gitFailureText belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function gitFailureText(error) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. error is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for gitFailureText

```
function gitFailureText(error) {  
  return [  
    error?.stderr,  
    error?.stdout,  
    error?.message  
  ].filter(Boolean).join("\n").trim();  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `gitFailureText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `gitFailureText`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside `gitFailureText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `remoteUrlForDisplay(remoteUrl = "")`

Role: Begin with the role of `remoteUrlForDisplay`. `remoteUrlForDisplay` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function remoteUrlForDisplay(remoteUrl = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `remoteUrlForDisplay`

```
function remoteUrlForDisplay(remoteUrl = "") {  
  return String(remoteUrl || "")  
    .replace(/^https:\\\\\/([^\:\/]+):([^\:\/]+)\/i, "https://$1:**@")  
    .replace(/^https:\\\\\/([^\:\/]+)\/i, "https://");  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `remoteUrlForDisplay` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `remoteUrlForDisplay`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside `remoteUrlForDisplay`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function validatePublishRemoteUrl(value = "")

Role: Begin with the role of validatePublishRemoteUrl. validatePublishRemoteUrl is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function validatePublishRemoteUrl(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for validatePublishRemoteUrl

```
function validatePublishRemoteUrl(value = "") {  
  const remoteUrl = String(value || "").trim();  
  if (!remoteUrl) return "";  
  if (/^git@github\.com:[A-Za-z0-9_-]+\[/A-Za-z0-9_-]+(?:\.git)?$/i.test(remoteUrl)) return remoteUrl;  
  try {  
    const parsed = new URL(remoteUrl);  
    if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {  
      throw new Error("Only GitHub HTTPS repository URLs are supported by the guided setup.");  
    }  
    const parts = parsed.pathname.replace(/^\/+/, "").split("/");  
    if (parts.length < 2 || !parts[0] || !parts[1]) {  
      throw new Error("GitHub repository URL must include owner and repository name.");  
    }  
    return `https://github.com/${parts[0]}/${parts[1].replace(/\.git$/i, "")}.git`;  
  } catch (error) {  
    if (error.message?.includes("Only GitHub") || error.message?.includes("owner")) throw error;  
    throw new Error("Enter a GitHub repository URL such as https://github.com/USERNAME/USERNAME.github.io.git.");  
  }  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means validatePublishRemoteUrl performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without validatePublishRemoteUrl, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local state inside validatePublishRemoteUrl explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

remoteUrl: remoteUrl stores a computed value for network or routing value. It is initialized from String(value || "").trim(). Within the function it is returned to the caller; passed into URL, test; assigned or updated later in the function.

parsed: parsed stores a constructed object for temporary calculation. It is initialized from new URL(remoteUrl). Within the function it is accesses properties/methods hostname, pathname, protocol; assigned or updated later in the function.

parts: parts stores a computed value for temporary calculation. It is initialized from parsed.pathname.replace(/^\/+/, "").split("/"). Within the function it is accesses properties/methods length; assigned or updated later in the function.

function validateCustomDomain(value = "")

Role: To understand validateCustomDomain, start with the problem it owns. validateCustomDomain is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function validateCustomDomain(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for validateCustomDomain

```
function validateCustomDomain(value = "") {  
  const domain = String(value || "").trim().toLowerCase();  
  if (!domain) return "";  
  if (!/^(?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\.)+[a-z]{2,63}$/i.test(domain)) {  
    throw new Error("Custom domain must look like example.com or portfolio.example.com.");  
  }  
  return domain;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means validateCustomDomain performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without validateCustomDomain, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside validateCustomDomain reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

domain: domain stores a computed value for temporary calculation. It is initialized from String(value || "").trim().toLowerCase(). Within the function it is returned to the caller; passed into Error, test; assigned or updated later in the function.

async function bumpPublishedAssetVersions(baseRoot = portfolioRoot)

Role: To understand bumpPublishedAssetVersions, start with the problem it owns.

bumpPublishedAssetVersions belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function bumpPublishedAssetVersions(baseRoot = portfolioRoot) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. baseRoot is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for bumpPublishedAssetVersions

```
async function bumpPublishedAssetVersions(baseRoot = portfolioRoot) {
  let html = "";
  const indexPath = path.join(baseRoot, "index.html");
  try {
    html = await readFile(indexPath, "utf8");
  } catch {
    return false;
  }
  const version = Date.now().toString();
  const next = html
    .replace(/(href="styles\.css)(?:\?v=^[^"]*)?"/g, ` $1?v=${version}`)
    .replace(/(src="script\.js)(?:\?v=^[^"]*)?"/g, ` $1?v=${version}`)
    .replace(/(src="electronics-search\.js)(?:\?v=^[^"]*)?"/g, ` $1?v=${version}`);
  if (next === html) return false;
  await writeFile(indexPath, next, "utf8");
  return true;
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; writes data to disk as part of the operation; reads data from disk as part of the operation.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means bumpPublishedAssetVersions performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without bumpPublishedAssetVersions, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside bumpPublishedAssetVersions reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

html: html stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into join; assigned or updated later in the function.

indexPath: indexPath stores a resolved filesystem or route value. It is initialized from path.join(baseRoot, "index.html"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads,

previews, compiler artifacts, or published assets. Within the function it is passed into `readFile`, `writeFile`; assigned or updated later in the function.

version: version stores a computed value for temporary calculation. It is initialized from `Date.now().toString()`. Within the function it is assigned or updated later in the function.

next: next stores a computed value for temporary calculation. It is initialized from `html`. Within the function it is passed into `writeFile`; assigned or updated later in the function.

async function getPublishTargetInfo()

Role: Read `getPublishTargetInfo` first as a named idea, not as syntax. `getPublishTargetInfo` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function getPublishTargetInfo() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for getPublishTargetInfo

```
async function getPublishTargetInfo() {
  const info = {
    workspace: root,
    portfolioWorkspace: portfolioRoot,
    separatedWorkspace: !samePath(root, portfolioRoot),
    defaultRepository: defaultSiteRepository,
    gitBacked: false,
    compatible: await workspaceHasCompatibleSiteFiles(),
    remote: "",
    branch: "",
    repository: "",
    customDomain: ""
  };

  try {
    info.gitBacked = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim() === "true";
  } catch {
    return info;
  }

  try {
    info.remote = remoteUrlForDisplay((await runPublishGit(["remote", "get-url", "origin"])).stdout.trim());
  } catch {
    info.remote = "";
  }

  try {
    info.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  } catch {
    info.branch = "";
  }

  const parsedRemote = parseGitHubRemote(info.remote);
  info.repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : info.remote;

  try {
    info.customDomain = (await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8")).trim();
  } catch {
    info.customDomain = "";
  }

  const accessShape = {
    branch: info.branch,
    remote: info.remote,
  }
}
```



```

    repository: info.repository
  };
  const cached = await readPublishAuthCache();
  if (publishAuthCacheIsFresh(cached, accessShape)) {
    info.authorizationChecked = true;
    info.authorizationCached = true;
    info.checkedAt = cached.checkedAt;
    info.expiresAt = publishAuthCacheExpiresAt(cached);
    info.extendedTrust = Boolean(cached.extendedTrust);
    info.successCountLastWeek = cached.successCountLastWeek || 0;
    info.trustDays = cached.trustDays || (cached.extendedTrust ? 30 : 1);
  } else {
    info.authorizationChecked = false;
    info.authorizationCached = false;
    info.checkedAt = "";
    info.expiresAt = "";
    info.extendedTrust = false;
    info.successCountLastWeek = cached?.successCountLastWeek || 0;
    info.trustDays = 0;
  }
  return info;
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

Collaborators: The function also has to be read through the helpers it calls. It works with `samePath`, `workspaceHasCompatibleSiteFiles`, `runPublishGit`, `remoteUrlForDisplay`, `parseGitHubRemote`, `resolveInsidePortfolioRoot`, `readPublishAuthCache`, `publishAuthCacheIsFresh`. Those calls show that `getPublishTargetInfo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `getPublishTargetInfo`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local objects and variables inside `getPublishTargetInfo` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

info: `info` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `authorizationCached`, `authorizationChecked`, `branch`, `checkedAt`, `customDomain`, `expiresAt`, `extendedTrust`, `gitBacked`; passed into `parseGitHubRemote`; assigned or updated later in the function.

parsedRemote: `parsedRemote` stores a computed value for network or routing value. It is initialized from `parseGitHubRemote(info.remote)`. Within the function it is accesses properties/methods `owner`, `repo`; assigned or updated later in the function.

accessShape: `accessShape` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is assigned or updated later in the function.

cached: `cached` stores the completed result of an awaited operation. It is initialized from `await readPublishAuthCache()`. Within the function it is accesses properties/methods `checkedAt`, `extendedTrust`, `successCountLastWeek`, `trustDays`; passed into `Boolean`, `publishAuthCacheExpiresAt`; assigned or updated later in the function.

async function runGit(args, options = {})

Role: To understand `runGit`, start with the problem it owns. `runGit` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Function entry point:

```
async function runGit(args, options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `runGit`

```
async function runGit(args, options = {}) {  
  let lastError = null;  
  for (const candidate of gitCandidates) {  
    try {  
      const result = await execFileAsync(candidate, args, {  
        cwd: options.cwd || root,  
        maxBuffer: options.maxBuffer || 10 * 1024 * 1024,  
        timeout: options.timeout || 0,  
        windowsHide: options.windowsHide !== false  
      });  
      return { ...result, git: candidate };  
    } catch (error) {  
      lastError = error;  
      if (error.code === "ENOENT") continue;  
      throw error;  
    }  
  }  
  throw lastError || new Error("Git executable was not found.");  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `runGit` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `runGit`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside `runGit` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lastError: `lastError` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into `execFileAsync`.

result: `result` stores the completed result of an awaited operation. It is initialized from `await execFileAsync(candidate, args, {`. Within the function it is assigned or updated later in the function.

async function runPublishGit(args, options = {})

Role: Begin with the role of runPublishGit. runPublishGit executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Function entry point:

```
async function runPublishGit(args, options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. args is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for runPublishGit

```
async function runPublishGit(args, options = {}) {  
  return runGit(args, { ...options, cwd: options.cwd || portfolioRoot });  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with runGit. Those calls show that runPublishGit is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without runPublishGit, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside runPublishGit, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function getGitStatus()

Role: Begin with the role of getGitStatus. getGitStatus belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function getGitStatus() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for getGitStatus

```
async function getGitStatus() {  
  let git = { ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." };  
  try {
```

```

    const gitResult = await runGit(["--version"]);
    git = { ok: true, stdout: gitResult.stdout || "", stderr: gitResult.stderr || "" };
  } catch (error) {
    git = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message || "Git for
Windows was not found." };
  }
  let credentialManager = { ok: false, version: "", error: "Git Credential Manager was not found." };
  if (git.ok) {
    let gcm = { ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." };
    try {
      const gcmResult = await runGit(["credential-manager", "--version"]);
      gcm = { ok: true, stdout: gcmResult.stdout || "", stderr: gcmResult.stderr || "" };
    } catch (error) {
      gcm = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message || "Git
Credential Manager was not found." };
    }
    credentialManager = {
      ok: gcm.ok,
      version: (gcm.stdout || gcm.stderr || "").trim(),
      error: gcm.ok ? "" : (gcm.stderr || gcm.error || "Git Credential Manager was not found.")
    };
  }

  return {
    git: {
      ok: git.ok,
      version: (git.stdout || git.stderr || "").trim(),
      error: git.ok ? "" : (git.stderr || git.error || "Git for Windows was not found.")
    },
    credentialManager,
    nodeRuntime: {
      ok: true,
      version: process.versions.node,
      bundled: true,
      note: "Bundled inside the desktop app. No manual Node.js install is required."
    },
    pnpm: {
      ok: true,
      required: false,
      note: "pnpm is only needed by developers building the installer from source."
    },
    app: {
      version: process.env.OMB_APP_VERSION || packageJson.version || "0.0.0",
      desktopApp: process.env.OMB_DESKTOP_APP === "1"
    }
  };
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `runGit`. Those calls show that `getGitStatus` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `getGitStatus`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local state inside `getGitStatus` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

git: git stores a structured object for Git publishing and authorization state. It is initialized from `{ ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." }`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `error`, `ok`, `stderr`, `stdout`; assigned or updated later in the function.

gitResult: gitResult stores the completed result of an awaited operation. It is initialized from await runGit(["--version"]). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stderr, stdout; assigned or updated later in the function.

credentialManager: credentialManager stores a structured object for Git publishing and authorization state. It is initialized from { ok: false, version: "", error: "Git Credential Manager was not found." }. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

gcm: gcm stores a structured object for temporary calculation. It is initialized from { ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." }. Within the function it is accesses properties/methods error, ok, stderr, stdout; assigned or updated later in the function.

gcmResult: gcmResult stores the completed result of an awaited operation. It is initialized from await runGit(["credential-manager", "--version"]). Within the function it is accesses properties/methods stderr, stdout; assigned or updated later in the function.

async function publishPathIsStageable(relativePath)

Role: Begin with the role of publishPathIsStageable. publishPathIsStageable belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function publishPathIsStageable(relativePath) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. relativePath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for publishPathIsStageable

```
async function publishPathIsStageable(relativePath) {  
  try {  
    await fsAccess(resolveInsidePortfolioRoot(relativePath));  
    return true;  
  } catch {  
    // Missing files can still be stageable if Git already tracks them and they were deleted.  
  }  
  
  try {  
    await runPublishGit(["ls-files", "--error-unmatch", "--", relativePath]);  
    return true;  
  } catch {  
    return false;  
  }  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with resolveInsidePortfolioRoot, runPublishGit. Those calls show that publishPathIsStageable is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without publishPathIsStageable, the publishing flow would lose one guardrail or one mechanical step.

Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside `publishPathsStageable`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function stageablePublishPaths(pathsToCheck)

Role: The best entry point for `stageablePublishPaths` is its responsibility in the surrounding workflow. `stageablePublishPaths` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function stageablePublishPaths(pathsToCheck) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `pathsToCheck` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for stageablePublishPaths

```
async function stageablePublishPaths(pathsToCheck) {  
  const stageable = [];  
  for (const relativePath of pathsToCheck) {  
    if (await publishPathIsStageable(relativePath)) stageable.push(relativePath);  
  }  
  return stageable;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `publishPathsStageable`. Those calls show that `stageablePublishPaths` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `stageablePublishPaths`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of `stageablePublishPaths` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

stageable: `stageable` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

relativePath: `relativePath` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`.

async function runGitWithInput(args, input, options = {})

Role: To understand runGitWithInput, start with the problem it owns. runGitWithInput executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

Function entry point:

```
async function runGitWithInput(args, input, options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. args is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. input is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for runGitWithInput

```
async function runGitWithInput(args, input, options = {}) {
  let lastError = null;
  for (const candidate of gitCandidates) {
    try {
      return await new Promise((resolve, reject) => {
        const child = spawn(candidate, args, {
          cwd: options.cwd || portfolioRoot,
          windowsHide: true,
          stdio: ["pipe", "pipe", "pipe"]
        });
        let stdout = "";
        let stderr = "";

        child.stdout.on("data", (chunk) => {
          stdout += chunk.toString();
        });
        child.stderr.on("data", (chunk) => {
          stderr += chunk.toString();
        });
        child.once("error", reject);
        child.once("close", (code) => {
          if (code === 0) {
            resolve({ stdout, stderr, git: candidate });
            return;
          }
          const error = new Error(stderr || stdout || `Git exited with code ${code}.`);
          error.stdout = stdout;
          error.stderr = stderr;
          error.code = code;
          error.git = candidate;
          reject(error);
        });

        child.stdin.end(input);
      });
    } catch (error) {
      lastError = error;
      if (error.code === "ENOENT") continue;
      throw error;
    }
  }
  throw lastError || new Error("Git executable was not found.");
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and runs an external command. It

returns the awaited result of another asynchronous operation, so its caller receives the completed value rather than a pending Promise.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `runGitWithInput` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `runGitWithInput`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside `runGitWithInput` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lastError: `lastError` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into `resolve`, `spawn`.

child: `child` stores a computed value for temporary calculation. It is initialized from `spawn(candidate, args, {`. Within the function it is accesses properties/methods `once`, `stderr`, `stdin`, `stdout`; assigned or updated later in the function.

stdout: `stdout` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is accesses properties/methods `on`; passed into `Error`, `resolve`; assigned or updated later in the function.

stderr: `stderr` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is accesses properties/methods `on`; passed into `Error`, `resolve`; assigned or updated later in the function.

error: `error` stores a constructed object for temporary calculation. It is initialized from `new Error(stderr || stdout || `Git exited with code ${code}.`)`. Within the function it is accesses properties/methods `code`, `git`, `stderr`, `stdout`; passed into `catch`, `once`, `reject`; assigned or updated later in the function.

async function storeGitCredentials(remoteUrl, username, password)

Role: The best entry point for `storeGitCredentials` is its responsibility in the surrounding workflow. `storeGitCredentials` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function storeGitCredentials(remoteUrl, username, password) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `username` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `password` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for storeGitCredentials

```
async function storeGitCredentials(remoteUrl, username, password) {
```

```

const cleanUsername = String(username || "").trim();
const cleanPassword = String(password || "");
if (!cleanUsername && !cleanPassword) return { stored: false };
if (!cleanUsername || !cleanPassword) {
  throw new Error("Both username and password/token are required when credentials are provided.");
}

let parsed = null;
try {
  parsed = new URL(remoteUrl);
} catch {
  throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
}

if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
  throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
}

const pathName = parsed.pathname.replace(/^\/+/, "");
await runPublishGit(["config", "--local", "credential.useHttpPath", "true"]);
const credentialInput = [
  "protocol=https",
  "host=github.com",
  `path=${pathName}`,
  `username=${cleanUsername}`,
  `password=${cleanPassword}`,
  ""
].join("\n");
await runGitWithInput(["credential", "approve"], credentialInput, { cwd: portfolioRoot });
return { stored: true, username: cleanUsername };
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `runPublishGit`, `runGitWithInput`. Those calls show that `storeGitCredentials` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `storeGitCredentials`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of `storeGitCredentials` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanUsername: `cleanUsername` stores a computed value for temporary calculation. It is initialized from `String(username || "").trim()`. Within the function it is assigned or updated later in the function.

cleanPassword: `cleanPassword` stores a computed value for temporary calculation. It is initialized from `String(password || "").trim()`. Within the function it is assigned or updated later in the function.

parsed: `parsed` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is accesses properties/methods `hostname`, `pathname`, `protocol`; assigned or updated later in the function.

pathName: `pathName` stores a computed value for filesystem location. It is initialized from `parsed.pathname.replace(/^\/+/, "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is assigned or updated later in the function.

credentialInput: `credentialInput` stores a list for Git publishing and authorization state. It is initialized from `[]`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runGitWithInput`; assigned or updated later in the function.

function validateCredentialPair(username, password)

Role: To understand validateCredentialPair, start with the problem it owns. validateCredentialPair is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function validateCredentialPair(username, password) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. username is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. password is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for validateCredentialPair

```
function validateCredentialPair(username, password) {  
  const cleanUsername = String(username || "").trim();  
  const cleanPassword = String(password || "");  
  if ((cleanUsername && !cleanPassword) || (!cleanUsername && cleanPassword)) {  
    throw new Error("Both username and password/token are required when credentials are provided.");  
  }  
  return { cleanUsername, cleanPassword };  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means validateCredentialPair performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without validateCredentialPair, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside validateCredentialPair reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanUsername: cleanUsername stores a computed value for temporary calculation. It is initialized from String(username || "").trim(). Within the function it is assigned or updated later in the function.

cleanPassword: cleanPassword stores a computed value for temporary calculation. It is initialized from String(password || ""). Within the function it is assigned or updated later in the function.

function normalizeGitBranchName(value = "")

Role: Begin with the role of normalizeGitBranchName. normalizeGitBranchName standardizes GitBranchName data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeGitBranchName(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `normalizeGitBranchName`

```
function normalizeGitBranchName(value = "") {  
  const branch = String(value || "").trim();  
  if (  
    !branch ||  
    branch.length > 180 ||  
    branch.includes("..") ||  
    branch.endsWith(".") ||  
    branch.includes("@{") ||  
    /[\\:\s~^?*[\]\x00-\x1f]/.test(branch) ||  
    branch.startsWith("/") ||  
    branch.endsWith("/") ||  
    branch.includes("//")  
  ) {  
    return "";  
  }  
  return branch;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `normalizeGitBranchName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `normalizeGitBranchName`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local state inside `normalizeGitBranchName` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

branch: `branch` stores a computed value for Git publishing and authorization state. It is initialized from `String(value || "").trim()`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; accesses properties/methods `endsWith`, `includes`, `length`, `startsWith`; passed into `test`; assigned or updated later in the function.

`async function detectRemoteDefaultBranch(remoteUrl = "")`

Role: Begin with the role of `detectRemoteDefaultBranch`. `detectRemoteDefaultBranch` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Function entry point:

```
async function detectRemoteDefaultBranch(remoteUrl = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using

it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for detectRemoteDefaultBranch

```
async function detectRemoteDefaultBranch(remoteUrl = "") {
  if (!remoteUrl) return "";
  try {
    const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
    const match = String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+)\s+HEAD$/m);
    return normalizeGitBranchName(match?.[1] || "");
  } catch {
    return "";
  }
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `runPublishGit`, `normalizeGitBranchName`. Those calls show that `detectRemoteDefaultBranch` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `detectRemoteDefaultBranch`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local state inside `detectRemoteDefaultBranch` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

result: `result` stores the completed result of an awaited operation. It is initialized from `await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 })`. Within the function it is accesses properties/methods `stdout`; passed into `String`; assigned or updated later in the function.

match: `match` stores a computed value for temporary calculation. It is initialized from `String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+)\s+HEAD$/m)`. Within the function it is passed into `normalizeGitBranchName`; assigned or updated later in the function.

async function originRemoteExists()

Role: Begin with the role of `originRemoteExists`. `originRemoteExists` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function originRemoteExists() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for originRemoteExists

```
async function originRemoteExists() {
  try {
```

```

    await runPublishGit(["remote", "get-url", "origin"]);
    return true;
  } catch {
    return false;
  }
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `runPublishGit`. Those calls show that `originRemoteExists` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `originRemoteExists`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside `originRemoteExists`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function setOriginRemote(remoteUrl)

Role: Begin with the role of `setOriginRemote`. `setOriginRemote` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```

async function setOriginRemote(remoteUrl) { ... }

```

Inputs: The function-specific inputs are worth reading before the body. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for setOriginRemote

```

async function setOriginRemote(remoteUrl) {
  if (await originRemoteExists()) {
    await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
  } else {
    await runPublishGit(["remote", "add", "origin", remoteUrl]);
  }
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `originRemoteExists`, `runPublishGit`. Those calls show that `setOriginRemote` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `setOriginRemote`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside `setOriginRemote`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function checkoutPublishBranch(branch)

Role: The best entry point for `checkoutPublishBranch` is its responsibility in the surrounding workflow. `checkoutPublishBranch` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function checkoutPublishBranch(branch) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `branch` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for checkoutPublishBranch

```
async function checkoutPublishBranch(branch) {  
  const cleanBranch = normalizeGitBranchName(branch) || "main";  
  const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();  
  if (current === cleanBranch) return cleanBranch;  
  try {  
    await runPublishGit(["checkout", cleanBranch]);  
  } catch {  
    await runPublishGit(["checkout", "-B", cleanBranch]);  
  }  
  return cleanBranch;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeGitBranchName`, `runPublishGit`. Those calls show that `checkoutPublishBranch` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `checkoutPublishBranch`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of `checkoutPublishBranch` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanBranch: `cleanBranch` stores a computed value for Git publishing and authorization state. It is initialized from `normalizeGitBranchName(branch) || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; passed into `runPublishGit`; assigned or updated later in the function.

current: `current` stores a function or callback value for temporary calculation. It is initialized from `(await runPublishGit(["branch", "--show-current"])).stdout.trim()`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

async function verifyPublishWriteAccess(access = {})

Role: To understand verifyPublishWriteAccess, start with the problem it owns. verifyPublishWriteAccess belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function verifyPublishWriteAccess(access = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. access is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for verifyPublishWriteAccess

```
async function verifyPublishWriteAccess(access = {}) {  
  const checkBranch = `omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}`;  
  try {  
    await runPublishGit(["push", "--dry-run", "origin", `HEAD:refs/heads/${checkBranch}`], {  
      timeout: 60 * 1000  
    });  
    return {  
      method: "temporary-dry-run-ref",  
      ref: checkBranch  
    };  
  } catch (error) {  
    throw publishAccessError(  
      "GitHub authorization is required before this website can be changed.",  
      gitFailureText(error),  
      access  
    );  
  }  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Next, trace the helper calls that surround the function. It works with safeSegment, runPublishGit, publishAccessError, gitFailureText. Those calls show that verifyPublishWriteAccess is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without verifyPublishWriteAccess, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside verifyPublishWriteAccess reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

checkBranch: checkBranch stores a string value for Git publishing and authorization state. It is initialized from `omb-auth-check-\${safeSegment(os.hostname(), "device")}-\${Date.now()}`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into runPublishGit; assigned or updated later in the function.

async function ensurePublishHeadForWriteCheck()

Role: The best entry point for ensurePublishHeadForWriteCheck is its responsibility in the surrounding workflow. ensurePublishHeadForWriteCheck creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Function entry point:

```
async function ensurePublishHeadForWriteCheck() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for ensurePublishHeadForWriteCheck

```
async function ensurePublishHeadForWriteCheck() {
  try {
    await runPublishGit(["rev-parse", "--verify", "HEAD"]);
    return { created: false };
  } catch {
    await runPublishGit([
      "-c",
      "user.name=OMB Portfolio Builder",
      "-c",
      "user.email=omb-builder@localhost",
      "commit",
      "--allow-empty",
      "-m",
      "Initialize publish authorization check"
    ]);
    return { created: true };
  }
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with runPublishGit. Those calls show that ensurePublishHeadForWriteCheck is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without ensurePublishHeadForWriteCheck, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside ensurePublishHeadForWriteCheck, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function synchronizePublishBranchFromRemote(branch, access = {})

Role: To understand synchronizePublishBranchFromRemote, start with the problem it owns. synchronizePublishBranchFromRemote reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

Function entry point:

```
async function synchronizePublishBranchFromRemote(branch, access = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. branch is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. access is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more

predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `synchronizePublishBranchFromRemote`

```
async function synchronizePublishBranchFromRemote(branch, access = {}) {
  const cleanBranch = normalizeGitBranchName(branch) || "main";
  try {
    const remoteBranch = await runPublishGit(["ls-remote", "--heads", "origin", cleanBranch], {
      timeout: 45 * 1000
    });
    if (!String(remoteBranch.stdout || "").trim()) {
      return {
        branch: cleanBranch,
        remoteBranchExists: false,
        synchronized: false
      };
    }
    await runPublishGit(["fetch", "origin", cleanBranch], { timeout: 2 * 60 * 1000 });
    await checkoutPublishBranch(cleanBranch);
    await runPublishGit(["reset", "--hard", `origin/${cleanBranch}`], { timeout: 60 * 1000 });
    return {
      branch: cleanBranch,
      remoteBranchExists: true,
      synchronized: true
    };
  } catch (error) {
    throw publishAccessError(
      "The selected website target could not be synchronized from GitHub.",
      gitFailureText(error),
      { ...access, branch: cleanBranch }
    );
  }
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeGitBranchName`, `runPublishGit`, `checkoutPublishBranch`, `publishAccessError`, `gitFailureText`. Those calls show that `synchronizePublishBranchFromRemote` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `synchronizePublishBranchFromRemote`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside `synchronizePublishBranchFromRemote` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanBranch: `cleanBranch` stores a computed value for Git publishing and authorization state. It is initialized from `normalizeGitBranchName(branch) || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `checkoutPublishBranch`, `runPublishGit`; assigned or updated later in the function.

remoteBranch: `remoteBranch` stores the completed result of an awaited operation. It is initialized from `await runPublishGit(["ls-remote", "--heads", "origin", cleanBranch], {`. It affects publishing, where mistakes can

change the public website or expose the wrong files. Within the function it is accesses properties/methods stdout; assigned or updated later in the function.

async function capturePublishTargetState()

Role: The best entry point for capturePublishTargetState is its responsibility in the surrounding workflow. capturePublishTargetState belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function capturePublishTargetState() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for capturePublishTargetState

```
async function capturePublishTargetState() {
  const snapshot = {
    branch: "",
    customDomainExists: false,
    customDomainText: "",
    remote: "",
    remoteExists: false
  };
  try {
    snapshot.remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
    snapshot.remoteExists = true;
  } catch {
    snapshot.remote = "";
    snapshot.remoteExists = false;
  }
  try {
    snapshot.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  } catch {
    snapshot.branch = "";
  }
  try {
    snapshot.customDomainText = await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8");
    snapshot.customDomainExists = true;
  } catch {
    snapshot.customDomainText = "";
    snapshot.customDomainExists = false;
  }
  return snapshot;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with runPublishGit, resolveInsidePortfolioRoot. Those calls show that capturePublishTargetState is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without capturePublishTargetState, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of capturePublishTargetState is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

snapshot: snapshot stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `branch`, `customDomainExists`, `customDomainText`, `remote`, `remoteExists`; assigned or updated later in the function.

async function restorePublishTargetState(snapshot = {})

Role: The best entry point for `restorePublishTargetState` is its responsibility in the surrounding workflow. `restorePublishTargetState` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function restorePublishTargetState(snapshot = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `snapshot` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for restorePublishTargetState

```
async function restorePublishTargetState(snapshot = {}) {
  try {
    if (snapshot.remoteExists && snapshot.remote) {
      await setOriginRemote(snapshot.remote);
    } else if (await originRemoteExists()) {
      await runPublishGit(["remote", "remove", "origin"]);
    }
  } catch {
    // Best-effort rollback; preserve the original error for the caller.
  }
  try {
    if (snapshot.branch) await checkoutPublishBranch(snapshot.branch);
  } catch {
    // Best-effort rollback; preserve the original error for the caller.
  }
  try {
    if (snapshot.customDomainExists) {
      await writeFile(resolveInsidePortfolioRoot("CNAME"), snapshot.customDomainText, "utf8");
    } else {
      await rm(resolveInsidePortfolioRoot("CNAME"), { force: true });
    }
  } catch {
    // Best-effort rollback; preserve the original error for the caller.
  }
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes data to disk as part of the operation.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `setOriginRemote`, `originRemoteExists`, `runPublishGit`, `checkoutPublishBranch`, `resolveInsidePortfolioRoot`. Those calls show that `restorePublishTargetState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `restorePublishTargetState`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside `restorePublishTargetState`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function writeTargetCustomDomain(domain, customDomainProvided)

Role: To understand `writeTargetCustomDomain`, start with the problem it owns. `writeTargetCustomDomain` writes `TargetCustomDomain` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Function entry point:

```
async function writeTargetCustomDomain(domain, customDomainProvided) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `domain` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `customDomainProvided` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for writeTargetCustomDomain

```
async function writeTargetCustomDomain(domain, customDomainProvided) {  
  const targetPaths = [resolveInsidePortfolioRoot("CNAME")];  
  if (!samePath(root, portfolioRoot)) targetPaths.push(resolveInsideRoot("CNAME"));  
  if (customDomainProvided && domain) {  
    for (const targetPath of targetPaths) await writeFile(targetPath, `${domain}\n`, "utf8");  
  } else if (customDomainProvided && !domain) {  
    for (const targetPath of targetPaths) await rm(targetPath, { force: true });  
  }  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes data to disk as part of the operation.

Collaborators: Next, trace the helper calls that surround the function. It works with `resolveInsidePortfolioRoot`, `samePath`, `resolveInsideRoot`. Those calls show that `writeTargetCustomDomain` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `writeTargetCustomDomain`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside `writeTargetCustomDomain` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

targetPaths: `targetPaths` stores a list for filesystem location. It is initialized from `[resolveInsidePortfolioRoot("CNAME")]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

targetPath: targetPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into rm, writeFile.

targetPath: targetPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into rm, writeFile.

async function ensureGitRepository()

Role: Read ensureGitRepository first as a named idea, not as syntax. ensureGitRepository creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Function entry point:

```
async function ensureGitRepository() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for ensureGitRepository

```
async function ensureGitRepository() {
  await mkdir(portfolioRoot, { recursive: true });
  try {
    const insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
    if (insideWorkTree === "true") return;
  } catch {
    await runPublishGit(["init"]);
  }

  const branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  if (!branch) {
    await runPublishGit(["checkout", "-B", "main"]);
  }
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: creates folders before writing files so missing directories do not crash the workflow.

Collaborators: The function also has to be read through the helpers it calls. It works with runPublishGit. Those calls show that ensureGitRepository is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without ensureGitRepository, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local objects and variables inside ensureGitRepository show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

insideWorkTree: insideWorkTree stores a function or callback value for temporary calculation. It is initialized from (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim(). Within the function it is assigned or updated later in the function.

branch: branch stores a function or callback value for Git publishing and authorization state. It is initialized from `(await runPublishGit(["branch", "--show-current"])).stdout.trim()`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

async function `configurePublishTarget(options = {})`

Role: Begin with the role of `configurePublishTarget`. `configurePublishTarget` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function configurePublishTarget(options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `configurePublishTarget`

```
async function configurePublishTarget(options = {}) {
  const {
    repositoryUrl = "",
    customDomain = "",
    authUsername = "",
    authPassword = ""
  } = options;
  const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
  const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
  const domain = validateCustomDomain(customDomain);
  validateCredentialPair(authUsername, authPassword);

  await ensureGitRepository();

  if (remoteUrl) {
    await setOriginRemote(remoteUrl);
    const defaultBranch = await detectRemoteDefaultBranch(remoteUrl);
    if (defaultBranch) await checkoutPublishBranch(defaultBranch);
  }

  await writeTargetCustomDomain(domain, customDomainProvided);

  const currentRemote = remoteUrl || (await getPublishTargetInfo()).remote;
  const credentials = await storeGitCredentials(currentRemote, authUsername, authPassword);
  const target = await getPublishTargetInfo();
  return {
    ...target,
    credentialsStored: credentials.stored,
    credentialUsername: credentials.username || ""
  };
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `validatePublishRemoteUrl`, `validateCustomDomain`, `validateCredentialPair`, `ensureGitRepository`, `setOriginRemote`, `detectRemoteDefaultBranch`, `checkoutPublishBranch`, `writeTargetCustomDomain`. Those calls

show that `configurePublishTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `configurePublishTarget`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local state inside `configurePublishTarget` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

customDomainProvided: `customDomainProvided` stores a computed value for temporary calculation. It is initialized from `Object.prototype.hasOwnProperty.call(options, "customDomain")`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

remoteUrl: `remoteUrl` stores a computed value for network or routing value. It is initialized from `validatePublishRemoteUrl(repositoryUrl)`. Within the function it is passed into `detectRemoteDefaultBranch`, `setOriginRemote`; assigned or updated later in the function.

domain: `domain` stores a computed value for temporary calculation. It is initialized from `validateCustomDomain(customDomain)`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

defaultBranch: `defaultBranch` stores the completed result of an awaited operation. It is initialized from `await detectRemoteDefaultBranch(remoteUrl)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `checkoutPublishBranch`; assigned or updated later in the function.

currentRemote: `currentRemote` stores the completed result of an awaited operation. It is initialized from `remoteUrl || (await getPublishTargetInfo()).remote`. Within the function it is passed into `storeGitCredentials`; assigned or updated later in the function.

credentials: `credentials` stores the completed result of an awaited operation. It is initialized from `await storeGitCredentials(currentRemote, authUsername, authPassword)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stored, `username`; assigned or updated later in the function.

target: `target` stores the completed result of an awaited operation. It is initialized from `await getPublishTargetInfo()`. Within the function it is assigned or updated later in the function.

async function readPublishAuthCache()

Role: To understand `readPublishAuthCache`, start with the problem it owns. `readPublishAuthCache` reads `PublishAuthCache` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

Function entry point:

```
async function readPublishAuthCache() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for readPublishAuthCache

```
async function readPublishAuthCache() {  
  try {  
    return JSON.parse(await readFile(publishAuthCachePath, "utf8"));  
  } catch {  
    return null;  
  }  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data; reads data from disk as part of the operation.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `readPublishAuthCache` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `readPublishAuthCache`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: Inside `readPublishAuthCache`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function writePublishAuthCache(access = {})

Role: Begin with the role of `writePublishAuthCache`. `writePublishAuthCache` writes `PublishAuthCache` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

Function entry point:

```
async function writePublishAuthCache(access = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for writePublishAuthCache

```
async function writePublishAuthCache(access = {}) {
  const previousCache = await readPublishAuthCache();
  const checkedAt = new Date();
  const checkedAtMs = checkedAt.getTime();
  const sameTarget = previousCache
    && previousCache.remote === access.remote
    && previousCache.branch === access.branch
    && previousCache.repository === access.repository;
  const previousHistory = sameTarget && Array.isArray(previousCache.successHistory)
    ? previousCache.successHistory
    : [];
  const successTimestamps = [
    ...previousHistory
      .map((entry) => parsePublishAuthTimestamp(entry))
      .filter((timestamp) => Number.isFinite(timestamp))
      .filter((timestamp) => checkedAtMs - timestamp < publishAuthExtendedTtlMs),
    checkedAtMs
  ];
  const successHistory = successTimestamps.map((timestamp) => new Date(timestamp).toISOString());
  const successCountLastWeek = successTimestamps
    .filter((timestamp) => checkedAtMs - timestamp < publishAuthHistoryWindowMs)
    .length;
  const extendedTrust = successCountLastWeek > publishAuthExtendedThreshold;
  const ttlMs = extendedTrust ? publishAuthExtendedTtlMs : publishAuthCacheTtlMs;
  const cache = {
    branch: access.branch || "",
    checkedAt: checkedAt.toISOString(),
    expiresAt: new Date(checkedAtMs + ttlMs).toISOString(),
  };
}
```

```

    extendedTrust,
    remote: access.remote || "",
    repository: access.repository || "",
    scope: publishAuthCacheScope(),
    successCountLastWeek,
    successHistory,
    trustDays: extendedTrust ? 30 : 1
  };
  await writeFile(publishAuthCachePath, `${JSON.stringify(cache, null, 2)}\n`, "utf8");
  await chmod(publishAuthCachePath, 0o600).catch(() => {});
  return cache;
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; writes data to disk as part of the operation.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `readPublishAuthCache`, `parsePublishAuthTimestamp`, `publishAuthCacheScope`. Those calls show that `writePublishAuthCache` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `writePublishAuthCache`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local state inside `writePublishAuthCache` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

previousCache: `previousCache` stores the completed result of an awaited operation. It is initialized from `await readPublishAuthCache()`. Within the function it is accesses properties/methods `branch`, `remote`, `repository`, `successHistory`; passed into `isArray`; assigned or updated later in the function.

checkedAt: `checkedAt` stores a constructed object for temporary calculation. It is initialized from `new Date()`. Within the function it is accesses properties/methods `getTime`, `toISOString`; assigned or updated later in the function.

checkedAtMs: `checkedAtMs` stores a computed value for temporary calculation. It is initialized from `checkedAt.getTime()`. Within the function it is passed into `Date`; assigned or updated later in the function.

sameTarget: `sameTarget` stores a computed value for temporary calculation. It is initialized from `previousCache`. Within the function it is assigned or updated later in the function.

previousHistory: `previousHistory` stores a computed value for runtime state or cache. It is initialized from `sameTarget && Array.isArray(previousCache.successHistory)`. Within the function it is assigned or updated later in the function.

successTimestamps: `successTimestamps` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

successHistory: `successHistory` stores a function or callback value for runtime state or cache. It is initialized from `successTimestamps.map((timestamp) => new Date(timestamp).toISOString())`. Within the function it is passed into `isArray`; assigned or updated later in the function.

successCountLastWeek: `successCountLastWeek` stores a computed value for temporary calculation. It is initialized from `successTimestamps`. Within the function it is assigned or updated later in the function.

extendedTrust: `extendedTrust` stores a computed value for temporary calculation. It is initialized from `successCountLastWeek > publishAuthExtendedThreshold`. Within the function it is assigned or updated later in the function.

ttlMs: ttlMs stores a computed value for temporary calculation. It is initialized from extendedTrust ?
publishAuthExtendedTtlMs : publishAuthCacheTtlMs. Within the function it is passed into Date; assigned or updated later in the function.

cache: cache stores a structured object for runtime state or cache. It is initialized from {}. Within the function it is returned to the caller; passed into writeFile; assigned or updated later in the function.

function publishAuthCacheScope()

Role: The best entry point for publishAuthCacheScope is its responsibility in the surrounding workflow. publishAuthCacheScope belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function publishAuthCacheScope() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for publishAuthCacheScope

```
function publishAuthCacheScope() {  
  let username = process.env.USERNAME || process.env.USER || "";  
  try {  
    username = os.userInfo().username || username;  
  } catch {  
    // Environment username is a sufficient fallback for cache scoping.  
  }  
  return createHash("sha256")  
    .update([  
      os.hostname(),  
      username,  
      root,  
      portfolioRoot  
    ].join("\n"))  
    .digest("hex");  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means publishAuthCacheScope performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without publishAuthCacheScope, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of publishAuthCacheScope is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

username: username stores a environment-derived value for temporary calculation. It is initialized from process.env.USERNAME || process.env.USER || "". Within the function it is assigned or updated later in the function.

function parsePublishAuthTimestamp(value = "")

Role: Begin with the role of parsePublishAuthTimestamp. parsePublishAuthTimestamp interprets PublishAuthTimestamp text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parsePublishAuthTimestamp(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for parsePublishAuthTimestamp

```
function parsePublishAuthTimestamp(value = "") {  
  const direct = Date.parse(String(value || ""));  
  if (Number.isFinite(direct)) return direct;  
  const normalized = String(value || "").replace(/(\.\d{3})\d+(Z|[-+]\d{2}:?\d{2})$/i, "$1$2");  
  return Date.parse(normalized);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means parsePublishAuthTimestamp performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without parsePublishAuthTimestamp, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local state inside parsePublishAuthTimestamp explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

direct: direct stores a computed value for filesystem location. It is initialized from Date.parse(String(value || "")). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is returned to the caller; assigned or updated later in the function.

normalized: normalized stores a computed value for temporary calculation. It is initialized from String(value || "").replace(/(\.\d{3})\d+(Z|[-+]\d{2}:?\d{2})\$/i, "\$1\$2"). Within the function it is passed into parse; assigned or updated later in the function.

function publishAuthCacheExpiresAt(cache)

Role: Read publishAuthCacheExpiresAt first as a named idea, not as syntax. publishAuthCacheExpiresAt belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:


```
function publishAuthCacheExpiresAt(cache) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `cache` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `publishAuthCacheExpiresAt`

```
function publishAuthCacheExpiresAt(cache) {  
  const explicitExpiresAt = parsePublishAuthTimestamp(cache?.expiresAt || "");  
  if (Number.isFinite(explicitExpiresAt)) return new Date(explicitExpiresAt).toISOString();  
  const checkedAt = parsePublishAuthTimestamp(cache?.checkedAt || "");  
  if (!Number.isFinite(checkedAt)) return "";  
  return new Date(checkedAt + publishAuthCacheTtlMs).toISOString();  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with `parsePublishAuthTimestamp`. Those calls show that `publishAuthCacheExpiresAt` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `publishAuthCacheExpiresAt`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The local objects and variables inside `publishAuthCacheExpiresAt` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

explicitExpiresAt: `explicitExpiresAt` stores a computed value for temporary calculation. It is initialized from `parsePublishAuthTimestamp(cache?.expiresAt || "")`. Within the function it is passed into `Date`; assigned or updated later in the function.

checkedAt: `checkedAt` stores a computed value for temporary calculation. It is initialized from `parsePublishAuthTimestamp(cache?.checkedAt || "")`. Within the function it is passed into `Date`, `parsePublishAuthTimestamp`; assigned or updated later in the function.

`function publishAuthCachelsFresh(cache, access)`

Role: To understand `publishAuthCachelsFresh`, start with the problem it owns. `publishAuthCachelsFresh` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function publishAuthCacheIsFresh(cache, access) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `cache` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for publishAuthCachelsFresh

```
function publishAuthCacheIsFresh(cache, access) {  
  if (!cache || !access) return false;  
  if (cache.remote !== access.remote || cache.branch !== access.branch || cache.repository !== access.repository)  
    return false;  
  if (cache.scope !== publishAuthCacheScope()) return false;  
  const expiresAt = Date.parse(publishAuthCacheExpiresAt(cache));  
  if (!Number.isFinite(expiresAt)) return false;  
  return Date.now() < expiresAt;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with publishAuthCacheScope, publishAuthCacheExpiresAt. Those calls show that publishAuthCachelsFresh is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without publishAuthCachelsFresh, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside publishAuthCachelsFresh reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

expiresAt: expiresAt stores a computed value for temporary calculation. It is initialized from Date.parse(publishAuthCacheExpiresAt(cache)). Within the function it is assigned or updated later in the function.

async function assertPublishAccess(options = {})

Role: The best entry point for assertPublishAccess is its responsibility in the surrounding workflow. assertPublishAccess is the gatekeeper that decides whether the current machine and target are allowed to change the website. It checks the configured publishing target, compares cached authorization against the current remote and branch, respects the once-per-day authentication window, and can force a fresh verification when needed. It returns the access object that publishing functions use as proof. Without this function, Apply to site would either ask for GitHub authorization too often or push too freely. The cache logic is what makes the app usable day to day while still preventing an unauthenticated installation from silently modifying Maurice's website.

Function entry point:

```
async function assertPublishAccess(options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assertPublishAccess

```
async function assertPublishAccess(options = {}) {
  let insideWorkTree = "";
  try {
    insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
  } catch (error) {
    throw publishAccessError(
      "This workspace is not connected to a Git repository.",
      gitFailureText(error),
      { repository: defaultSiteRepository }
    );
  }

  if (insideWorkTree !== "true") {
    throw publishAccessError(
      "This workspace is local-only and cannot publish yet.",
      "Clone or associate a GitHub Pages/static-site repository before applying to site.",
      { repository: defaultSiteRepository }
    );
  }

  let remote = "";
  let branch = "";
  try {
    remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
    branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  } catch (error) {
    throw publishAccessError(
      "A publish remote or branch is missing.",
      gitFailureText(error),
      { repository: defaultSiteRepository }
    );
  }

  if (!remote || !branch) {
    throw publishAccessError(
      "A publish remote or branch is missing.",
      "Set the origin remote and use a named branch before applying to site.",
      { remote: remoteUrlForDisplay(remote), branch }
    );
  }

  if (options.requireCompatible !== false && !await workspaceHasCompatibleSiteFiles()) {
    throw publishAccessError(
      "This repository does not look like a compatible static portfolio website.",
      "The workspace must include index.html, styles.css, and script.js before it can be used as a publish target.",
      { remote: remoteUrlForDisplay(remote), branch }
    );
  }

  const parsedRemote = parseGitHubRemote(remote);
  const repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : remoteUrlForDisplay(remote);

  const access = {
    branch,
    remote: remoteUrlForDisplay(remote),
    repository,
    authorizationChecked: false
  };

  const cached = await readPublishAuthCache();
  if (!options.force && publishAuthCacheIsFresh(cached, access)) {
    return {
      ...access,
      authorizationCached: true,
      authorizationChecked: true,
      checkedAt: cached.checkedAt,
      expiresAt: publishAuthCacheExpiresAt(cached),
      extendedTrust: Boolean(cached.extendedTrust),
      successCountLastWeek: cached.successCountLastWeek || 0,
      trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1)
    };
  }

  const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
```

```

const localHead = await ensurePublishHeadForWriteCheck();
const writeCheck = await verifyPublishWriteAccess(access);

const authCache = await writePublishAuthCache(access);
return {
  ...access,
  remoteSync,
  localHead,
  writeCheck,
  authorizationChecked: true,
  authorizationCached: false,
  checkedAt: authCache.checkedAt,
  expiresAt: authCache.expiresAt,
  extendedTrust: Boolean(authCache.extendedTrust),
  successCountLastWeek: authCache.successCountLastWeek || 0,
  trustDays: authCache.trustDays || 1
};
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `runPublishGit`, `publishAccessError`, `gitFailureText`, `remoteUrlForDisplay`, `workspaceHasCompatibleSiteFiles`, `parseGitHubRemote`, `readPublishAuthCache`, `publishAuthCachesFresh`. Those calls show that `assertPublishAccess` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assertPublishAccess`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of `assertPublishAccess` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

insideWorkTree: `insideWorkTree` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is assigned or updated later in the function.

remote: `remote` stores a string value for network or routing value. It is initialized from `""`. Within the function it is passed into `parseGitHubRemote`, `publishAccessError`, `remoteUrlForDisplay`, `runPublishGit`; assigned or updated later in the function.

branch: `branch` stores a string value for Git publishing and authorization state. It is initialized from `""`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `publishAccessError`, `runPublishGit`, `synchronizePublishBranchFromRemote`; assigned or updated later in the function.

parsedRemote: `parsedRemote` stores a computed value for network or routing value. It is initialized from `parseGitHubRemote(remote)`. Within the function it is accesses properties/methods `owner`, `repo`; assigned or updated later in the function.

repository: `repository` stores a computed value for temporary calculation. It is initialized from `parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : remoteUrlForDisplay(remote)`. Within the function it is passed into `publishAccessError`; assigned or updated later in the function.

access: `access` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into `synchronizePublishBranchFromRemote`, `verifyPublishWriteAccess`, `writePublishAuthCache`; assigned or updated later in the function.

cached: `cached` stores the completed result of an awaited operation. It is initialized from `await readPublishAuthCache()`. Within the function it is accesses properties/methods `checkedAt`, `extendedTrust`,

successCountLastWeek, trustDays; passed into Boolean, publishAuthCacheExpiresAt; assigned or updated later in the function.

remoteSync: remoteSync stores the completed result of an awaited operation. It is initialized from await synchronizePublishBranchFromRemote(branch, access). Within the function it is assigned or updated later in the function.

localHead: localHead stores the completed result of an awaited operation. It is initialized from await ensurePublishHeadForWriteCheck(). Within the function it is assigned or updated later in the function.

writeCheck: writeCheck stores the completed result of an awaited operation. It is initialized from await verifyPublishWriteAccess(access). Within the function it is assigned or updated later in the function.

authCache: authCache stores the completed result of an awaited operation. It is initialized from await writePublishAuthCache(access). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it accesses properties/methods checkedAt, expiresAt, extendedTrust, successCountLastWeek, trustDays; passed into Boolean; assigned or updated later in the function.

async function syncFromPublishTarget()

Role: The best entry point for syncFromPublishTarget is its responsibility in the surrounding workflow. syncFromPublishTarget is the import path. After a target is authenticated, this function pulls or fetches the latest public portfolio files from the selected repository and copies compatible website assets back into the local builder workspace. It is what lets a fresh machine reconstruct the current portfolio state from the website repository. This function is deliberately separate from authentication. Authenticate target proves access. Load from target reads the target. Keeping those concepts separate makes the UI clearer and reduces accidental overwrites.

Function entry point:

```
async function syncFromPublishTarget() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for syncFromPublishTarget

```
async function syncFromPublishTarget() {
  const publishAccess = await assertPublishAccess({ requireCompatible: false });
  const branch = publishAccess.branch || "main";
  const remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
  const importRoot = await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"));
  const cloneRoot = path.join(importRoot, "target");
  const importPaths = [
    "projects.json",
    "assets",
    "docs",
    "Backgrounds",
    "CNAME",
    ".nojekyll",
    "robots.txt",
    "sitemap.xml"
  ];

  try {
    await runGit(["clone", "--depth", "1", "--branch", branch, remote, cloneRoot], {
      cwd: importRoot,
      timeout: 3 * 60 * 1000
    });

    const availablePaths = [];
    for (const importPath of importPaths) {
      try {
        await fsAccess(path.join(cloneRoot, importPath));
        availablePaths.push(importPath);
      } catch {

```

```

    // Optional path is absent in the selected site repository.
  }
}

if (!availablePaths.includes("projects.json")) {
  throw publishAccessError(
    "The selected repository does not contain a projects.json portfolio catalog.",
    "The target repository must contain a compatible OMB Portfolio Builder catalog before it can be imported.",
    publishAccess
  );
}

const backupRoot = resolveInsideRoot(
  ".omb-backups",
  `load-target-${new Date().toISOString().replace(/[:.]/g, "-")}`
);
await mkdir(backupRoot, { recursive: true });

for (const importPath of availablePaths) {
  const sourcePath = path.join(cloneRoot, importPath);
  const targetPath = resolveInsideRoot(importPath);
  try {
    await fsAccess(targetPath);
    await cp(targetPath, path.join(backupRoot, importPath), { recursive: true, force: true });
    await rm(targetPath, { recursive: true, force: true });
  } catch {
    // Nothing Local to back up for this path.
  }
  await cp(sourcePath, targetPath, { recursive: true, force: true });
}

const importedCatalog = await readFile(path.join(cloneRoot, "projects.json"), "utf8");
await writeFile(draftPath, importedCatalog, "utf8");
const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: false });

return {
  ...publishAccess,
  imported: availablePaths,
  portfolioSync,
  backup: path.relative(root, backupRoot),
  draftUpdated: true
};
} finally {
  await rm(importRoot, { recursive: true, force: true });
}
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `assertPublishAccess`, `runPublishGit`, `runGit`, `publishAccessError`, `resolveInsideRoot`, `syncPortfolioPublishFiles`. Those calls show that `syncFromPublishTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `syncFromPublishTarget`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: The easiest way to follow the body of `syncFromPublishTarget` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

publishAccess: `publishAccess` stores the completed result of an awaited operation. It is initialized from `await assertPublishAccess({ requireCompatible: false })`. It affects publishing, where mistakes can change the

public website or expose the wrong files. Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

branch: branch stores a computed value for Git publishing and authorization state. It is initialized from publishAccess.branch || "main". It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into runGit; assigned or updated later in the function.

remote: remote stores a function or callback value for network or routing value. It is initialized from (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim(). Within the function it is passed into runGit, runPublishGit; assigned or updated later in the function.

importRoot: importRoot stores a resolved filesystem or route value. It is initialized from await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-")). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into join, rm, runGit; assigned or updated later in the function.

cloneRoot: cloneRoot stores a resolved filesystem or route value. It is initialized from path.join(importRoot, "target"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into fsAccess, join, readFile, runGit; assigned or updated later in the function.

importPaths: importPaths stores a list for filesystem location. It is initialized from []. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is assigned or updated later in the function.

availablePaths: availablePaths stores a list for filesystem location. It is initialized from []. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is mutated as a collection or DOM container; accesses properties/methods includes, push; assigned or updated later in the function.

importPath: importPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, fsAccess, join, push, resolveInsideRoot.

backupRoot: backupRoot stores a computed value for filesystem location. It is initialized from resolveInsideRoot(). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, mkdir, relative; assigned or updated later in the function.

importPath: importPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, fsAccess, join, push, resolveInsideRoot.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from path.join(cloneRoot, importPath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp; assigned or updated later in the function.

targetPath: targetPath stores a resolved filesystem or route value. It is initialized from resolveInsideRoot(importPath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, fsAccess, rm; assigned or updated later in the function.

importedCatalog: importedCatalog stores a resolved filesystem or route value. It is initialized from await readFile(path.join(cloneRoot, "projects.json"), "utf8"). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into writeFile; assigned or updated later in the function.

portfolioSync: portfolioSync stores the completed result of an awaited operation. It is initialized from await syncPortfolioPublishFiles({ removeMissing: false }). Within the function it is assigned or updated later in the function.

async function installGitForWindows()

Role: To understand installGitForWindows, start with the problem it owns. installGitForWindows installs or prepares external tooling. It belongs in the backend because installation touches the machine, not just the browser page.

Function entry point:

```
async function installGitForWindows() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for installGitForWindows

```
async function installGitForWindows() {  
  const winget = await runOptionalCommand("winget", ["--version"]);  
  const downloadUrl = "https://git-scm.com/download/win";  
  if (!winget.ok) {  
    return {  
      launched: false,  
      downloadUrl,  
      message: "winget was not found. Open the Git for Windows download page and install Git with Git Credential  
Manager enabled."  
    };  
  }  
  
  const child = spawn("winget", ["install", "--id", "Git.Git", "-e", "--source", "winget"], {  
    detached: true,  
    shell: true,  
    stdio: "ignore",  
    windowsHide: false  
  });  
  child.unref();  
  return {  
    launched: true,  
    downloadUrl,  
    message: "The Git for Windows installer was launched through winget. Follow the installer prompts, then reopen  
the builder."  
  };  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Next, trace the helper calls that surround the function. It works with runOptionalCommand. Those calls show that installGitForWindows is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without installGitForWindows, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside installGitForWindows reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

winget: winget stores the completed result of an awaited operation. It is initialized from await runOptionalCommand("winget", ["--version"]). Within the function it is accesses properties/methods ok; passed into runOptionalCommand, spawn; assigned or updated later in the function.

downloadUrl: downloadUrl stores a string value for network or routing value. It is initialized from "https://git-scm.com/download/win". Within the function it is assigned or updated later in the function.

child: child stores a computed value for temporary calculation. It is initialized from spawn("winget", ["install", "--id", "Git.Git", "-e", "--source", "winget"], {). Within the function it is accesses properties/methods unref; assigned or updated later in the function.

async function syncPortfolioPublishFiles(options = {})

Role: To understand syncPortfolioPublishFiles, start with the problem it owns. syncPortfolioPublishFiles reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

Function entry point:

```
async function syncPortfolioPublishFiles(options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for syncPortfolioPublishFiles

```
async function syncPortfolioPublishFiles(options = {}) {
  if (samePath(root, portfolioRoot)) {
    return { copied: [], skipped: [], workspace: portfolioRoot, separatedWorkspace: false };
  }

  const { removeMissing = false } = options;
  const copied = [];
  const skipped = [];

  await mkdir(portfolioRoot, { recursive: true });

  for (const relativePath of publishPaths) {
    const sourcePath = resolveInsideRoot(relativePath);
    const targetPath = resolveInsidePortfolioRoot(relativePath);
    if (await pathExists(sourcePath)) {
      await rm(targetPath, { recursive: true, force: true });
      await mkdir(path.dirname(targetPath), { recursive: true });
      await cp(sourcePath, targetPath, { recursive: true, force: true });
      copied.push(relativePath);
    } else if (removeMissing) {
      await rm(targetPath, { recursive: true, force: true });
      skipped.push(relativePath);
    } else {
      skipped.push(relativePath);
    }
  }

  return {
    copied,
    skipped,
    workspace: portfolioRoot,
    separatedWorkspace: true
  };
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a

structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: creates folders before writing files so missing directories do not crash the workflow.

Collaborators: Next, trace the helper calls that surround the function. It works with `samePath`, `resolveInsideRoot`, `resolveInsidePortfolioRoot`, `pathExists`. Those calls show that `syncPortfolioPublishFiles` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `syncPortfolioPublishFiles`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside `syncPortfolioPublishFiles` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

copied: `copied` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

skipped: `skipped` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

relativePath: `relativePath` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`, `resolveInsidePortfolioRoot`, `resolveInsideRoot`.

sourcePath: `sourcePath` stores a resolved filesystem or route value. It is initialized from `resolveInsideRoot(relativePath)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`; assigned or updated later in the function.

targetPath: `targetPath` stores a resolved filesystem or route value. It is initialized from `resolveInsidePortfolioRoot(relativePath)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`, `mkdir`, `rm`; assigned or updated later in the function.

async function publishSiteChanges(publishAccess = null)

Role: To understand `publishSiteChanges`, start with the problem it owns. `publishSiteChanges` is the final publishing operation behind `Apply to site`. It assumes authorization has been proven or supplied, synchronizes publishable files into the portfolio mirror, checks Git status, stages only approved paths, creates a commit when there are changes, and pushes to the selected branch. It is `async` because Git commands and file synchronization are external operations that take time and can fail. This function is important because publishing is the one place where local draft work becomes public website content. It must therefore be strict. If it staged arbitrary files or pushed without checking state, builder internals, private drafts, or stale branches could leak to the website. The function works with `assertPublishAccess`, `syncPortfolioPublishFiles`, `getGitStatus`, `runPublishGit`, and `stageablePublishPaths`.

Function entry point:

```
async function publishSiteChanges(publishAccess = null) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `publishAccess` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for publishSiteChanges

```
async function publishSiteChanges(publishAccess = null) {
  const access = publishAccess || await assertPublishAccess();
  const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
  const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: true });
  await bumpPublishedAssetVersions(portfolioRoot);

  const stageablePaths = await stageablePublishPaths(publishPaths);
  if (!stageablePaths.length) {
    throw new Error("No compatible site files were available to publish.");
  }

  await runPublishGit(["add", "-A", "--", ...stageablePaths]);
  const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
  const hasChanges = status.stdout.trim().length > 0;

  let commit = null;
  if (hasChanges) {
    const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}`;
    commit = await runPublishGit(["commit", "-m", message]);
  }

  const pushArgs = branch ? ["push", "origin", branch] : ["push"];
  const push = await runPublishGit(pushArgs);

  return {
    ...access,
    portfolioWorkspace: portfolioRoot,
    remoteSync,
    portfolioSync,
    branch: branch || "current branch",
    committed: hasChanges,
    commitOutput: commit?.stdout || commit?.stderr || "",
    pushed: true,
    pushOutput: push.stdout || push.stderr || ""
  };
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Collaborators: Next, trace the helper calls that surround the function. It works with `assertPublishAccess`, `runPublishGit`, `synchronizePublishBranchFromRemote`, `syncPortfolioPublishFiles`, `bumpPublishedAssetVersions`, `stageablePublishPaths`. Those calls show that `publishSiteChanges` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `publishSiteChanges`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Local values: After the main flow, the working values inside `publishSiteChanges` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

access: access stores the completed result of an awaited operation. It is initialized from `publishAccess || await assertPublishAccess()`. Within the function it is accesses properties/methods `branch`; passed into `synchronizePublishBranchFromRemote`; assigned or updated later in the function.

branch: branch stores the completed result of an awaited operation. It is initialized from `access.branch || (await runPublishGit(["branch", "--show-current"]).stdout.trim())`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runPublishGit`, `synchronizePublishBranchFromRemote`; assigned or updated later in the function.

remoteSync: remoteSync stores the completed result of an awaited operation. It is initialized from `await synchronizePublishBranchFromRemote(branch, access)`. Within the function it is assigned or updated later in the function.

portfolioSync: portfolioSync stores the completed result of an awaited operation. It is initialized from `await syncPortfolioPublishFiles({ removeMissing: true })`. Within the function it is assigned or updated later in the function.

stageablePaths: stageablePaths stores a resolved filesystem or route value. It is initialized from `await stageablePublishPaths(publishPaths)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `length`; passed into `runPublishGit`; assigned or updated later in the function.

status: status stores a resolved filesystem or route value. It is initialized from `await runPublishGit(["status", "--porcelain", "--", ...stageablePaths])`. Within the function it is accesses properties/methods `stdout`; passed into `runPublishGit`; assigned or updated later in the function.

hasChanges: hasChanges stores a computed value for temporary calculation. It is initialized from `status.stdout.trim().length > 0`. Within the function it is assigned or updated later in the function.

commit: commit stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

message: message stores a string value for temporary calculation. It is initialized from ``Update portfolio site ${new Date().toISOString().slice(0, 10)}``. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

pushArgs: pushArgs stores a computed value for temporary calculation. It is initialized from `branch ? ["push", "origin", branch] : ["push"]`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

push: push stores the completed result of an awaited operation. It is initialized from `await runPublishGit(pushArgs)`. Within the function it is accesses properties/methods `stderr`, `stdout`; assigned or updated later in the function.

Application update and reporting

The functions in this group all support application update and reporting. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function compareVersions(left = "", right = "")

Role: Begin with the role of `compareVersions`. `compareVersions` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function compareVersions(left = "", right = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `left` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more

predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. right is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for compareVersions

```
function compareVersions(left = "", right = "") {
  const leftParts = String(left || "0").split(/[-.]/).map((part) => Number.parseInt(part, 10) || 0);
  const rightParts = String(right || "0").split(/[-.]/).map((part) => Number.parseInt(part, 10) || 0);
  const length = Math.max(leftParts.length, rightParts.length);
  for (let index = 0; index < length; index += 1) {
    const delta = (leftParts[index] || 0) - (rightParts[index] || 0);
    if (delta !== 0) return delta;
  }
  return 0;
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means compareVersions performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without compareVersions, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside compareVersions explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

leftParts: leftParts stores a function or callback value for lookup collection. It is initialized from String(left || "0").split(/[-.]/).map((part) => Number.parseInt(part, 10) || 0). Within the function it is accesses properties/methods length; passed into max; assigned or updated later in the function.

rightParts: rightParts stores a function or callback value for lookup collection. It is initialized from String(right || "0").split(/[-.]/).map((part) => Number.parseInt(part, 10) || 0). Within the function it is accesses properties/methods length; passed into max; assigned or updated later in the function.

length: length stores a computed value for temporary calculation. It is initialized from Math.max(leftParts.length, rightParts.length). Within the function it is passed into max; assigned or updated later in the function.

index: index stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is assigned or updated later in the function.

delta: delta stores a function or callback value for temporary calculation. It is initialized from (leftParts[index] || 0) - (rightParts[index] || 0). Within the function it is returned to the caller; assigned or updated later in the function.

async function getUpdateInfo()

Role: The best entry point for getUpdateInfo is its responsibility in the surrounding workflow. getUpdateInfo belongs to application update and reporting. This function supports release checks, installer handoff, version

comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function getUpdateInfo() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for getUpdateInfo

```
async function getUpdateInfo() {
  const currentVersion = process.env.OMB_APP_VERSION || packageJson.version || "0.0.0";
  const owner = "otienomaurice";
  const repo = "omb-portfolio-builder";
  try {
    const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
      headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
    });
    if (!response.ok) throw new Error(`GitHub returned ${response.status}.`);
    const release = await response.json();
    const latestVersion = String(release.tag_name || "").replace(/^builder-v/i, "");
    const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
    const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
    const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
    return {
      ok: true,
      currentVersion,
      latestVersion,
      updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
      updateBlocked,
      blockedReason: updateBlocked
        ? `Builder ${latestVersion} is skipped because its installer can hang while running the old uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
        : "",
      releaseUrl: release.html_url || "",
      installerUrl: installer?.browser_download_url || "",
      installerName: installer?.name || "",
      portableUrl: portable?.browser_download_url || ""
    };
  } catch (error) {
    return {
      ok: false,
      currentVersion,
      latestVersion: currentVersion,
      updateAvailable: false,
      error: error.message || "Could not check for updates."
    };
  }
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `compareVersions`. Those calls show that `getUpdateInfo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `getUpdateInfo`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `getUpdateInfo` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

currentVersion: currentVersion stores a environment-derived value for temporary calculation. It is initialized from process.env.OMB_APP_VERSION || packageJson.version || "0.0.0". Within the function it is passed into compareVersions; assigned or updated later in the function.

owner: owner stores a string value for temporary calculation. It is initialized from "otienomaurice". Within the function it is passed into fetch; assigned or updated later in the function.

repo: repo stores a string value for temporary calculation. It is initialized from "omb-portfolio-builder". Within the function it is passed into fetch; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from await fetch(`https://api.github.com/repos/\${owner}/\${repo}/releases/latest`, {). Within the function it is accesses properties/methods json, ok, status; passed into Error; assigned or updated later in the function.

release: release stores the completed result of an awaited operation. It is initialized from await response.json(). Within the function it is accesses properties/methods assets, html_url, tag_name; passed into String; assigned or updated later in the function.

latestVersion: latestVersion stores a computed value for temporary calculation. It is initialized from String(release.tag_name || "").replace(/^builder-v/i, ""). Within the function it is passed into compareVersions, has; assigned or updated later in the function.

installer: installer stores a function or callback value for temporary calculation. It is initialized from (release.assets || []).find((asset) => /Setup-.*\.exe\$/i.test(asset.name)). Within the function it is assigned or updated later in the function.

portable: portable stores a function or callback value for temporary calculation. It is initialized from (release.assets || []).find((asset) => /Portable-.*\.exe\$/i.test(asset.name)). Within the function it is assigned or updated later in the function.

updateBlocked: updateBlocked stores a computed value for temporary calculation. It is initialized from blockedAppUpdateVersions.has(latestVersion). Within the function it is assigned or updated later in the function.

async function getBuilderReleaseDownloadReport()

Role: Begin with the role of getBuilderReleaseDownloadReport. getBuilderReleaseDownloadReport belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function getBuilderReleaseDownloadReport() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for getBuilderReleaseDownloadReport

```
async function getBuilderReleaseDownloadReport() {
  const owner = "otienomaurice";
  const repo = "omb-portfolio-builder";
  const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
    headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
  });
  if (!response.ok) throw new Error(`GitHub returned ${response.status}.`);
  const release = await response.json();
  const assets = (release.assets || []).map((asset) => ({
    name: asset.name || "",
    downloadCount: Number(asset.download_count || 0),
    sizeBytes: Number(asset.size || 0),
    updatedAt: asset.updated_at || "",
    url: asset.browser_download_url || ""
  }));
  return {
```

```

    releaseName: release.name || release.tag_name || "latest",
    releaseTag: release.tag_name || "",
    releaseUrl: release.html_url || "",
    totalDownloads: assets.reduce((total, asset) => total + asset.downloadCount, 0),
    assets
  };
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `getBuilderReleaseDownloadReport` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `getBuilderReleaseDownloadReport`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `getBuilderReleaseDownloadReport` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

owner: owner stores a string value for temporary calculation. It is initialized from "otienomaurice". Within the function it is passed into fetch; assigned or updated later in the function.

repo: repo stores a string value for temporary calculation. It is initialized from "omb-portfolio-builder". Within the function it is passed into fetch; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch('https://api.github.com/repos/${owner}/${repo}/releases/latest', {`. Within the function it is accesses properties/methods `json, ok, status`; passed into `Error`; assigned or updated later in the function.

release: release stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it is accesses properties/methods `assets, html_url, name, tag_name`; assigned or updated later in the function.

assets: assets stores a function or callback value for lookup collection. It is initialized from `(release.assets || []).map((asset) => ({`. Within the function it is iterated or transformed as a collection; accesses properties/methods `reduce`; assigned or updated later in the function.

async function getSecurityReport()

Role: Begin with the role of `getSecurityReport`. `getSecurityReport` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function getSecurityReport() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for getSecurityReport

```

async function getSecurityReport() {
  const [downloadResult, authCacheResult, targetResult] = await Promise.allSettled([
    getBuilderReleaseDownloadReport(),
    readPublishAuthCache(),

```

```

    getPublishTargetInfo()
  ]);
  const authCache = authCacheResult.status === "fulfilled" ? authCacheResult.value : null;
  const target = targetResult.status === "fulfilled" ? targetResult.value : null;
  return {
    ok: true,
    generatedAt: new Date().toISOString(),
    appDownloads: downloadResult.status === "fulfilled"
      ? { ok: true, ...downloadResult.value }
      : { ok: false, error: downloadResult.reason?.message || "GitHub release download counts were not available."
    },
    publishingAuth: {
      targetRepository: target?.repository || "",
      branch: target?.branch || "",
      authenticated: Boolean(authCache && publishAuthCacheIsFresh(authCache, {
        remote: target?.remote || "",
        repository: target?.repository || "",
        branch: target?.branch || ""
      })),
      expiresAt: publishAuthCacheExpiresAt(authCache) || "",
      trustDays: Number(authCache?.trustDays || 0),
      successCountLastWeek: Number(authCache?.successCountLastWeek || 0)
    },
    websiteAccess: {
      available: false,
      summary: "Visitor counts and visitor IP addresses are not available from a static GitHub Pages site or GitHub
release download counts.",
      recommendedSetup: [
        "Proxy the custom domain through Cloudflare instead of DNS-only.",
        "Enable Cloudflare Web Analytics for aggregate page-view counts.",
        "Use Cloudflare Logpush, Analytics Engine, or a Worker-backed endpoint if you need IP-level security logs.",
        "Publish a privacy notice before collecting raw IP addresses or persistent identifiers."
      ]
    },
    securityControls: [
      "Local builder write APIs are restricted to local requests.",
      "Publishing target details are only returned to the local machine.",
      "GitHub publishing authorization is cached for one day by default, or longer after repeated successful
authorizations.",
      "The desktop app runs with Electron nodeIntegration disabled, contextIsolation enabled, and sandbox enabled.",
      "Deployable website security headers and security.txt metadata are included in the publishable site files."
    ]
  };
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `getBuilderReleaseDownloadReport`, `readPublishAuthCache`, `getPublishTargetInfo`, `publishAuthCacheIsFresh`, `publishAuthCacheExpiresAt`. Those calls show that `getSecurityReport` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `getSecurityReport`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `getSecurityReport` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

destructured [downloadResult, authCacheResult, targetResult]: destructured [downloadResult, authCacheResult, targetResult] stores the completed result of an awaited operation. It is initialized from `await Promise.allSettled([`. It affects publishing, where mistakes can change the public website or expose the wrong files.

authCache: `authCache` stores a computed value for runtime state or cache. It is initialized from `authCacheResult.status === "fulfilled" ? authCacheResult.value : null`. It affects publishing, where mistakes

can change the public website or expose the wrong files. Within the function it is passed into Boolean, Number, publishAuthCacheExpiresAt; assigned or updated later in the function.

target: target stores a computed value for temporary calculation. It is initialized from `targetResult.status === "fulfilled" ? targetResult.value : null`. Within the function it is passed into Boolean; assigned or updated later in the function.

function safeUpdateFileSegment(value = "")

Role: To understand `safeUpdateFileSegment`, start with the problem it owns. `safeUpdateFileSegment` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

Function entry point:

```
function safeUpdateFileSegment(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for safeUpdateFileSegment

```
function safeUpdateFileSegment(value = "") {  
  return String(value || "")  
    .replace(/^[^a-z0-9._-]+/gi, "-")  
    .replace(/^-+|-+$/g, "")  
    .slice(0, 80) || "latest";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `safeUpdateFileSegment` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `safeUpdateFileSegment`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `safeUpdateFileSegment`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function downloadAndLaunchAppUpdate()

Role: Read `downloadAndLaunchAppUpdate` first as a named idea, not as syntax.

`downloadAndLaunchAppUpdate` is the updater handoff. It checks release information, downloads the installer, writes a PowerShell script that can wait for the current process to exit, launches the installer or updater quietly, and schedules the current app to close. It must be `async` because it performs network download, file writes, and process launches. The function is needed because a running Electron app cannot safely replace all of its own files while those files are in use. The handoff script becomes a temporary helper that finishes the update after the current process gets out of the way.

Function entry point:

```
async function downloadAndLaunchAppUpdate() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for downloadAndLaunchAppUpdate

```
async function downloadAndLaunchAppUpdate() {
  const update = await getUpdateInfo();
  if (!update.ok) throw new Error(update.error || "Could not check for updates.");
  if (update.updateBlocked) {
    throw new Error(update.blockedReason || `Builder ${update.latestVersion} is temporarily blocked from automatic updates.`);
  }
  if (!update.updateAvailable) {
    throw new Error(`OMB Portfolio Builder ${update.currentVersion} || "" is already up to date.`);
  }
  if (!update.installerUrl) {
    throw new Error("The latest GitHub Release does not include a Windows setup installer.");
  }
  if (process.platform !== "win32") {
    throw new Error("Automatic installer updates are currently available only on Windows.");
  }

  const updateRoot = path.join(os.tmpdir(), "omb-portfolio-builder-updates");
  await mkdir(updateRoot, { recursive: true });
  const installerName = update.installerName && /\.exe$/i.test(update.installerName)
    ? update.installerName
    : `OMB-Portfolio-Builder-Setup-${safeUpdateFileSegment(update.latestVersion)}-x64.exe`;
  const installerPath = path.join(updateRoot, installerName);

  const response = await fetch(update.installerUrl, {
    headers: { "Accept": "application/octet-stream", "User-Agent": "OMB-Portfolio-Builder" }
  });
  if (!response.ok) throw new Error(`GitHub returned ${response.status} while downloading the installer.`);
  const installerBuffer = Buffer.from(await response.arrayBuffer());
  if (installerBuffer.length < 1024 * 1024) {
    throw new Error("The downloaded installer was unexpectedly small, so the update was stopped.");
  }
  await writeFile(installerPath, installerBuffer);

  const updateLogPath = path.join(updateRoot, `update-${safeUpdateFileSegment(update.latestVersion)}.log`);
  const currentExecutable = /OMB Portfolio Builder\.exe$/i.test(process.execPath || "") ? process.execPath : "";
  const currentInstallDirectory = currentExecutable ? path.dirname(currentExecutable) : "";
  const localAppDataExecutable = process.env.LOCALAPPDATA
    ? path.join(process.env.LOCALAPPDATA, "Programs", "OMB Portfolio Builder", "OMB Portfolio Builder.exe")
    : "";
  const legacyManagedApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder", "OMB Portfolio Builder.exe");
  const legacyFlatApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder.exe");
  const relaunchCandidates = Array.from(new Set([
    localAppDataExecutable,
    currentExecutable,
    currentInstallDirectory ? path.join(currentInstallDirectory, "OMB Portfolio Builder.exe") : "",
    legacyFlatApplicationExecutable,
    legacyManagedApplicationExecutable,
    process.env.ProgramFiles ? path.join(process.env.ProgramFiles, "OMB Portfolio Builder", "OMB Portfolio Builder.exe") : "",
    process.env["ProgramFiles(x86)"] ? path.join(process.env["ProgramFiles(x86)"], "OMB Portfolio Builder", "OMB Portfolio Builder.exe") : ""
  ]).filter(Boolean));
  const relaunchCandidatesPs = `@(${relaunchCandidates.map(powershellSingleQuoted).join(", ")}`;
  const launcherPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.ps1`);
  const launcherCommandPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.cmd`);
  const launcherScript = [
    "$ErrorActionPreference = 'Continue'",
    "$parentPid = ${process.pid}",
    "$installer = ${powershellSingleQuoted(installerPath)}",
    "$log = ${powershellSingleQuoted(updateLogPath)}",
    "$candidates = ${relaunchCandidatesPs}"
  ]
```

```

"function Write-UpdateLog([string]$Message) {"
"  try { Add-Content -LiteralPath $log -Value ((Get-Date).ToString('s') + ' ' + $Message) } catch {"
"}",
"function Wait-ForExecutable([string[]]$Paths, [int]$Seconds) {"
"  $deadline = (Get-Date).AddSeconds($Seconds)",
"  do {"
"    foreach ($candidate in $Paths) {"
"      if ($candidate -and (Test-Path -LiteralPath $candidate)) { return $candidate }",
"    }",
"    Start-Sleep -Seconds 1",
"  } while ((Get-Date) -lt $deadline)",
"  return $null",
"}",
"function Stop-BuilderProcesses([int]$ProcessId, [string[]]$Paths) {"
"  try {"
"    $normalizedPaths = @($Paths | Where-Object { $_ } | ForEach-Object {"
"      try { [System.IO.Path]::GetFullPath($_).ToLowerInvariant() } catch { $_.ToLowerInvariant() }",
"    })",
"    $processes = Get-CimInstance Win32_Process | Where-Object {"
"      $_.ProcessId -eq $ProcessId -or",
"      $_.Name -eq 'OMB Portfolio Builder.exe' -or",
"      ($_.ExecutablePath -and ($normalizedPaths -contains",
([System.IO.Path]::GetFullPath($_.ExecutablePath).ToLowerInvariant()))",
"    } | Sort-Object ProcessId -Unique",
"    foreach ($process in $processes) {"
"      try {"
"        Write-UpdateLog ('Stopping previous builder process PID ' + $process.ProcessId + ' at ' +
$process.ExecutablePath)",
"        Stop-Process -Id $process.ProcessId -Force -ErrorAction SilentlyContinue",
"      } catch { Write-UpdateLog ('Could not stop PID ' + $process.ProcessId + ': ' + $_.Exception.Message) }",
"    }",
"    Start-Sleep -Seconds 2",
"  } catch { Write-UpdateLog ('Could not stop previous builder processes: ' + $_.Exception.Message) }",
"}",
"function Run-Installer([switch]$Elevated) {"
"  $process = $null",
"  if ($Elevated) {"
"    Write-UpdateLog 'Starting installer with Windows elevation prompt.'",
"    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -Verb RunAs -PassThru",
"  } else {"
"    Write-UpdateLog 'Starting installer silently.'",
"    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -WindowStyle Hidden -PassThru",
"  }",
"  if (-not $process) { return 0 }",
"  if (-not $process.WaitForExit(900000)) {"
"    Write-UpdateLog ('Installer timed out after 15 minutes. PID: ' + $process.Id)",
"    try { Stop-Process -Id $process.Id -Force -ErrorAction SilentlyContinue } catch { Write-UpdateLog ('Could
not stop timed-out installer: ' + $_.Exception.Message) }",
"    return 124",
"  }",
"  return $process.ExitCode",
"}",
"Write-UpdateLog 'Update launcher started.'",
"Start-Sleep -Seconds 3",
"try { Wait-Process -Id $parentPid -Timeout 8; Write-UpdateLog 'Previous app process exited.' } catch { Write-
UpdateLog ('Previous app wait finished: ' + $_.Exception.Message) }",
"Stop-BuilderProcesses $parentPid $candidates",
"Start-Sleep -Seconds 1",
"$installerProcess = $null",
"$installerExit = 1",
"try {"
"  Write-UpdateLog ('Starting installer: ' + $installer)",
"  $installerExit = Run-Installer",
"  Write-UpdateLog ('Installer exit code: ' + $installerExit)",
"} catch {"
"  Write-UpdateLog ('Installer launch failed: ' + $_.Exception.Message)",
"  exit 1",
"}",
"if ($installerExit -ne 0) {"
"  try {"
"    Write-UpdateLog 'Installer did not finish cleanly; retrying with Windows elevation prompt.'",
"    $installerExit = Run-Installer -Elevated",
"    Write-UpdateLog ('Elevated installer exit code: ' + $installerExit)",
"  } catch { Write-UpdateLog ('Elevated installer launch failed: ' + $_.Exception.Message) }",
"}",
"if ($installerExit -ne 0) { exit $installerExit }",
"$target = Wait-ForExecutable $candidates 90",
"if ($target) {"

```



```

    " try {",
    "   Start-Sleep -Seconds 2",
    "   Start-Process -FilePath $target -WorkingDirectory (Split-Path -Parent $target)",
    "   Write-UpdateLog ('Relaunched app: ' + $target)",
    "   exit 0",
    " } catch {",
    "   Write-UpdateLog ('Relaunch failed: ' + $_.Exception.Message)",
    "   exit 3",
    " }",
    "}",
    "}",
    "Write-UpdateLog 'No installed app executable was found after update.'",
    "exit 2"
].join("\r\n");
await writeFile(launcherPath, launcherScript, "utf8");
const launcherCommandScript = [
  "@echo off",
  `start "" /min powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File "${launcherPath}"`
].join("\r\n");
await writeFile(launcherCommandPath, launcherCommandScript, "utf8");

const child = spawn("cmd.exe", ["/d", "/c", launcherCommandPath], {
  detached: true,
  stdio: "ignore",
  windowsHide: true
});
child.unref();

setTimeout(() => {
  process.emit("omb-builder-update-started", { installerPath, launcherPath, updateLogPath });
}, 600);
setTimeout(() => {
  process.exit(0);
}, 3000);

return {
  ...update,
  installerPath,
  launcherPath,
  updateLogPath,
  launcherCommandPath,
  started: true
};
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work, calls a network resource, runs an external command, reads or writes files and uses a timer to avoid blocking or to wait for another process. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

Collaborators: The function also has to be read through the helpers it calls. It works with `getUpdateInfo`, `safeUpdateFileSegment`, `powershellSingleQuoted`. Those calls show that `downloadAndLaunchAppUpdate` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `downloadAndLaunchAppUpdate`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `downloadAndLaunchAppUpdate` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

update: update stores the completed result of an awaited operation. It is initialized from `await getUpdateInfo()`. Within the function it is accesses properties/methods `blockedReason`, `currentVersion`, `error`, `installerName`, `installerUrl`, `latestVersion`, `ok`, `updateAvailable`; passed into `Error`, `emit`, `fetch`, `join`, `safeUpdateFileSegment`, `test`; assigned or updated later in the function.

updateRoot: updateRoot stores a resolved filesystem or route value. It is initialized from `path.join(os.tmpdir(), "omb-portfolio-builder-updates")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `mkdir`; assigned or updated later in the function.

installerName: installerName stores a computed value for temporary calculation. It is initialized from `update.installerName && /\.exe$/i.test(update.installerName)`. Within the function it is passed into `join`, `test`; assigned or updated later in the function.

installerPath: installerPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, installerName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `emit`, `powershellSingleQuoted`, `writeFile`; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch(update.installerUrl, {`. Within the function it is accesses properties/methods `arrayBuffer`, `ok`, `status`; passed into `Error`, `from`; assigned or updated later in the function.

installerBuffer: installerBuffer stores the completed result of an awaited operation. It is initialized from `Buffer.from(await response.arrayBuffer())`. Within the function it is accesses properties/methods `length`; passed into `writeFile`; assigned or updated later in the function.

updateLogPath: updateLogPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, `update-${safeUpdateFileSegment(update.latestVersion)}.log`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `emit`, `powershellSingleQuoted`; assigned or updated later in the function.

currentExecutable: currentExecutable stores a resolved filesystem or route value. It is initialized from `/OMB Portfolio Builder\exe$/i.test(process.execPath || "") ? process.execPath : ""`. Within the function it is passed into `dirname`, `from`; assigned or updated later in the function.

currentInstallDirectory: currentInstallDirectory stores a resolved filesystem or route value. It is initialized from `currentExecutable ? path.dirname(currentExecutable) : ""`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `from`; assigned or updated later in the function.

localAppDataExecutable: localAppDataExecutable stores a environment-derived value for temporary calculation. It is initialized from `process.env.LOCALAPPDATA`. Within the function it is passed into `from`; assigned or updated later in the function.

legacyManagedApplicationExecutable: legacyManagedApplicationExecutable stores a resolved filesystem or route value. It is initialized from `path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder", "OMB Portfolio Builder.exe")`. Within the function it is assigned or updated later in the function.

legacyFlatApplicationExecutable: legacyFlatApplicationExecutable stores a resolved filesystem or route value. It is initialized from `path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder.exe")`. Within the function it is assigned or updated later in the function.

relaunchCandidates: relaunchCandidates stores a computed value for lookup collection. It is initialized from `Array.from(new Set([`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

relaunchCandidatesPs: relaunchCandidatesPs stores a string value for lookup collection. It is initialized from ``@($relaunchCandidates.map(powershellSingleQuoted).join(", "))``. Within the function it is assigned or updated later in the function.

launcherPath: launcherPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.ps1`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `emit`, `writeFile`; assigned or updated later in the function.

launcherCommandPath: launcherCommandPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.cmd`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `spawn`, `writeFile`; assigned or updated later in the function.

launcherScript: launcherScript stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `writeFile`; assigned or updated later in the function.

launcherCommandScript: launcherCommandScript stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `writeFile`; assigned or updated later in the function.

child: child stores a resolved filesystem or route value. It is initialized from `spawn("cmd.exe", ["/d", "/c", launcherCommandPath], {`. Within the function it is accesses properties/methods `unref`; assigned or updated later in the function.